



School of Artificial Intelligence
Jilin University

第五章 贪心方法

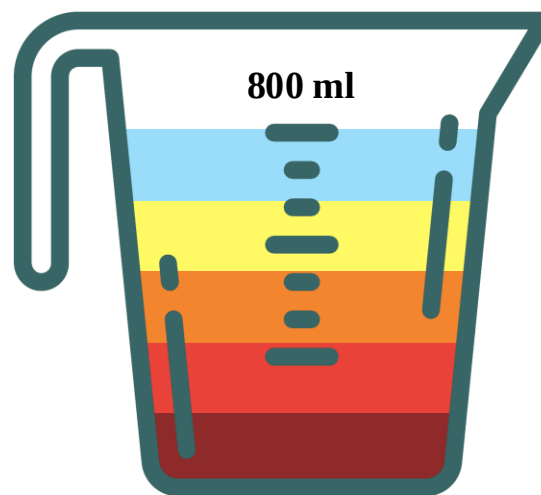
吉林大学人工智能学院

王元宏

背包问题

- 参赛者拥有容量为**800ml**的杯子，可任选不超过体积上限的饮料进行混合
- 调制饮品价格为各所使用饮料的价格之和，所得饮品价格之和最高者获胜

饮料	价格(元)	体积(ml)
 苏打水	60	600
 汽水	10	250
 橙汁	36	200
 苹果汁	16	100
 西瓜汁	45	300



思考：如何使调制的饮品价格最高？

- **最高性价比**优先

- 性价比 = 价格/体积
- 优先选择**高性价比**饮料全部装入，尽可能装满杯子

饮料	价格 (元)	体积 (ml)	性价比 (元/ml)
 苏打水	60	600	0.10
 汽水	10	250	0.04
 橙汁	36	200	0.18
 苹果汁	16	100	0.16
 西瓜汁	45	300	0.15

正确性证明

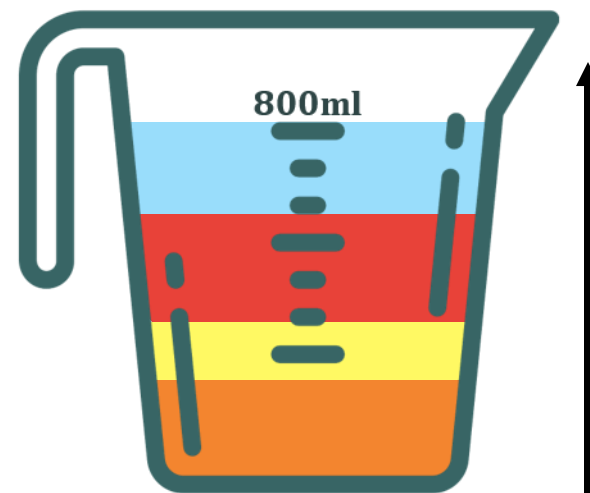
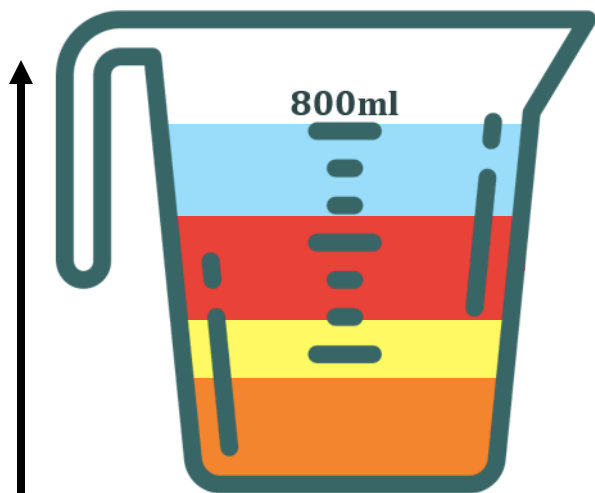
- 最高性价比优先
- 证明：贪心解不劣于最优解

最优解 S^*

替换为

贪心解 S'

按性价比
递减顺序
进行替换



替换后单位体积价值均不减少
故贪心解不劣于最优解

贪心方法适用的问题特点

- 有这样一类问题：它有 n 个输入，而它的解就是这 n 个输入的某个子集，这些子集必须满足某些事先给定的条件。
- 约束条件：必须满足的条件
- 可行解：满足约束条件的子集
- 目标函数：为了衡量可行解的优劣，预先给定衡量标准，以函数形式给出
- 最优解：使目标函数取极值(极大值或极小值)的可行解

各饮料体积



总体积不能
大于800ml

饮品价格



提出贪心策略



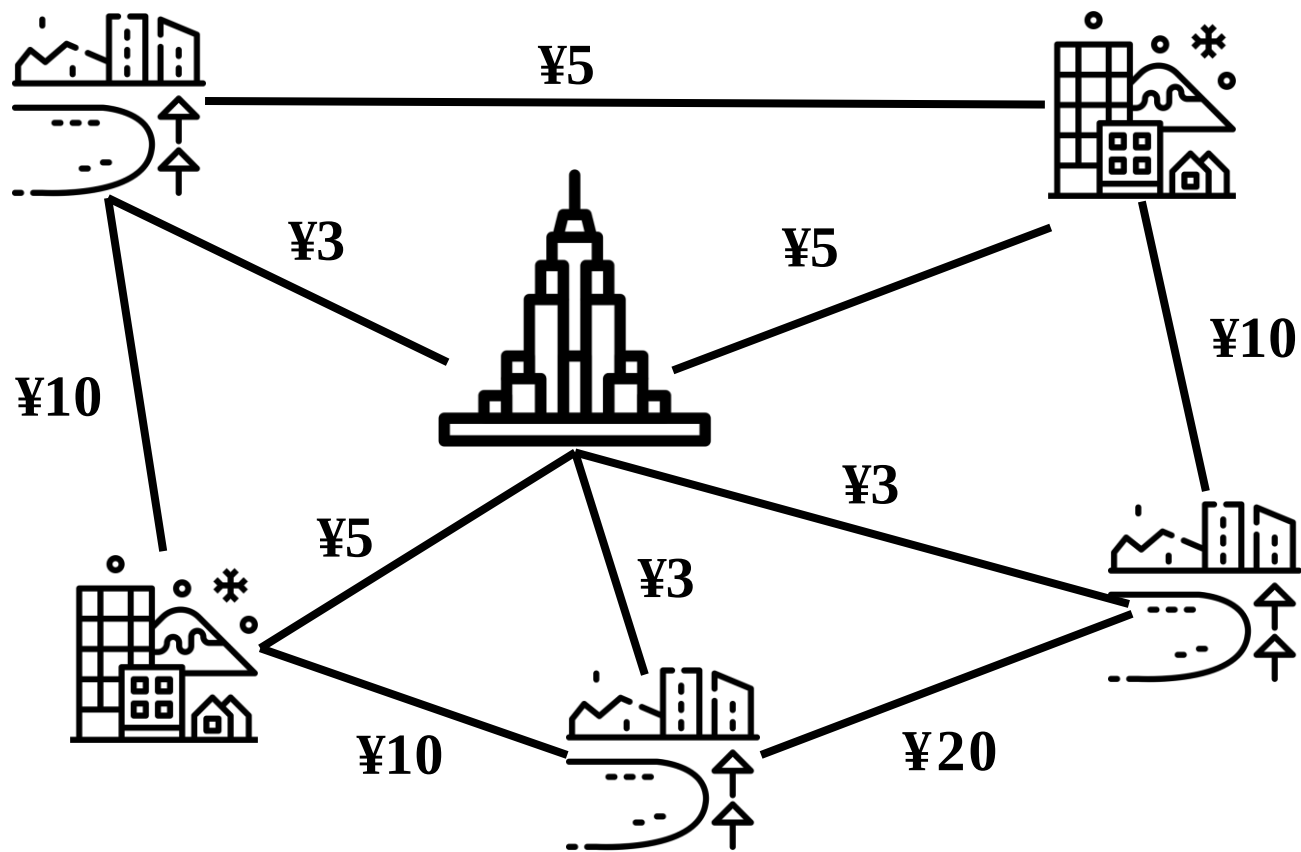
证明策略正确

观察问题特征，构造贪心选择

假设最优方案，通过替换证明

最小生成树问题

- 需要修建道路连通城市，各道路花费不同

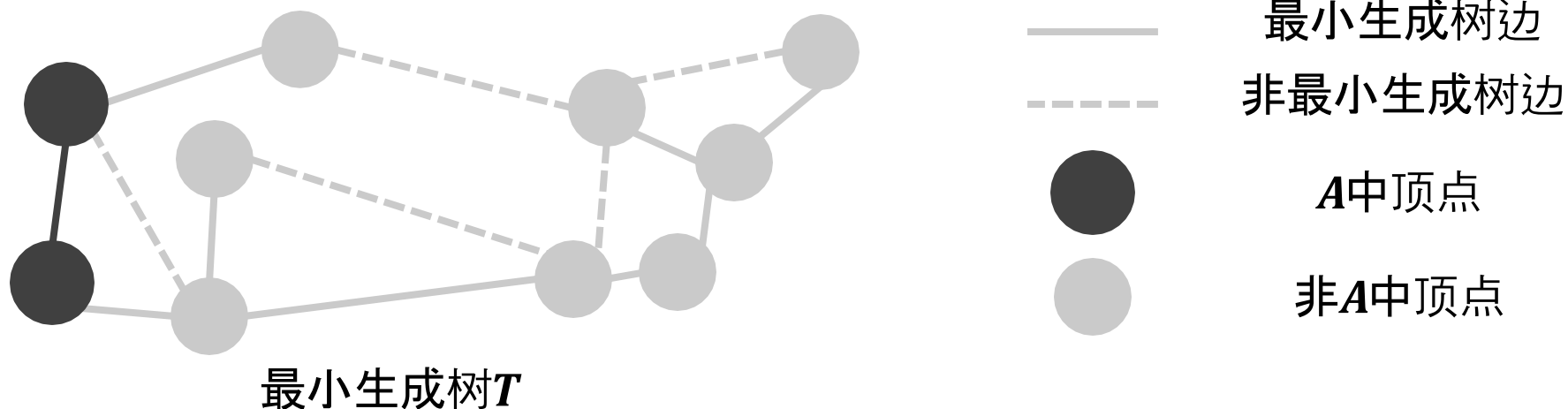


问题：连通各城市的最小花费是多少？

方案	花费
	¥74
	¥38
	¥19

权重最小的连通生成子图

- 安全边(Safe Edge)
 - A 是某棵最小生成树 T 边的子集, $A \subseteq T$
 - $A \cup \{(u, v)\}$ 仍是某棵最小生成树边的一个子集, 则称 (u, v) 是 A 的**安全边**

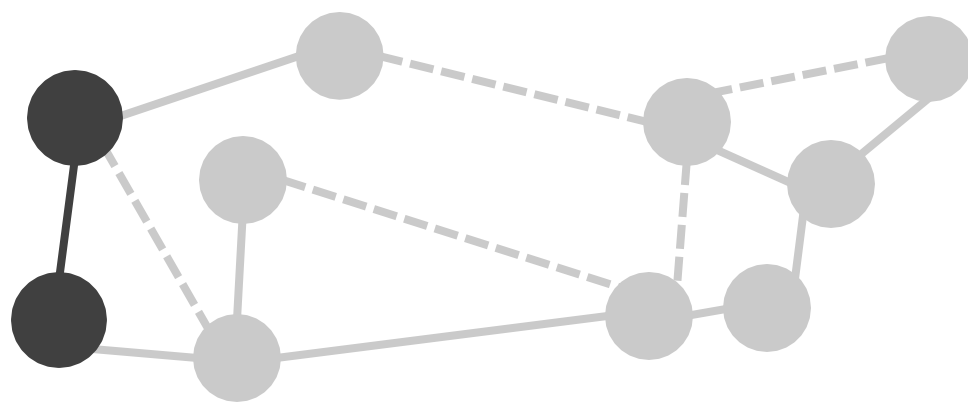


• Generic-MST(G)

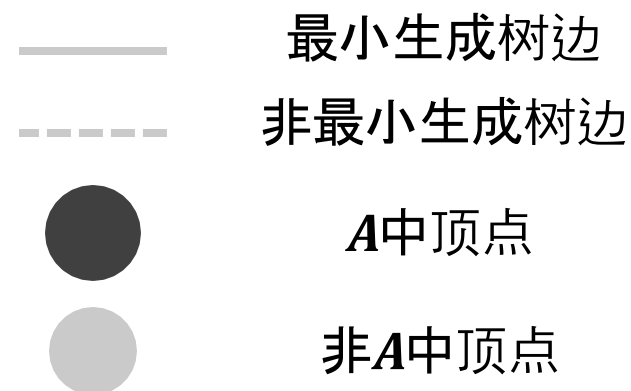
```
A ← ∅  
while 没有形成最小生成树 do  
    | 寻找 $A$ 的安全边 $(u, v)$   
    |  $A \leftarrow A \cup (u, v)$   
end  
return  $A$ 
```

问题：如何有效辨识安全边？

- 安全边(Safe Edge)
 - A 是某棵最小生成树 T 边的子集, $A \subseteq T$
 - $A \cup \{(u, v)\}$ 仍是某棵最小生成树边的一个子集, 则称 (u, v) 是 A 的**安全边**



最小生成树 T



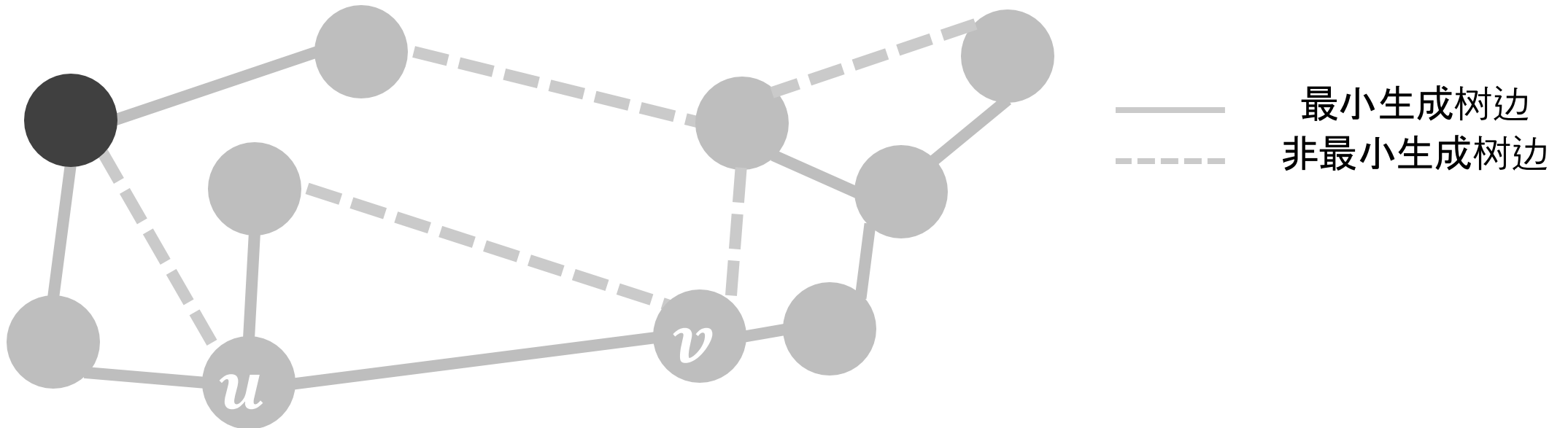
- Generic-MST(G)**

```
A ← ∅  
while 没有形成最小生成树 do  
    | 寻找A的安全边( $u, v$ )  
    | A ← A ∪ ( $u, v$ )  
end  
return A
```

Prim算法

Kruskal算法

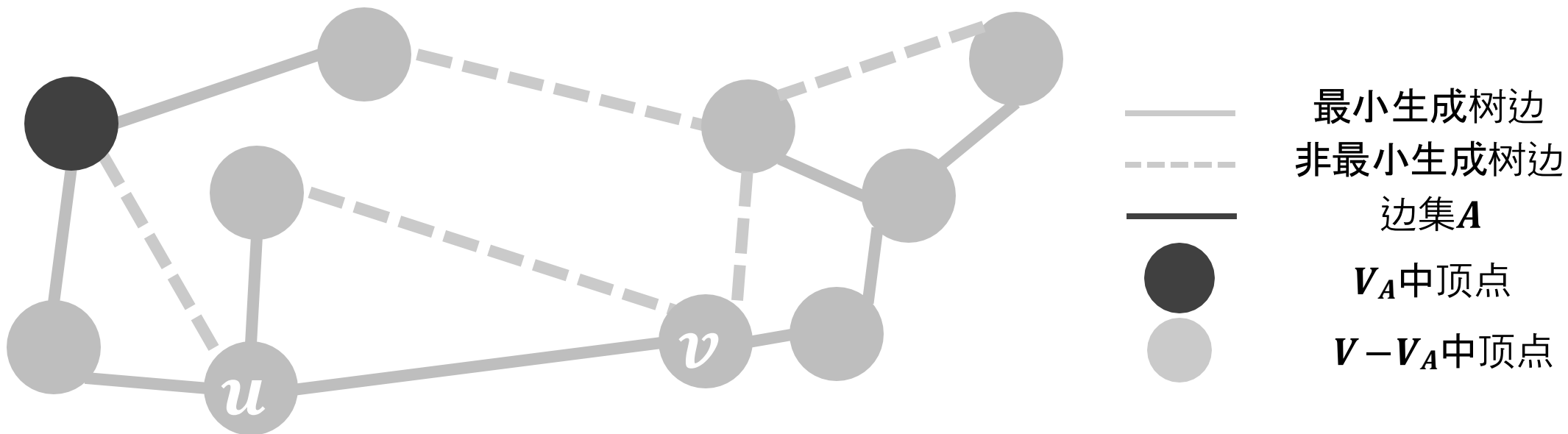
- 算法思想
 - 步骤1：选择任意一个顶点，作为生成树的起始顶点



- 算法思想

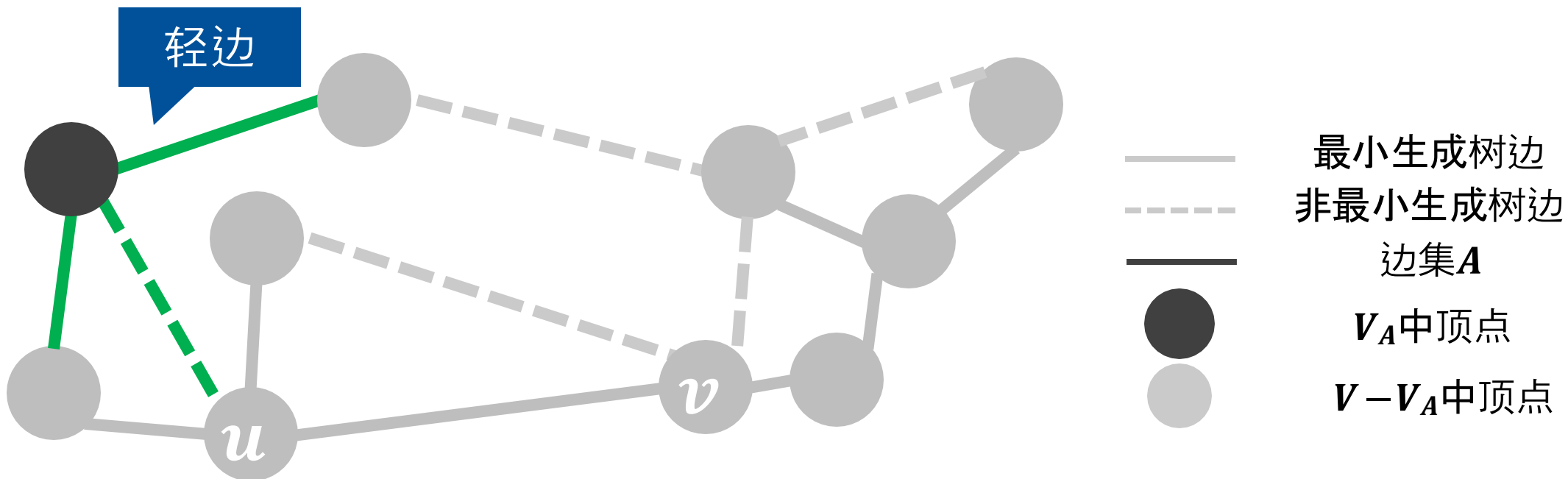
- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$

边集 A 的顶点



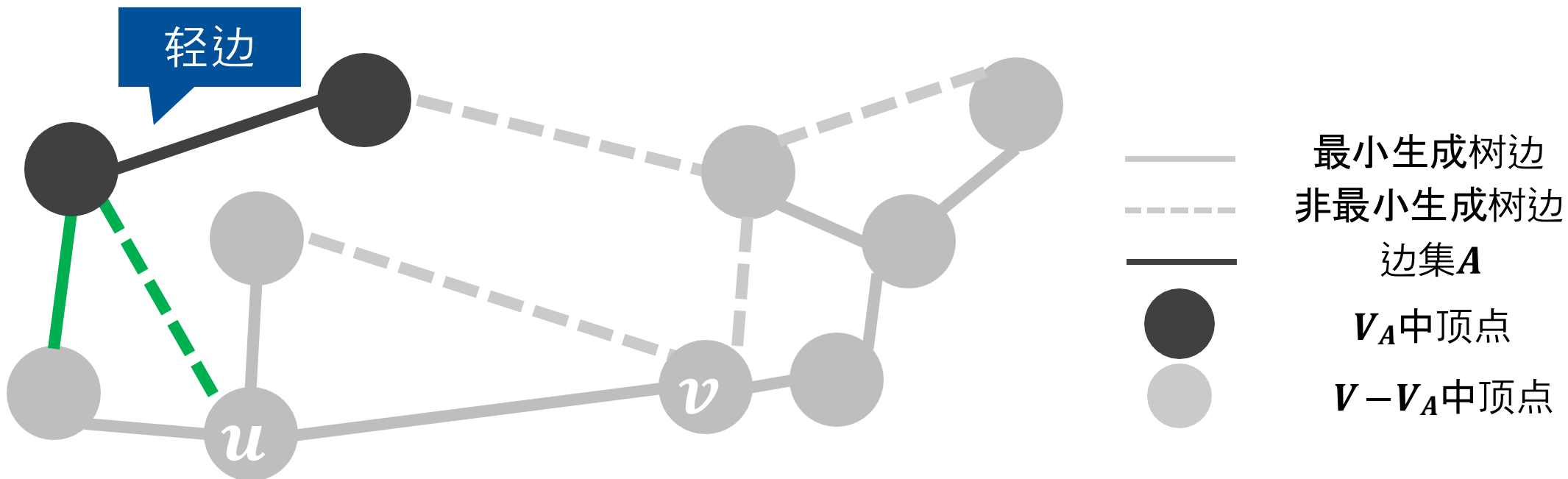
- 算法思想

- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$
- 步骤3：选择横跨割 $(V_A, V - V_A)$ 的轻边



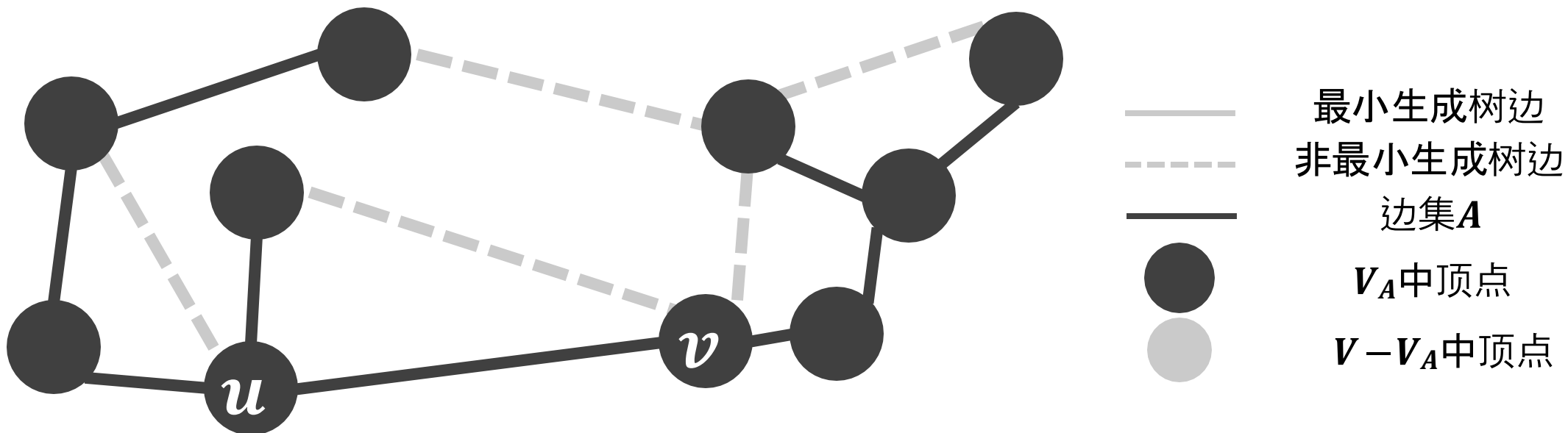
- 算法思想

- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$
- 步骤3：选择横跨割 $(V_A, V - V_A)$ 的**轻边**，添加到边集 A 中



- 算法思想

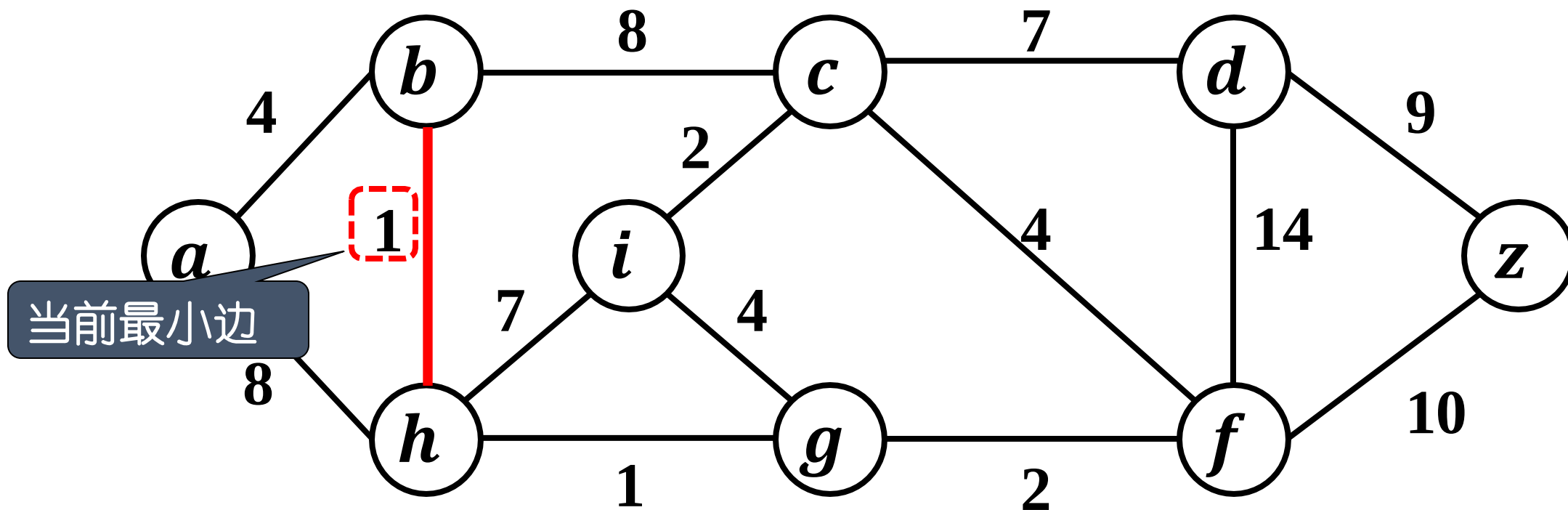
- 步骤1：选择任意一个顶点，作为生成树的起始顶点
- 步骤2：保持边集 A 始终为一棵树，选择割 $(V_A, V - V_A)$
- 步骤3：选择横跨割 $(V_A, V - V_A)$ 的**轻边**，添加到边集 A 中
- 步骤4：重复步骤2和步骤3，直至覆盖所有顶点



Kruskal算法

- 算法思想：直接实现通用框架
 - 需保证边集 A 仍是一个无环图
 - 选边时避免成环
 - 需保证边集 A 仍是最小生成树的子集
 - 每次选择当前权重最小边

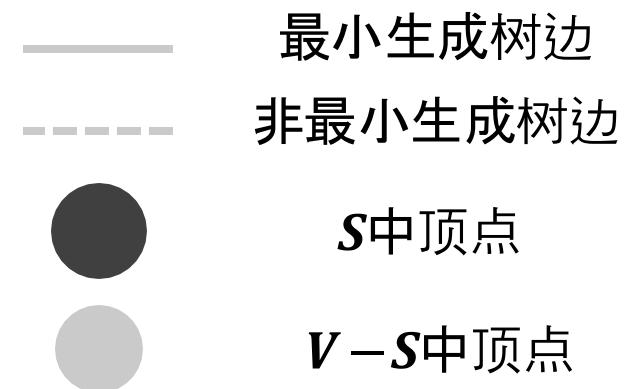
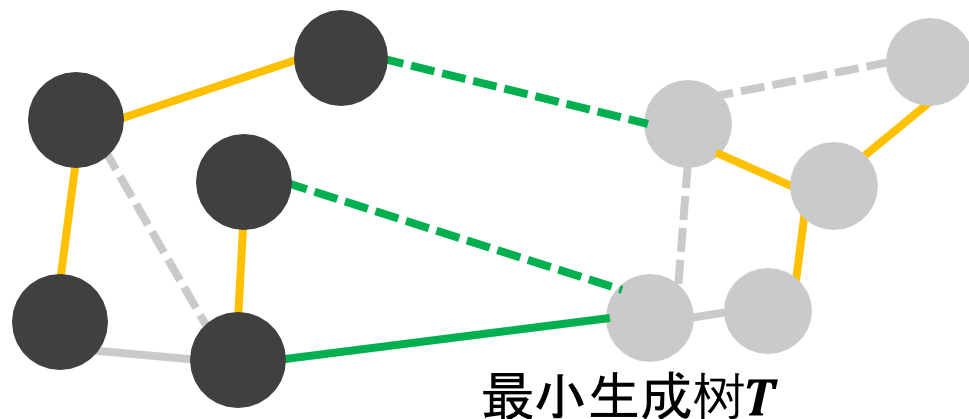
```
 $A \leftarrow \emptyset$   
while 没有形成最小生成树 do  
    | 寻找 $A$ 的安全边 $(u, v)$   
    |  $A \leftarrow A \cup (u, v)$   
end  
return  $A$ 
```



Prim算法和Kruskal算法比较

	Prim算法	Kruskal算法
核心思想	保持一棵树，不断扩展	子树森林，合并为一棵树
数据结构	优先队列	不相交集合
复杂度	$O(E \log V)$	$O(E \log V)$
算法策略	都是采用贪心策略的图算法	

- 割(Cut)
 - 图 $G = \langle V, E \rangle$ 是一个连通无向图, 割 $(S, V - S)$ 将图 G 的顶点集 V 划分为两部分
- 横跨(Cross)
 - 给定割 $(S, V - S)$ 和边 (u, v) , $u \in S, v \in V - S$, 称边 (u, v) 横跨割 $(S, V - S)$
- 轻边(Light Edge)
 - 横跨割的所有边中, 权重最小的称为横跨这个割的一条轻边
- 不妨害(Respect)
 - 如果一个边集 A 中没有边横跨某割, 则称该割**不妨害**边集 A



- 给定图 $G = \langle V, E \rangle$ 是一个带权的连通无向图，令 A 为边集 E 的一个子集，且 A 包含在图 G 的某棵最小生成树中
 - 若割 $(S, V - S)$ 是图 G 中 **不妨害** 边集 A 的任意割，且 (u, v) 是横跨该割的 **轻边**
 - 则对于边集 A ，边 (u, v) 是其 **安全边**

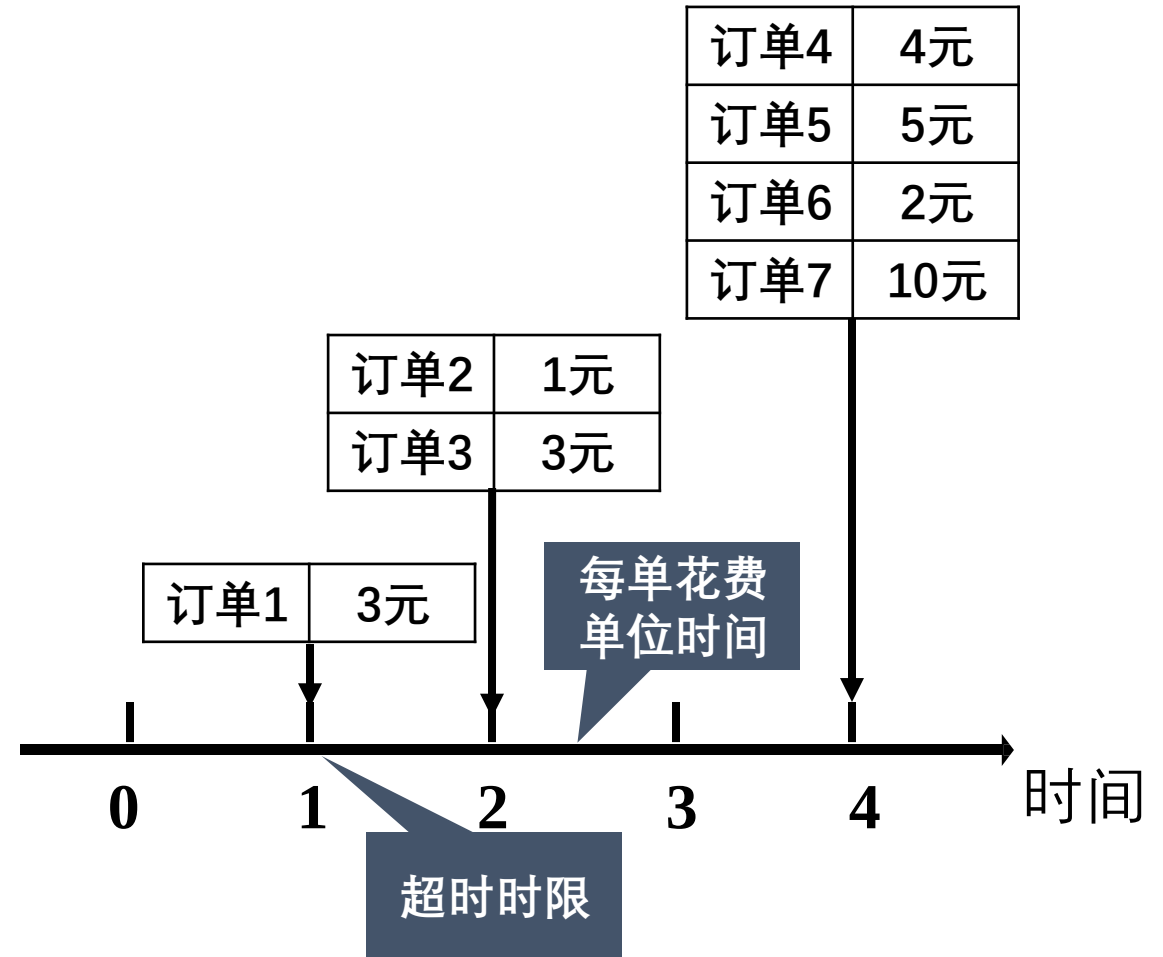
Prim算法

Kruskal算法

- 根据边集构建（边集不妨碍的）割
- 每次选择的都是割的轻边
- 所以每步添加的都是安全边

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 最优前缀码问题
- 5.5 货郎担问题
- 5.6 小结

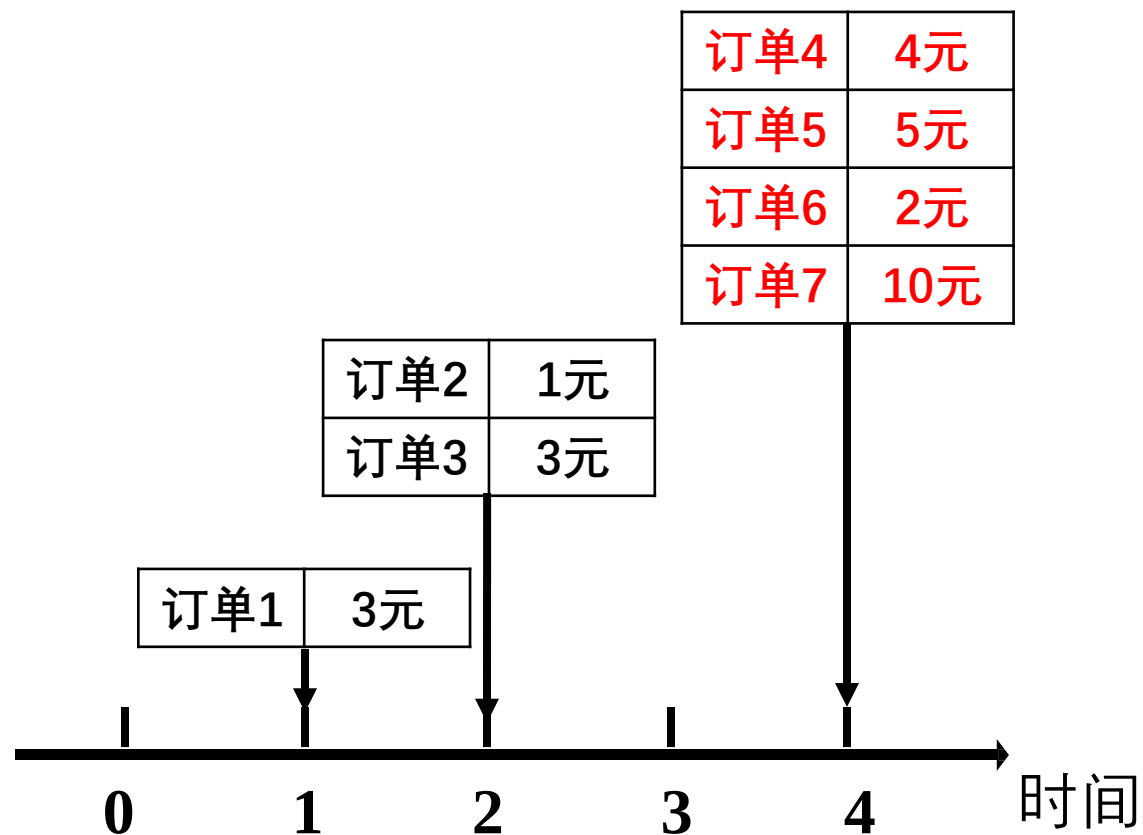
外卖送单



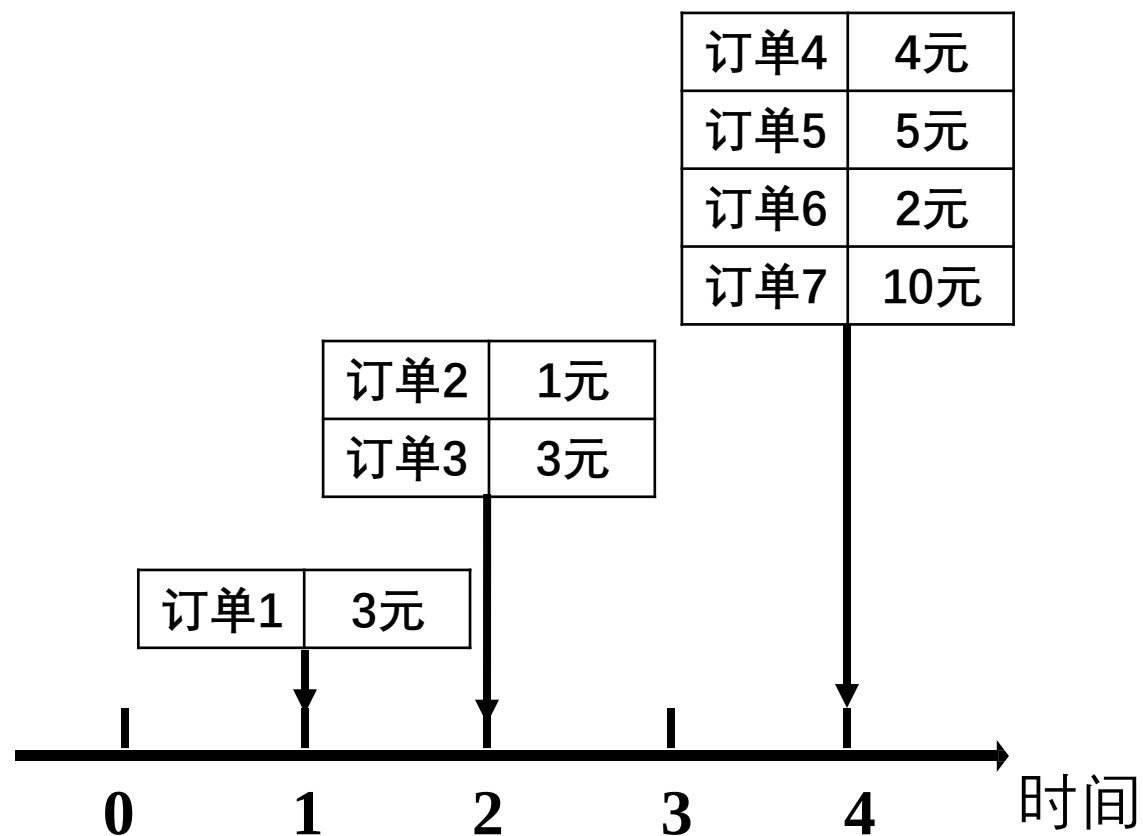
思考：如何选择订单？

- 超时时限优先：

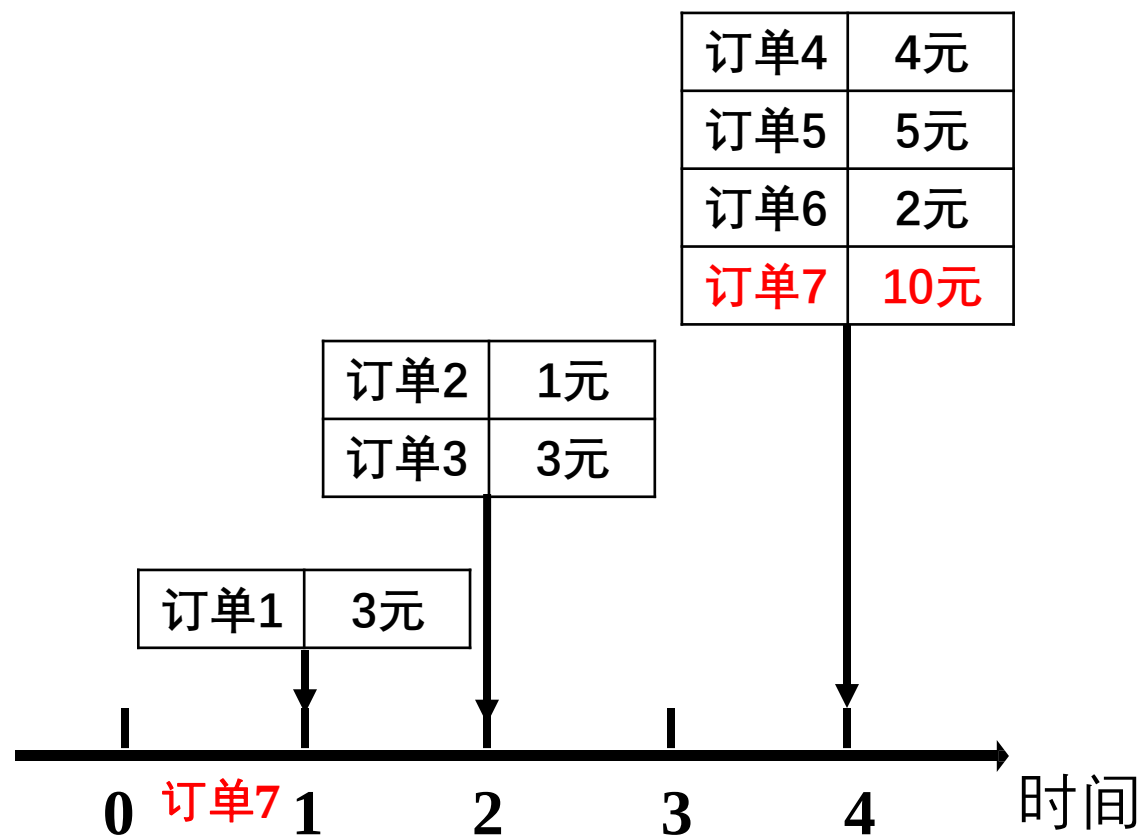
- 选择订单4、5、6、7
- 送餐费： $4+5+2+10=21$



- 超时时限优先：
 - 选择订单4、5、6、7
 - 送餐费： $4+5+2+10=21$
- 送餐费优先：



- 超时时限优先：
 - 选择订单4、5、6、7
 - 送餐费： $4+5+2+10=21$
- 送餐费优先：
 - 选择订单7

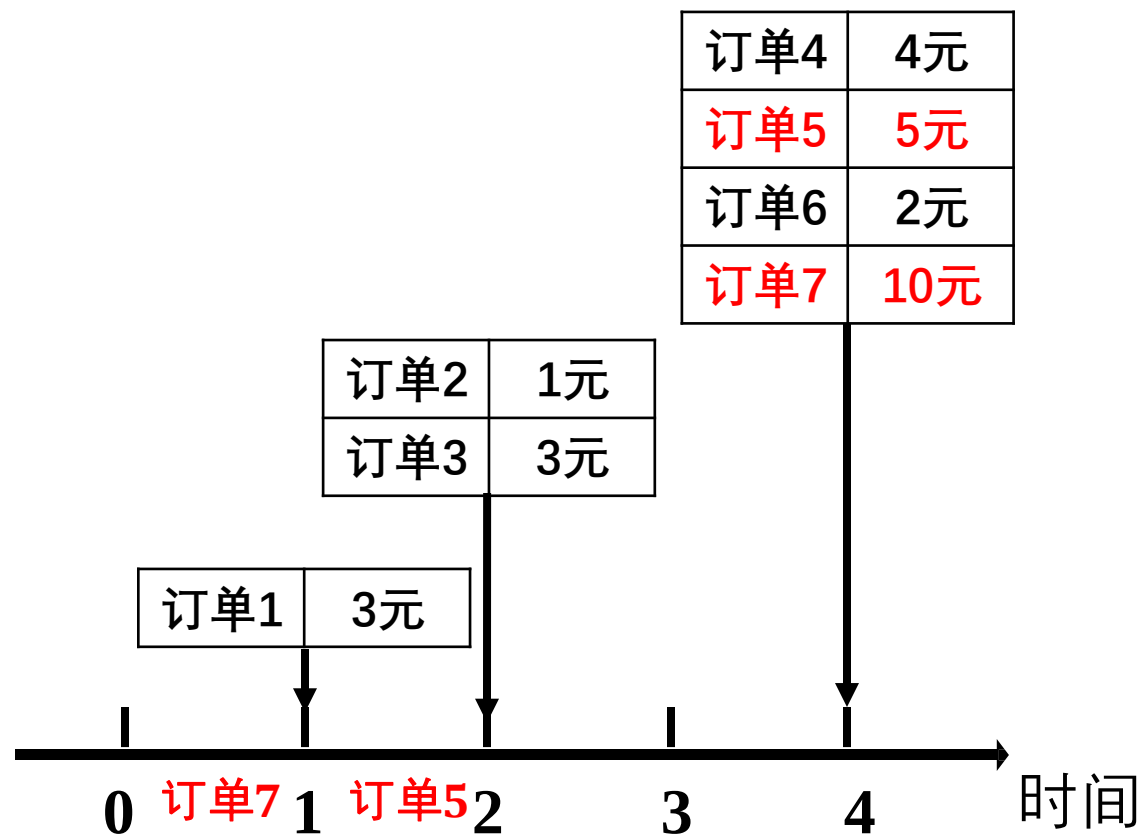


- 超时时限优先：

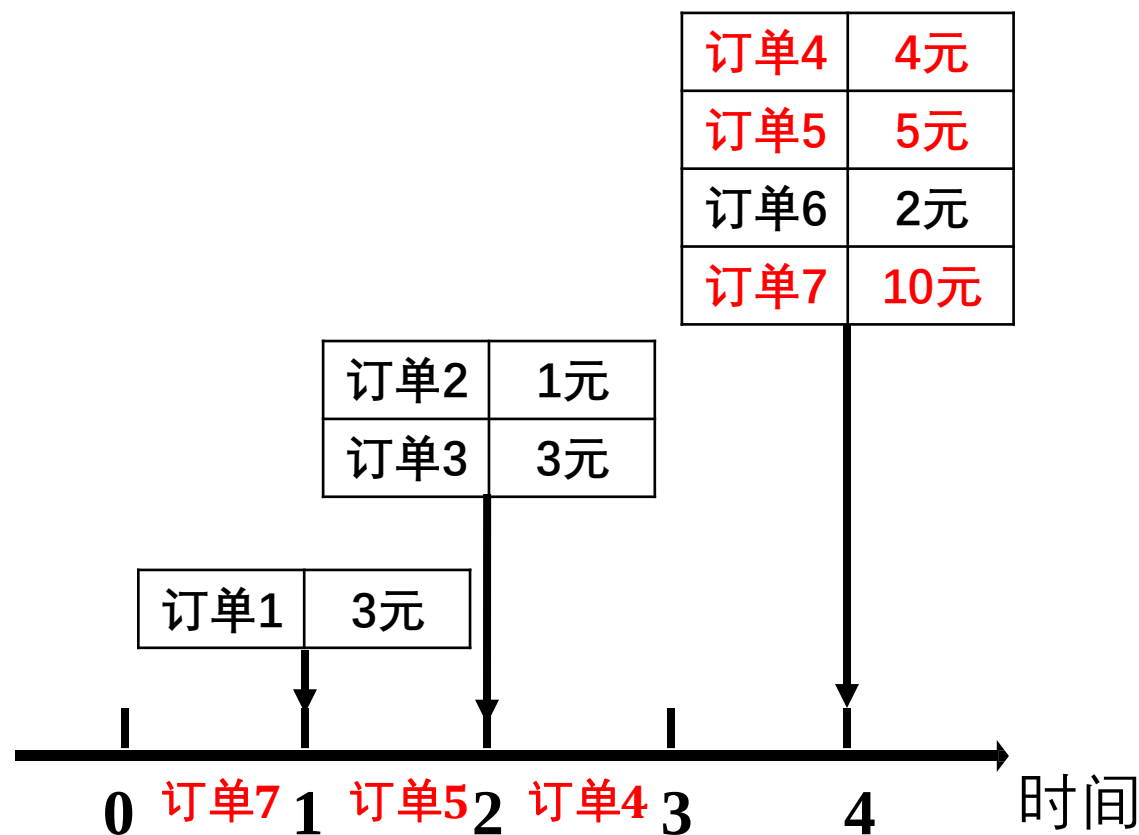
- 选择订单4、5、6、7
- 送餐费： $4+5+2+10=21$

- 送餐费优先：

- 选择订单7
- 选择订单5



- 超时时限优先：
 - 选择订单4、5、6、7
 - 送餐费： $4+5+2+10=21$
- 送餐费优先：
 - 选择订单7
 - 选择订单5
 - 选择订单4

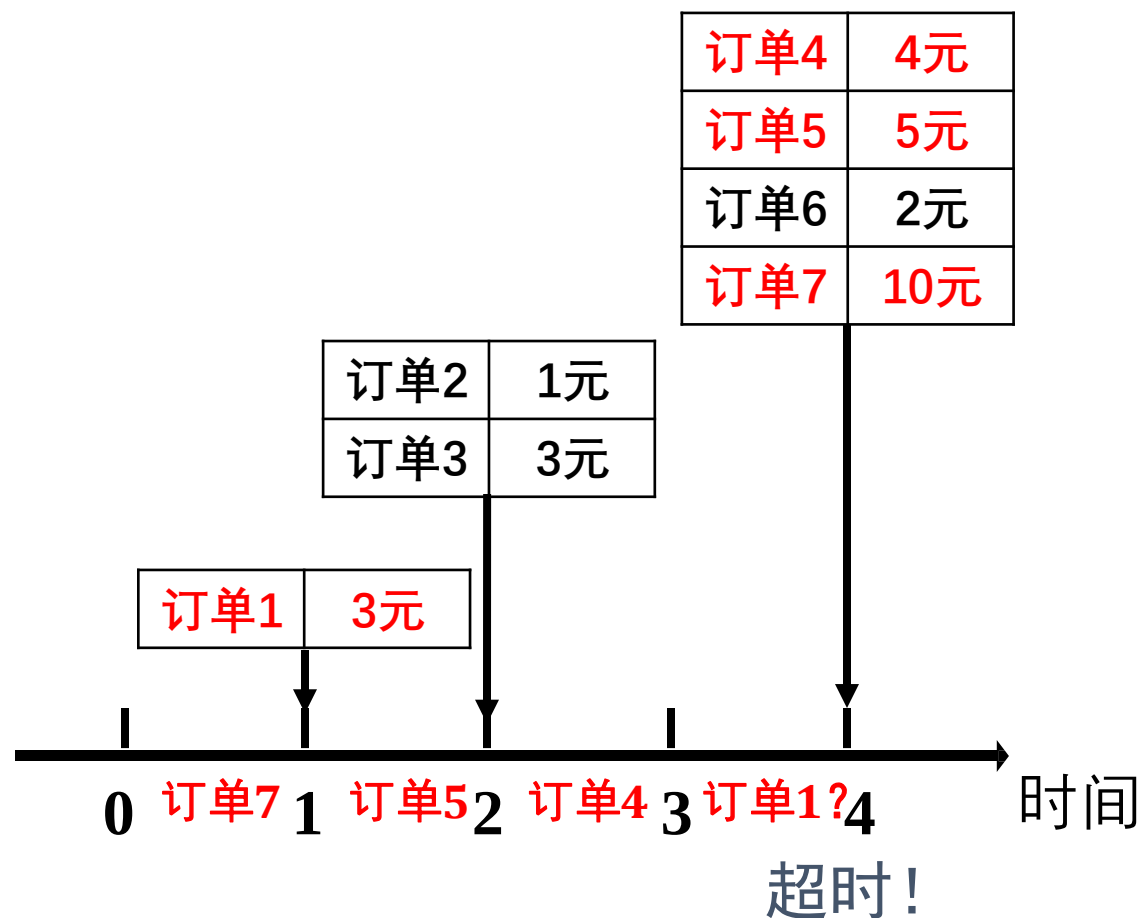


- 超时时限优先：

- 选择订单4、5、6、7
- 送餐费： $4+5+2+10=21$

- 送餐费优先：

- 选择订单7
- 选择订单5
- 选择订单4
- 选择订单1

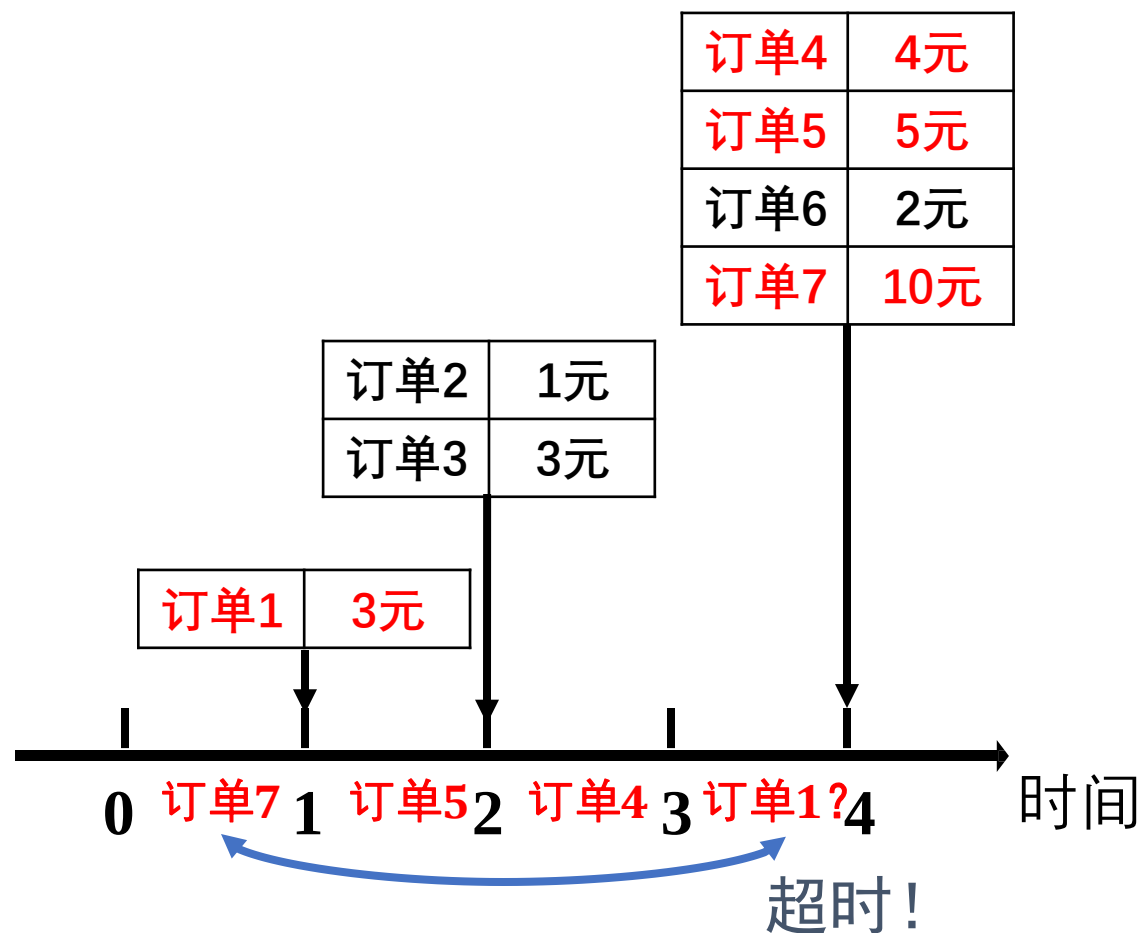


- 超时时限优先：

- 选择订单4、5、6、7
- 送餐费： $4+5+2+10=21$

- 送餐费优先：

- 选择订单7
- 选择订单5
- 选择订单4
- 选择订单1

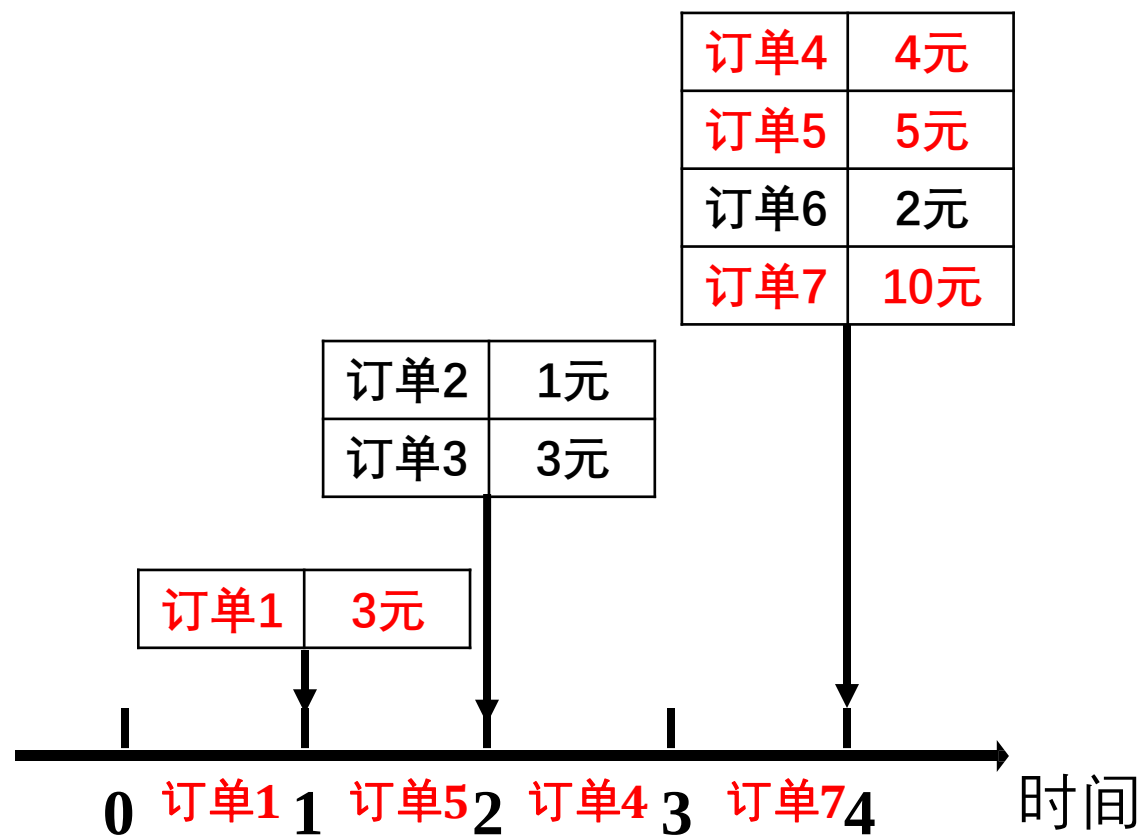


- 超时时限优先：

- 选择订单4、5、6、7
- 送餐费： $4+5+2+10=21$

- 送餐费优先：

- 选择订单7
- 选择订单5
- 选择订单4
- 选择订单1
- 送餐费： $10+5+4+3=22$



- 假定只能在一台机器上处理 n 个作业, 每个作业均可在单位时间内完成; 又假定每个作业 i 都有一个截止期限 $d_i > 0$ (d_i 是整数), 当且仅当作业 i 在它的期限截止之前被完成时获得 $p_i > 0$ 的效益。
- 该问题的一个可行解
 - 是这 n 个作业的一个子集合 J : J 中的每个作业都能在各自的截止期限之前完成, 可行解的效益值是 J 中这些作业的效益之和 $\sum p_j$
- 该问题的一个最优解
 - 具有最大效益值的可行解

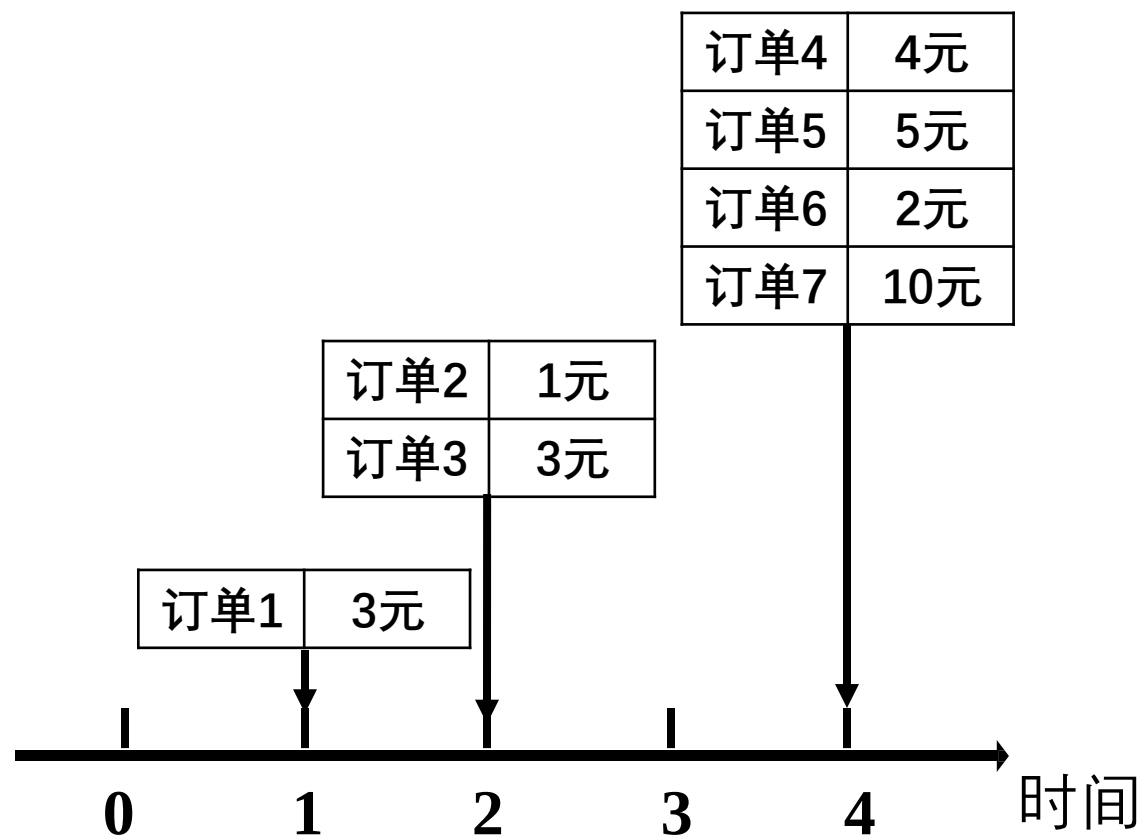
限制条件

目标函数

外卖送单实例

- 输入：
 - 效益($p_1, p_2, p_3, p_4, p_5, p_6, p_7$) =
(3, 1, 3, 4, 5, 2, 10)
 - 期限($d_1, d_2, d_3, d_4, d_5, d_6, d_7$) =
(1, 2, 2, 4, 4, 4, 4)
- 输出：总效益最大的可行解
- 寻找最优量度标准
 - 以目标函数 $\sum p_j$ 作为量度标准, 将各作业按效益 p_i 降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$

当前作业一旦满足约束条件,
问题就能获得的最大效益增量



- 输入：
 - 效益($p_1, p_2, p_3, p_4, p_5, p_6, p_7$) =
(3, 1, 3, 4, 5, 2, 10)
 - 期限($d_1, d_2, d_3, d_4, d_5, d_6, d_7$) =
(1, 2, 2, 4, 4, 4, 4)
- 输出：总效益最大的可行解
- 寻找最优量度标准
 - 以目标函数 $\sum p_j$ 作为量度标准, 将各作业按效益 p_i 降序排列: $p_1 \geq p_2 \geq \dots \geq p_n$

当前作业一旦满足约束条件,
问题就能获得的最大效益增量

```
J ← {1}
for i ← 2 to n do
    if (JU{i}的所有作业都能在它们的截止
        期限前完成)
        then J ← JU{i}
    end
end
```

- 正确策略：效益优先

最优解 I

贪心解 J

$I = J?$

J 是最优解!

$I \subset J?$

I 是最优解, 不可能!

$I \supset J?$

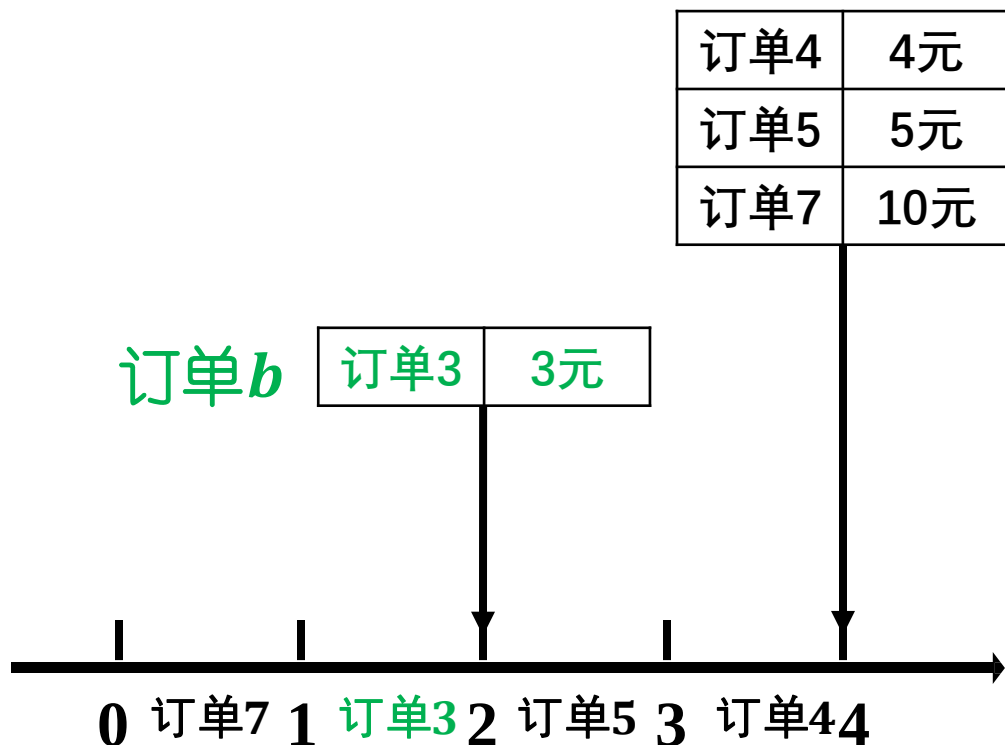
由贪心策略, 不可能!

存在 $a \in J \setminus I, b \in I \setminus J$

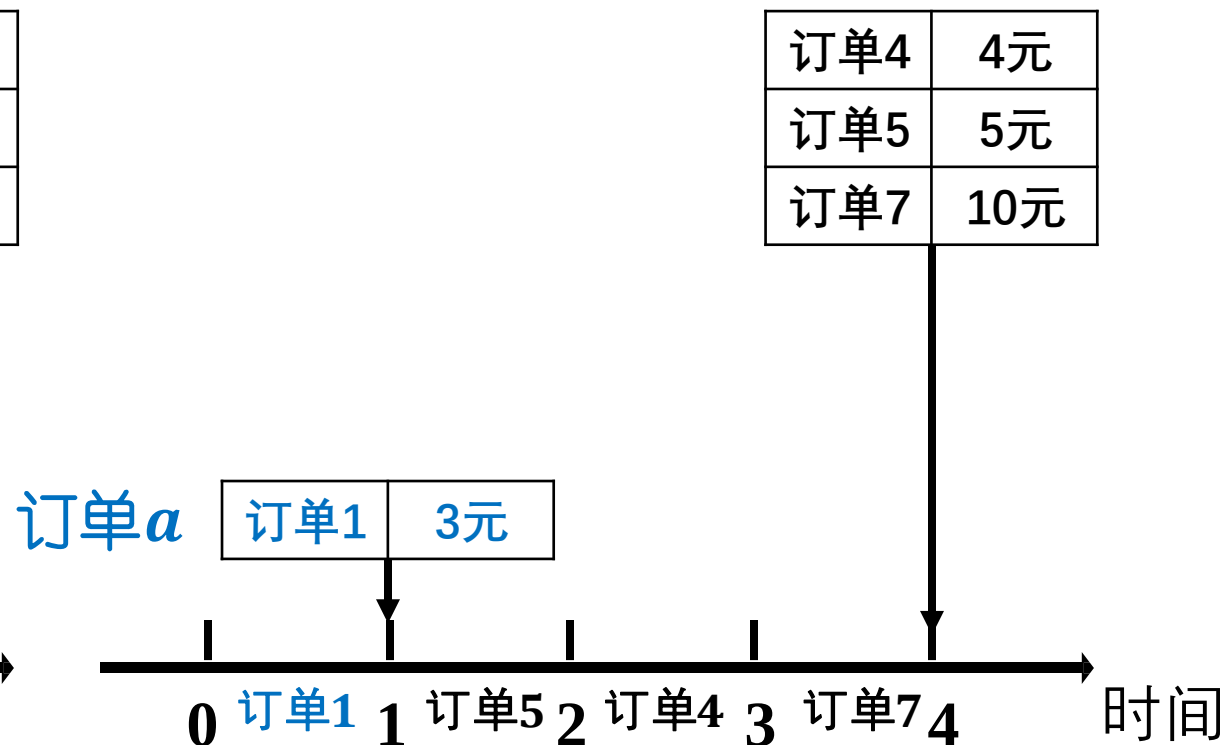
设 a 为 $J \setminus I$ 中效益最大的作业,
则 $p_a \geq \max_{b \in I \setminus J} p_b$

- 正确策略：效益优先

最优解 I



贪心解 J

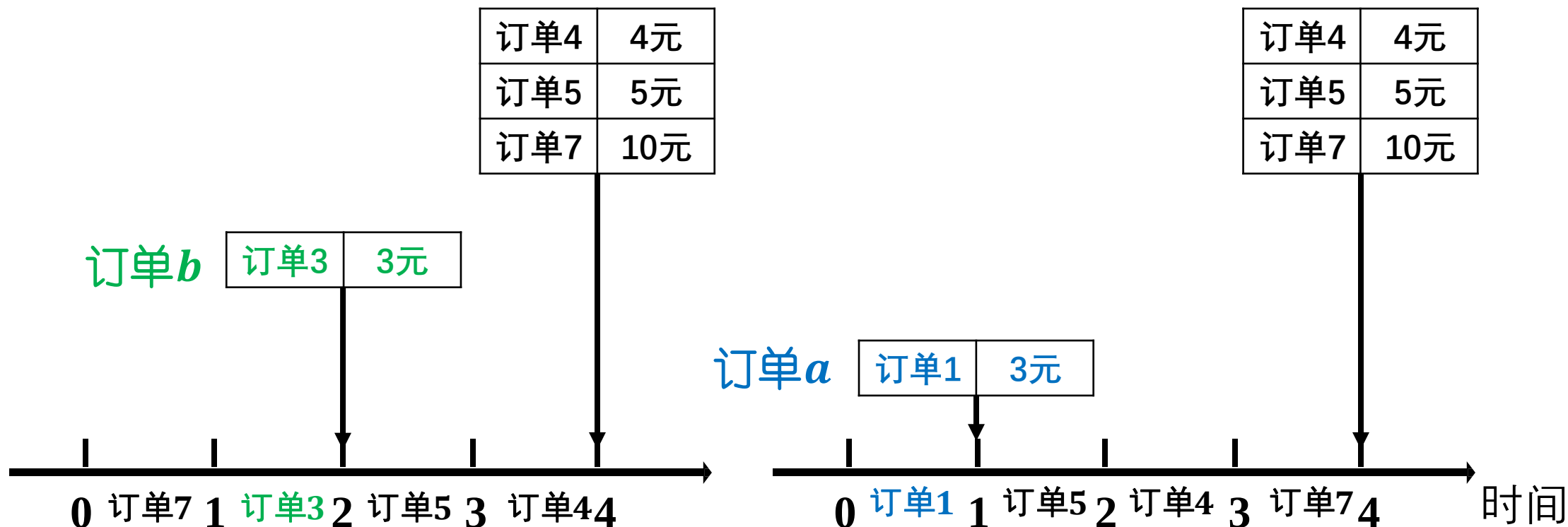


- 正确策略：效益优先

最优解 I

依次检查并替换

贪心解 J



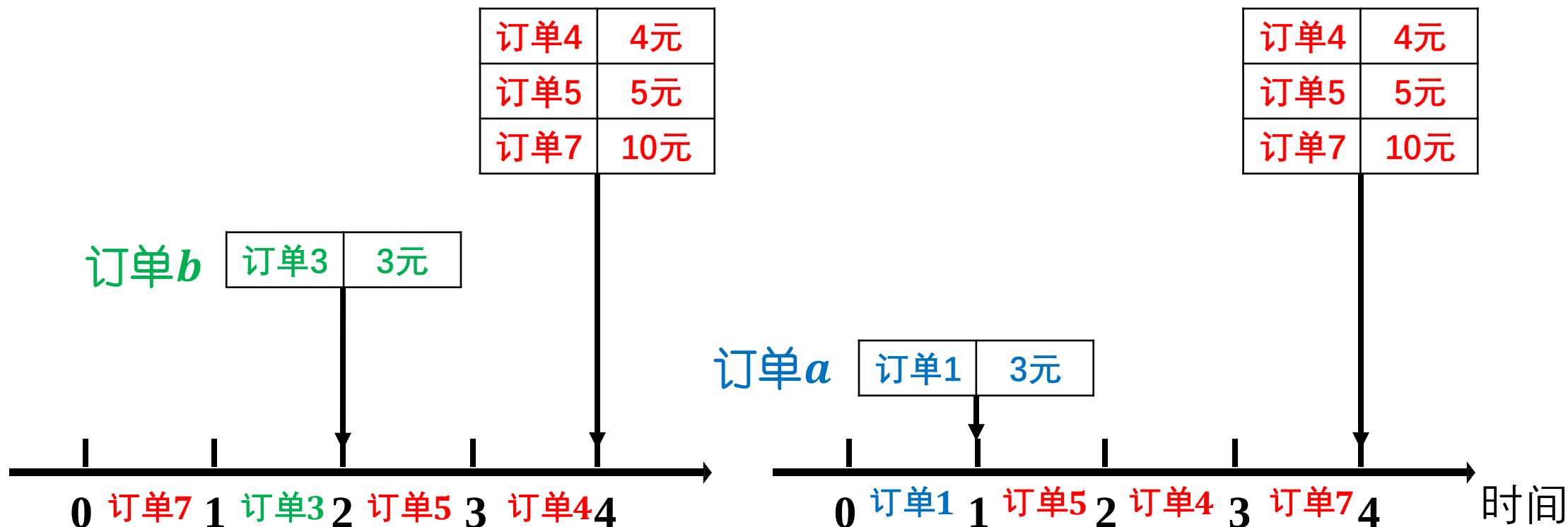
正确性证明

- 正确策略：效益优先

最优解 I

依次检查并替换
重新排列相同作业

贪心解 J



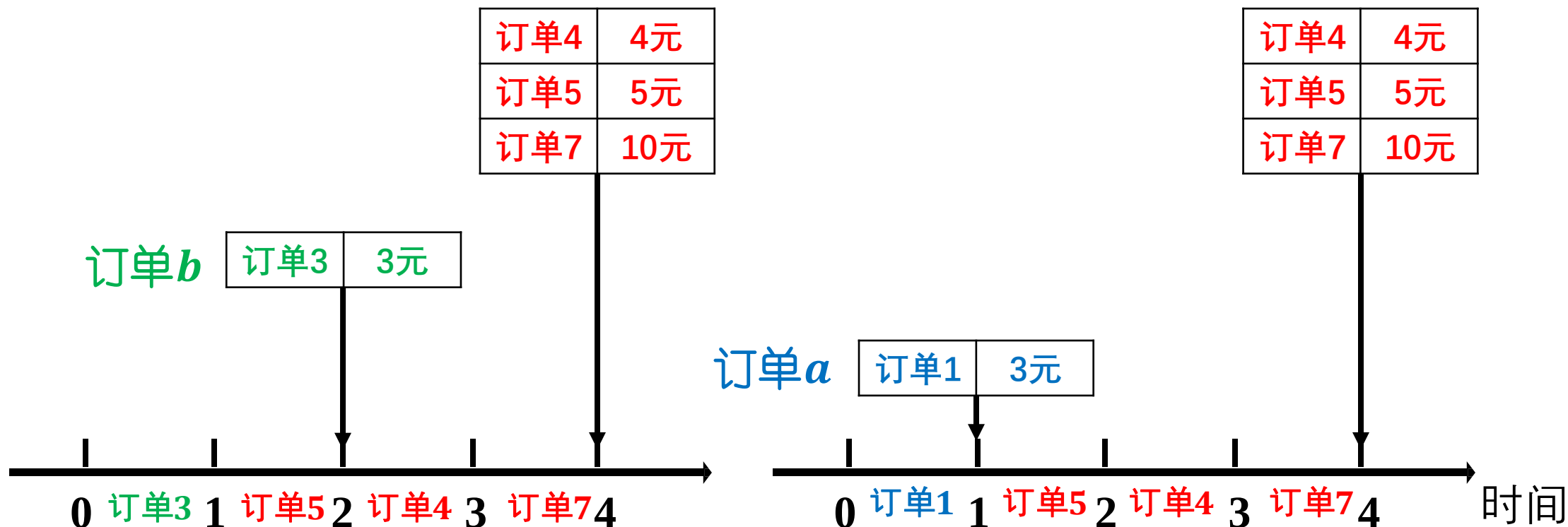
正确性证明

- 正确策略：效益优先

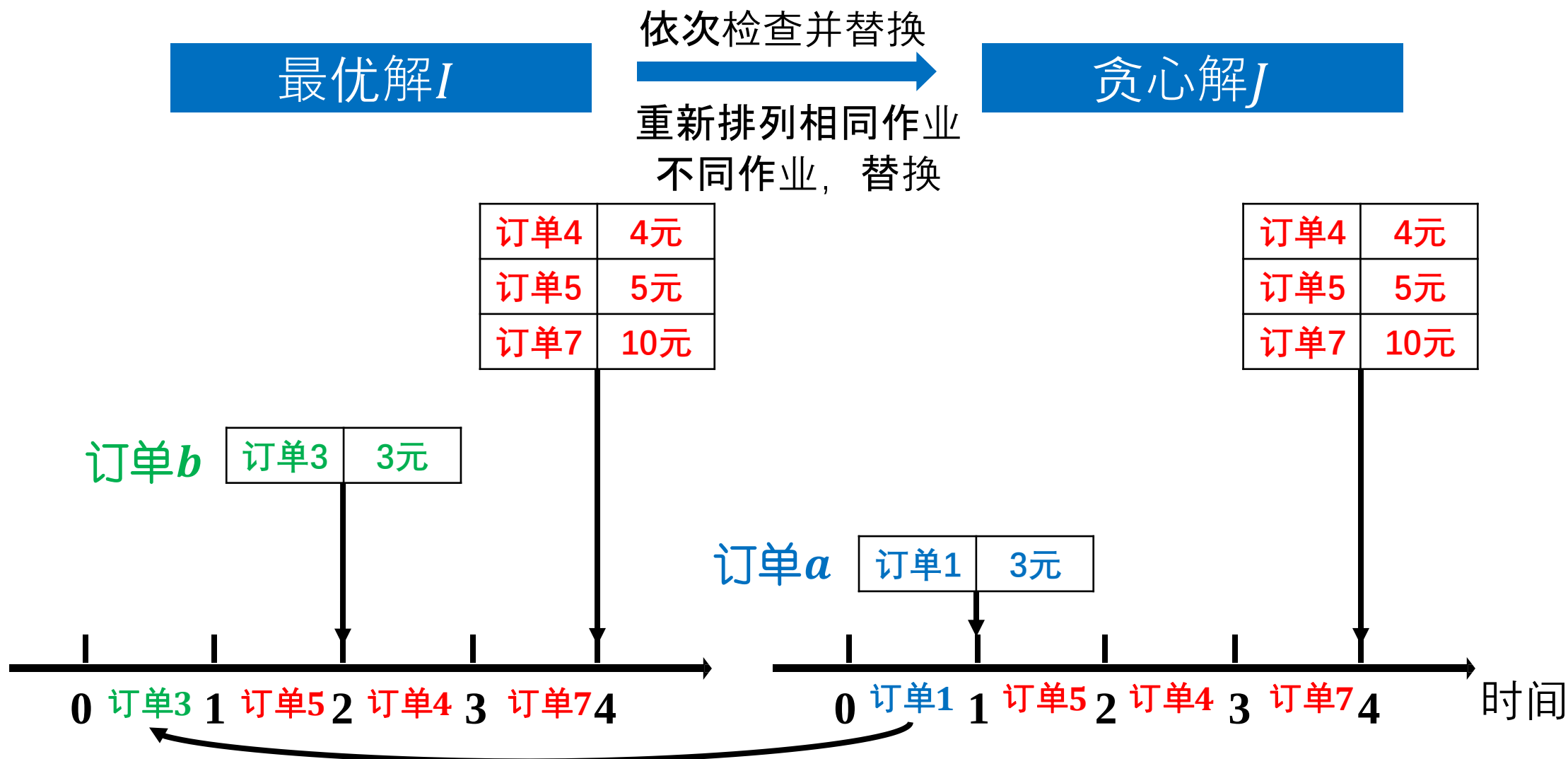
最优解 I

依次检查并替换
重新排列相同作业

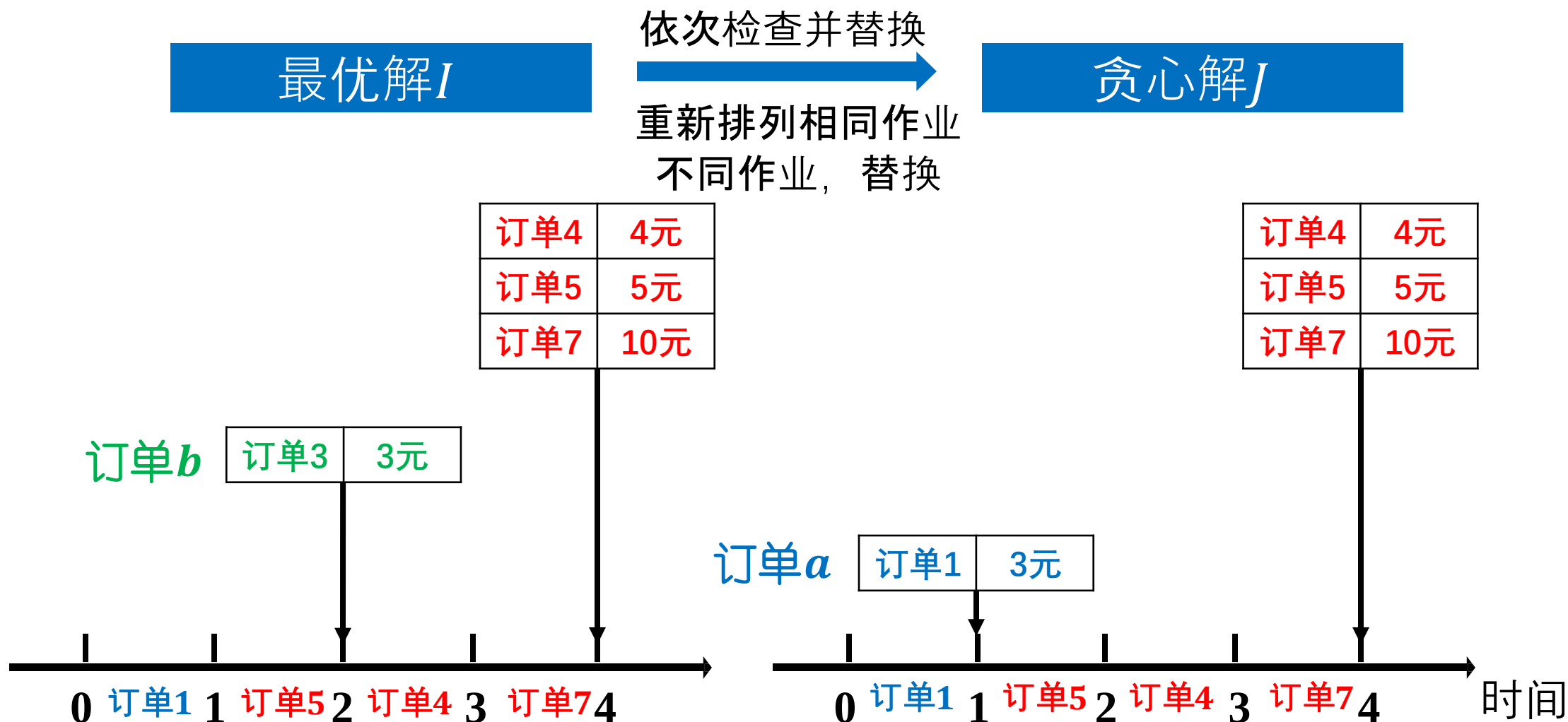
贪心解 J



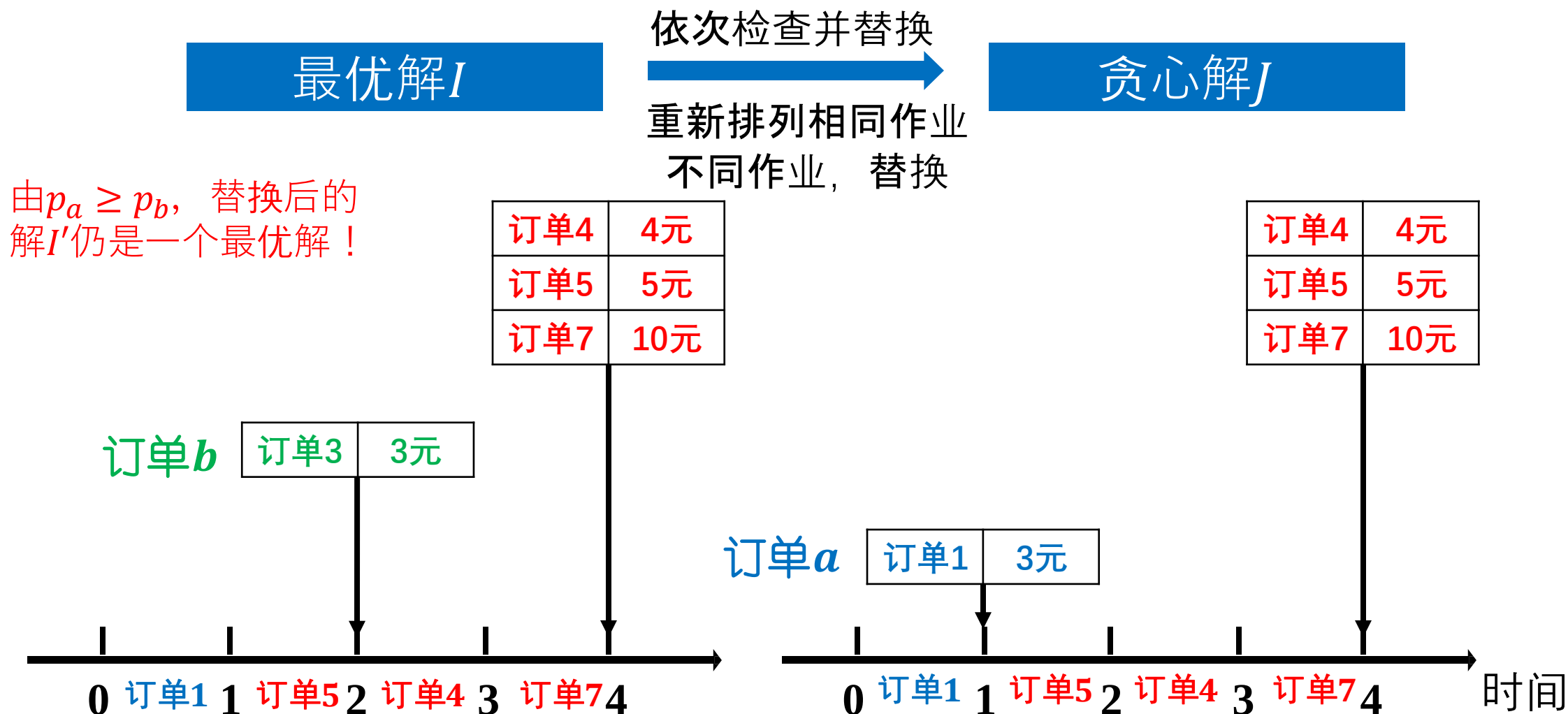
- 正确策略：效益优先



- 正确策略：效益优先



- 正确策略：效益优先



- 正确策略：效益优先

最优解 I

依次检查并替换

重新排列相同作业
不同作业，替换

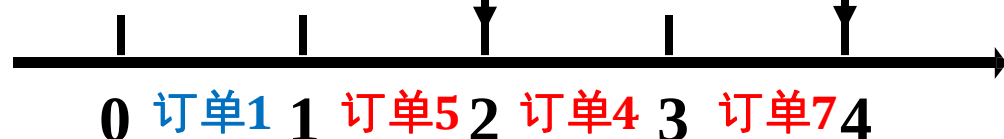
贪心解 J

由 $p_a \geq p_b$ ，替换后的解 I' 仍是一个最优解！
但 I' 相比于 I 少一个与 J 不同的作业！

订单4	4元
订单5	5元
订单7	10元

订单 b

订单3	3元
-----	----



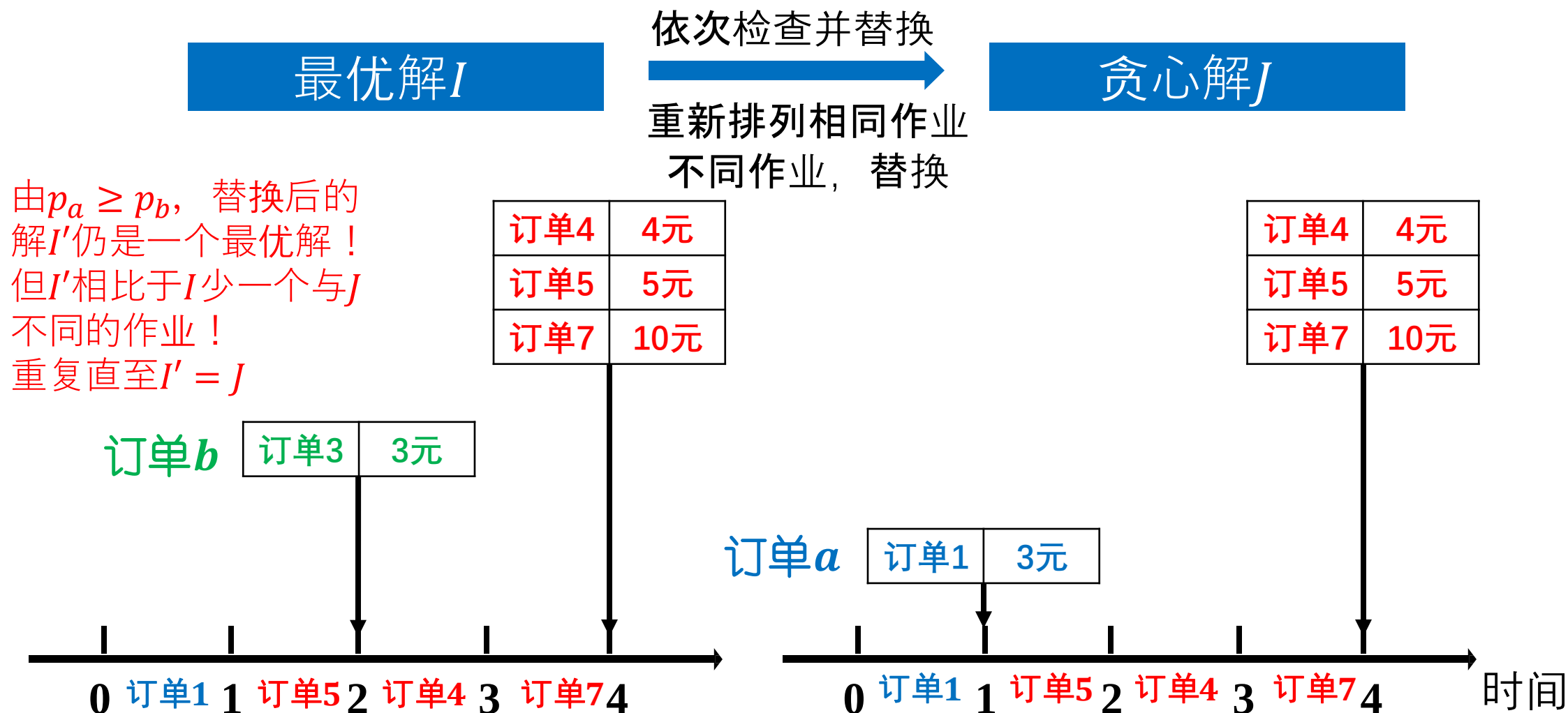
订单 a

订单1	3元
-----	----

订单4	4元
订单5	5元
订单7	10元



- 正确策略：效益优先



如何判断J是可行解的策略？

```
J ← {1}
for i ← 2 to n do
    if (JU{i}的所有作业都能在它们的截止日期前完成)
        then J ← JU{i}
    end
end
```

如何判断J是可行解的策略？

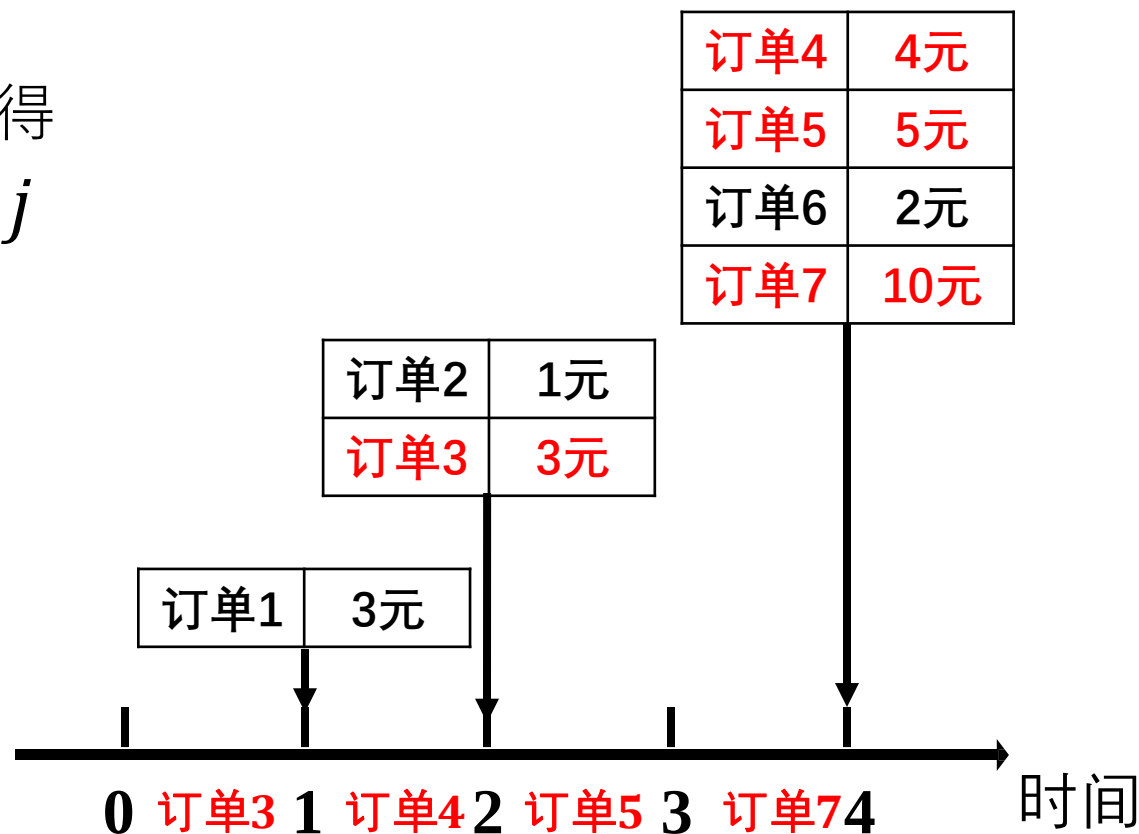
- 输入：作业期限($d_1, d_2, \dots, d_k$)
- 输出：是否一个排列 $i_1, i_2, i_3, \dots, i_k$, 使得
$$\forall j = 1, 2, \dots, k, d_{i_j} \geq j$$

- 贪心策略：

- 按**期限**升序对作业进行排列

	订单3	订单4	订单5	订单7
序号	1	2	3	4
期限	2	4	4	4

- 可行当且仅当**期限** $\geq$ **序号**



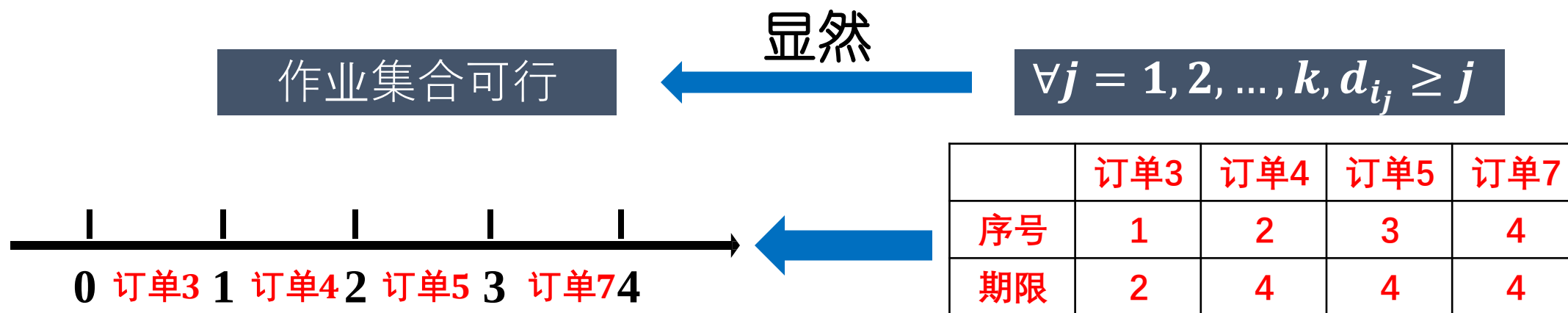
- 贪心策略：
 - 按**期限**升序对作业进行排列

作业集合可行



$$\forall j = 1, 2, \dots, k, d_{i_j} \geq j$$

- 贪心策略：
 - 按**期限**升序对作业进行排列



- 贪心策略：
 - 按**期限**升序对作业进行排列



- 贪心策略：
 - 按**期限**升序对作业进行排列

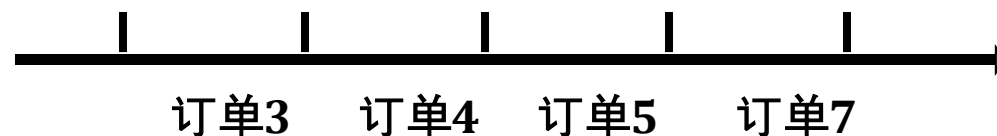
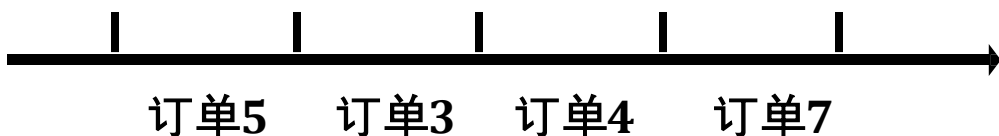
作业集合可行

可行解

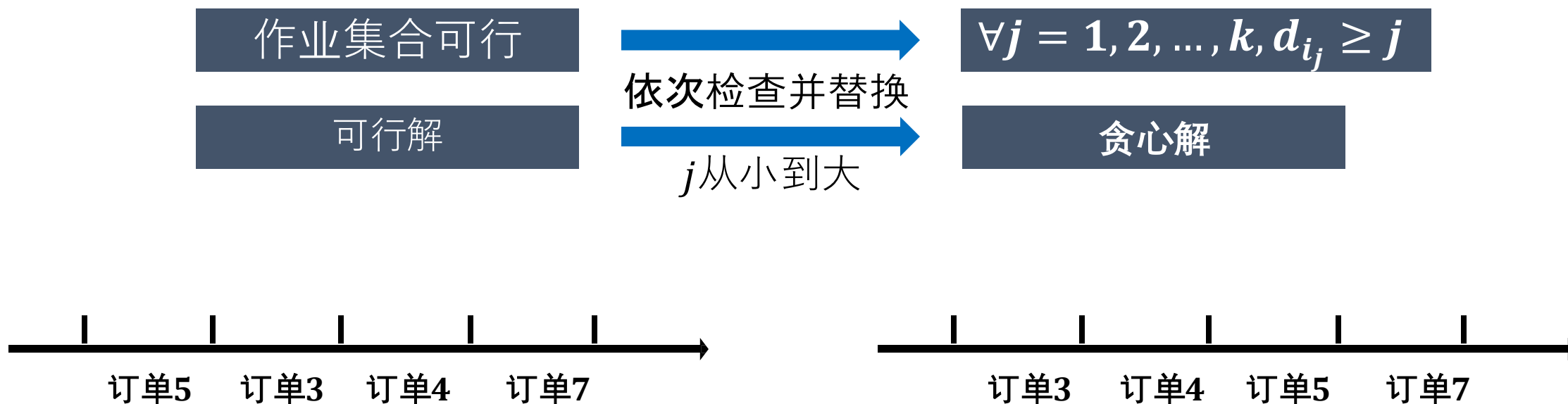
依次检查并替换

$\forall j = 1, 2, \dots, k, d_{i_j} \geq j$

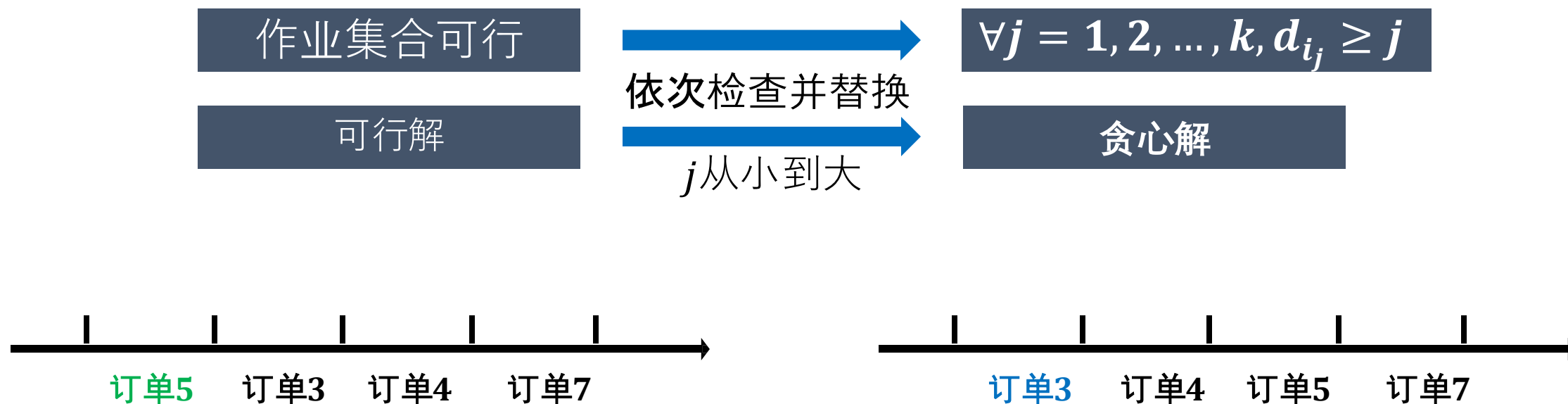
贪心解



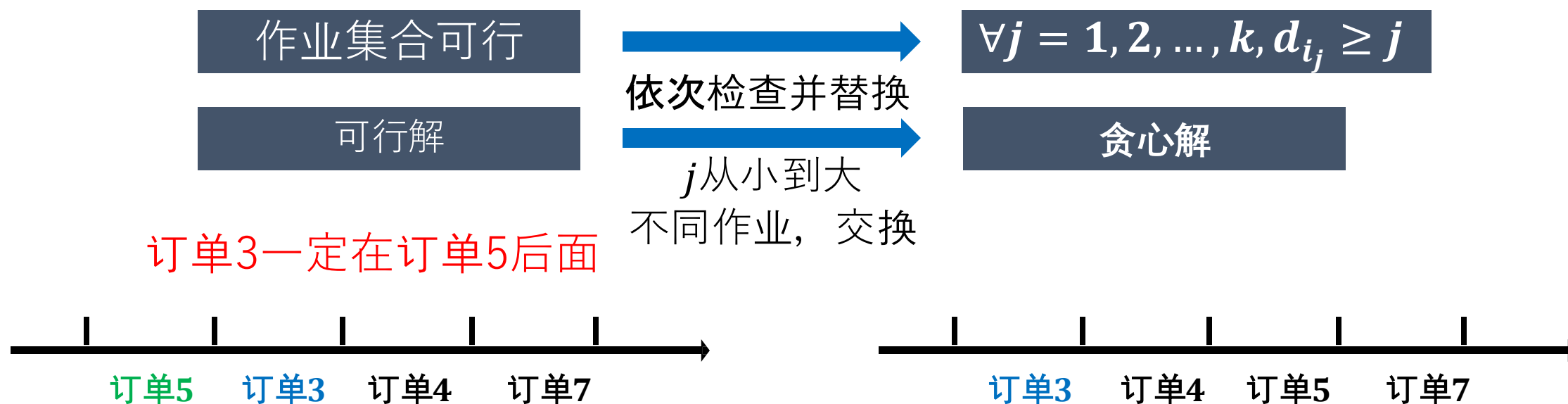
- 贪心策略：
 - 按**期限**升序对作业进行排列



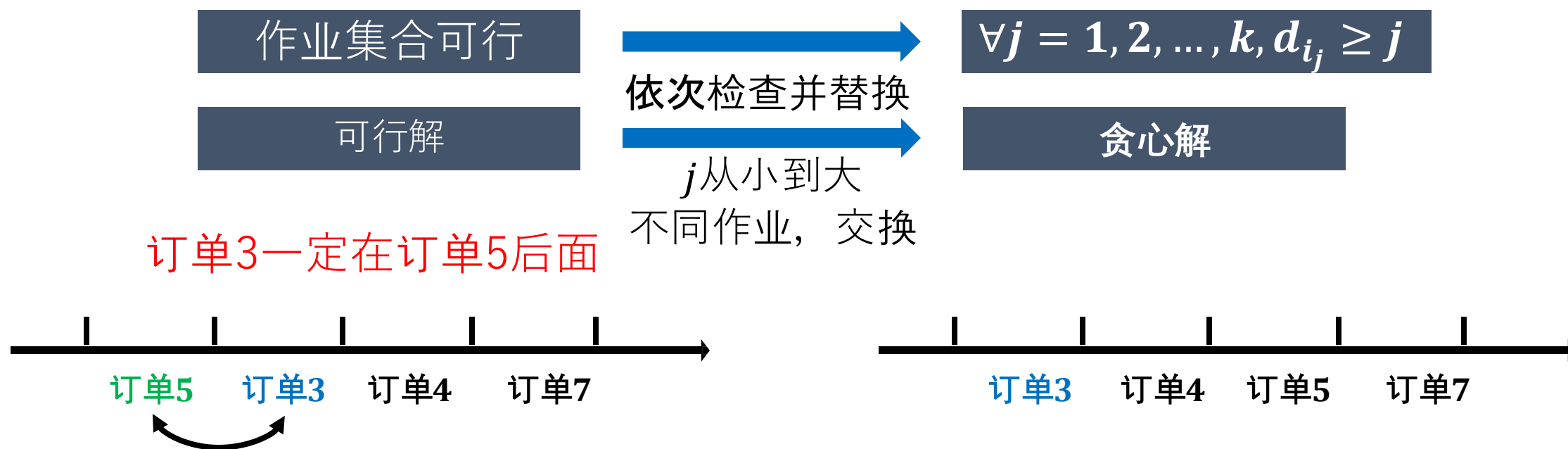
- 贪心策略：
 - 按**期限**升序对作业进行排列



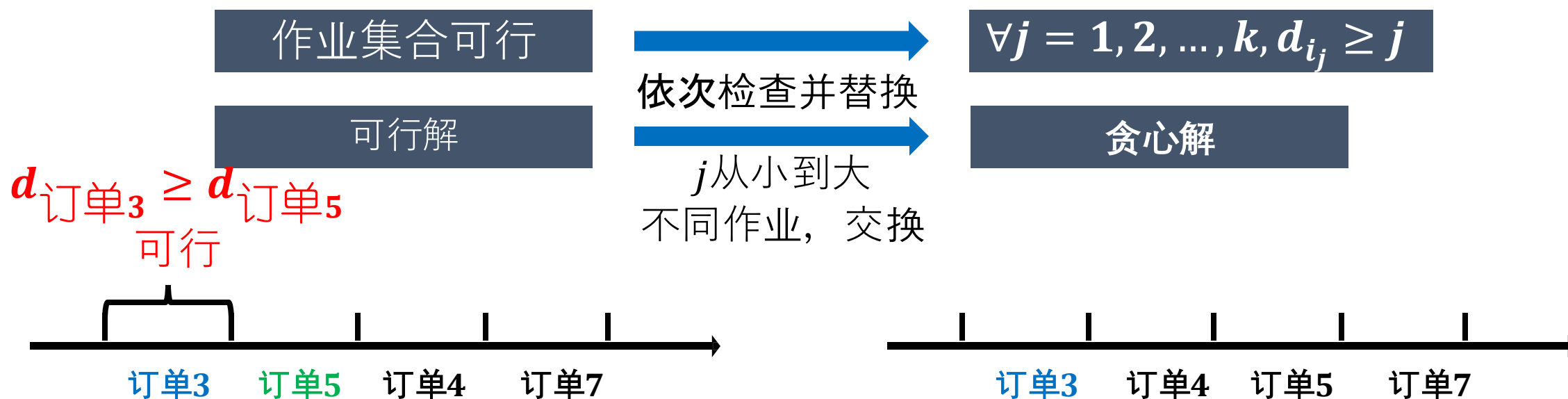
- 贪心策略：
 - 按**期限**升序对作业进行排列



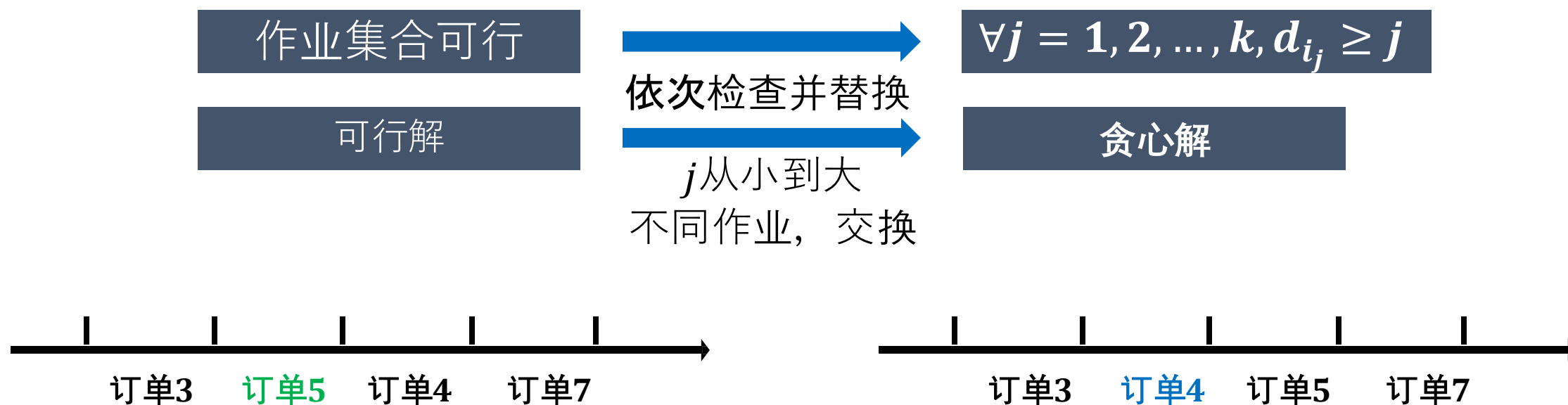
- 贪心策略：
 - 按**期限**升序对作业进行排列



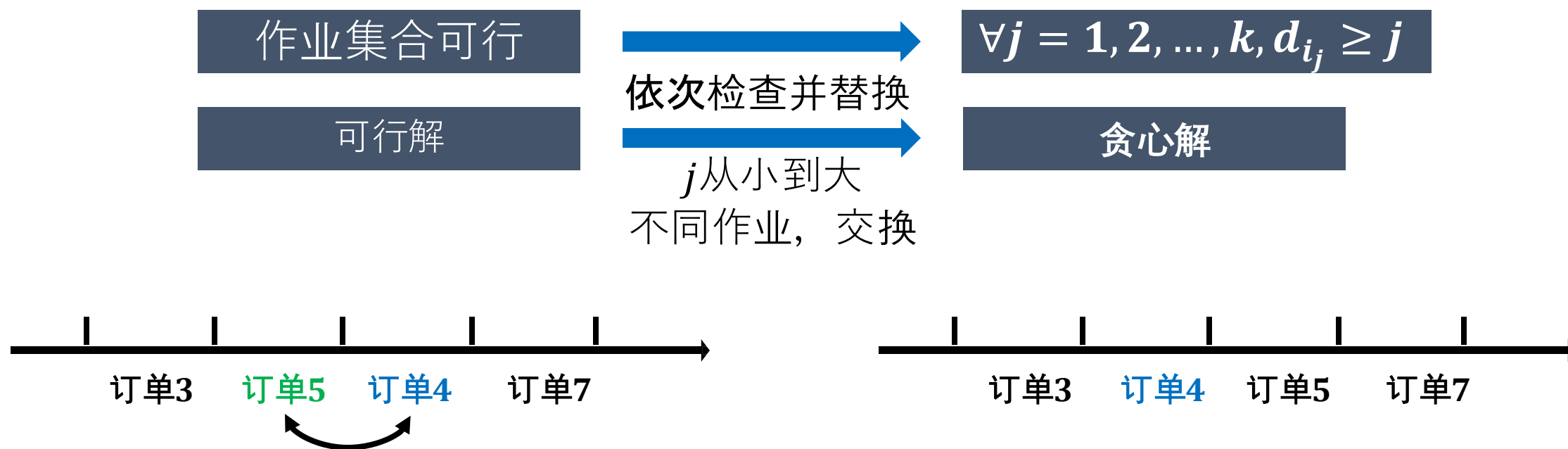
- 贪心策略：
 - 按**期限**升序对作业进行排列



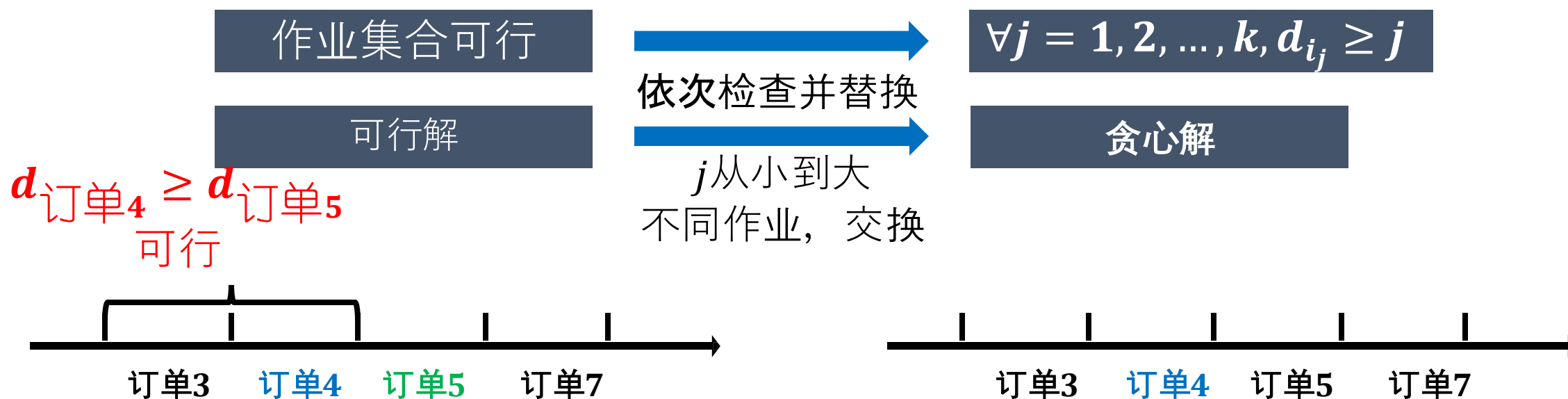
- 贪心策略：
 - 按**期限**升序对作业进行排列



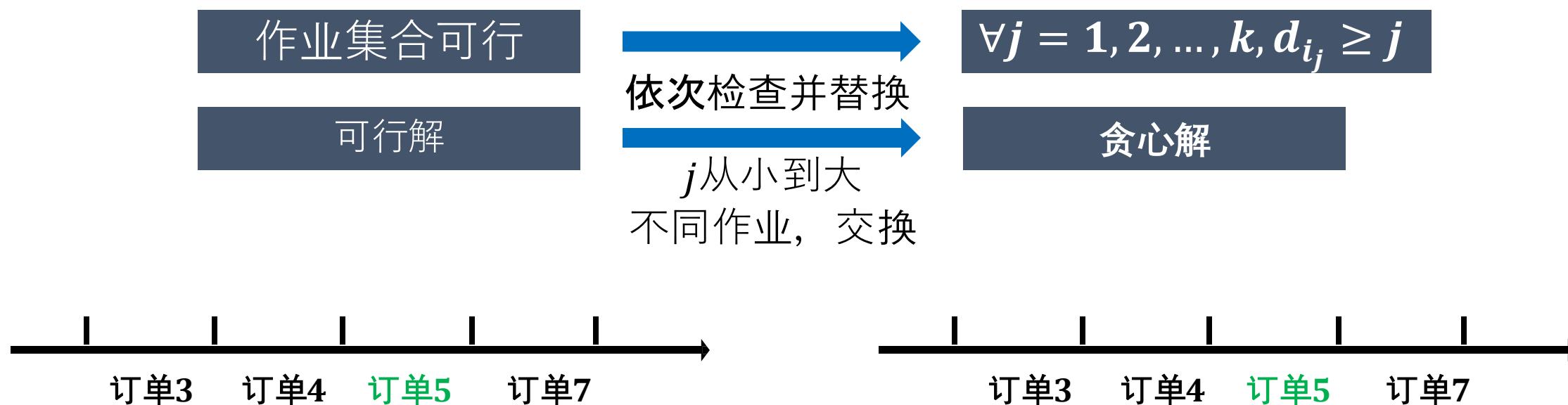
- 贪心策略：
 - 按**期限**升序对作业进行排列



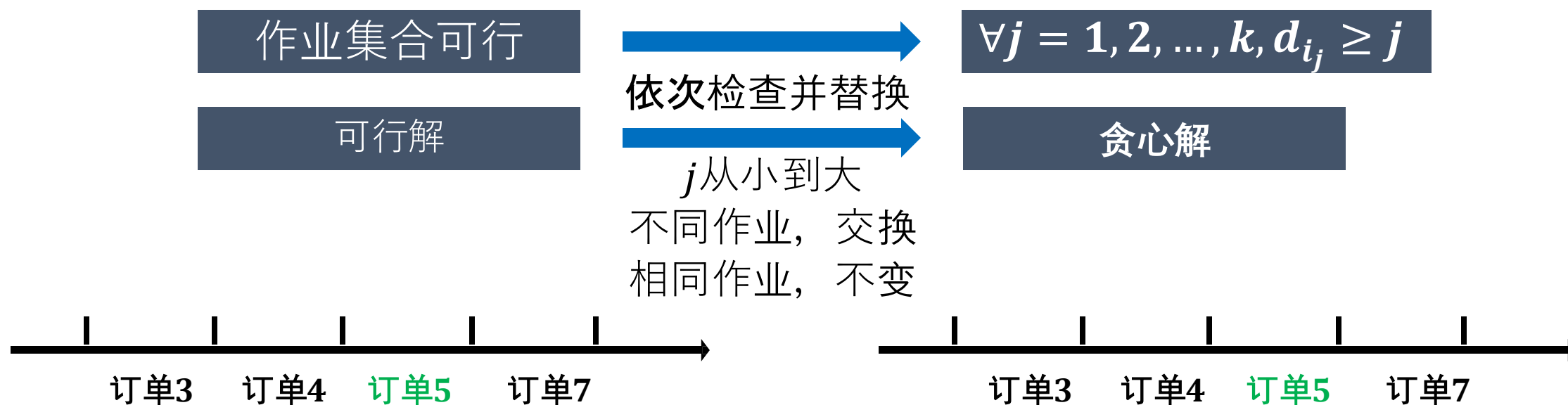
- 贪心策略：
 - 按**期限**升序对作业进行排列



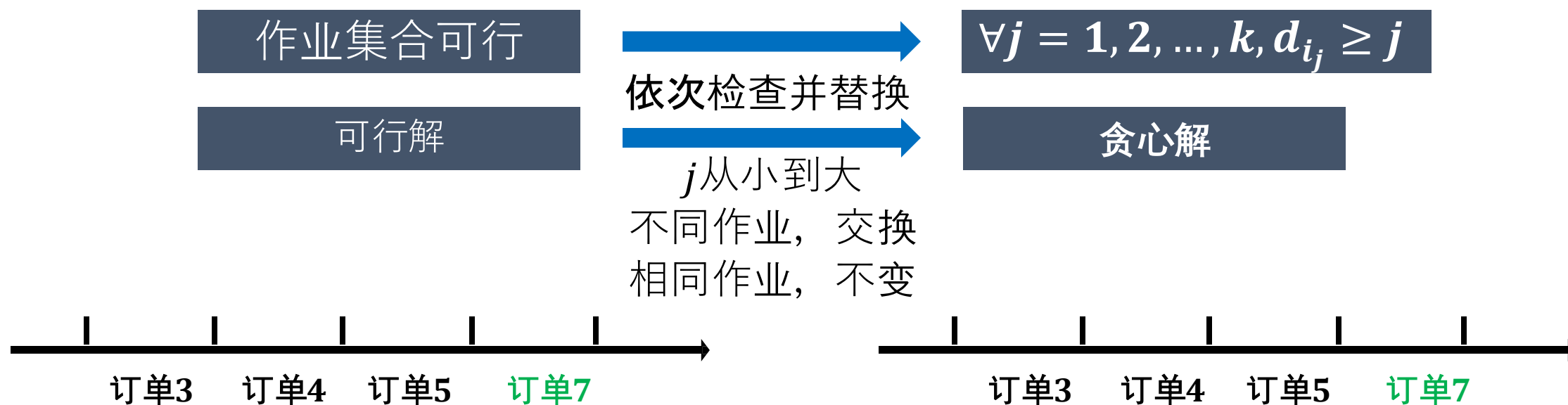
- 贪心策略：
 - 按**期限**升序对作业进行排列



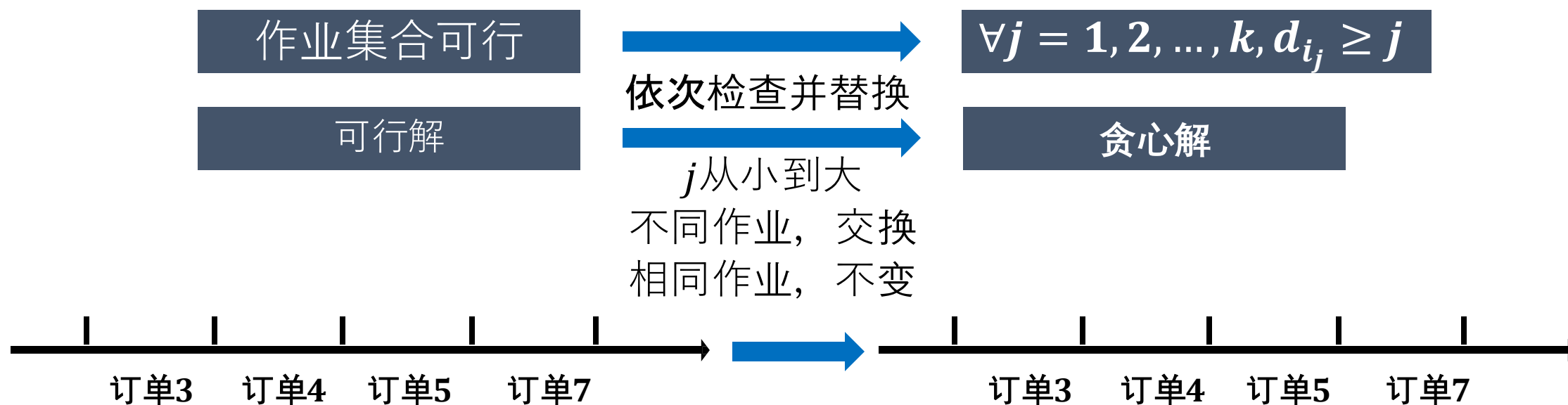
- 贪心策略：
 - 按**期限**升序对作业进行排列



- 贪心策略：
 - 按**期限**升序对作业进行排列



- 贪心策略：
 - 按**期限**升序对作业进行排列



- 假设已处理了 $i-1$ 个作业, 有 k 个作业已存入 J 中, 且 $d_{J(1)} \leq d_{J(2)} \leq \cdots \leq d_{J(k)}$
- 在 J 中**从后向前**为作业 i 寻找**合适的位置 $r+1$**
- i 插入 $r+1$ 位置后, J 中作业仍按照期限值从小到大排列, 且不违反期限值

$D(0) \leftarrow J(0) \leftarrow 0$; $k \leftarrow 1$; $J(1) \leftarrow 1$

for $i \leftarrow 2$ to n do

$r \leftarrow k$

 while $D(J(r)) > D(i)$ and $D(J(r)) \neq r$ do

$r \leftarrow r - 1$

 end

 if $D(J(r)) \leq D(i)$ and $D(i) > r$ then

 for $l \leftarrow k$ to $r+1$ by -1 do

$J(l+1) \leftarrow J(l)$

 end

$J(r+1) \leftarrow i$; $k \leftarrow k+1$;

 end

end

end JS

思考：该条件的
目的？

寻找 $r+1$ 位置

```
D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
```

判断可行性

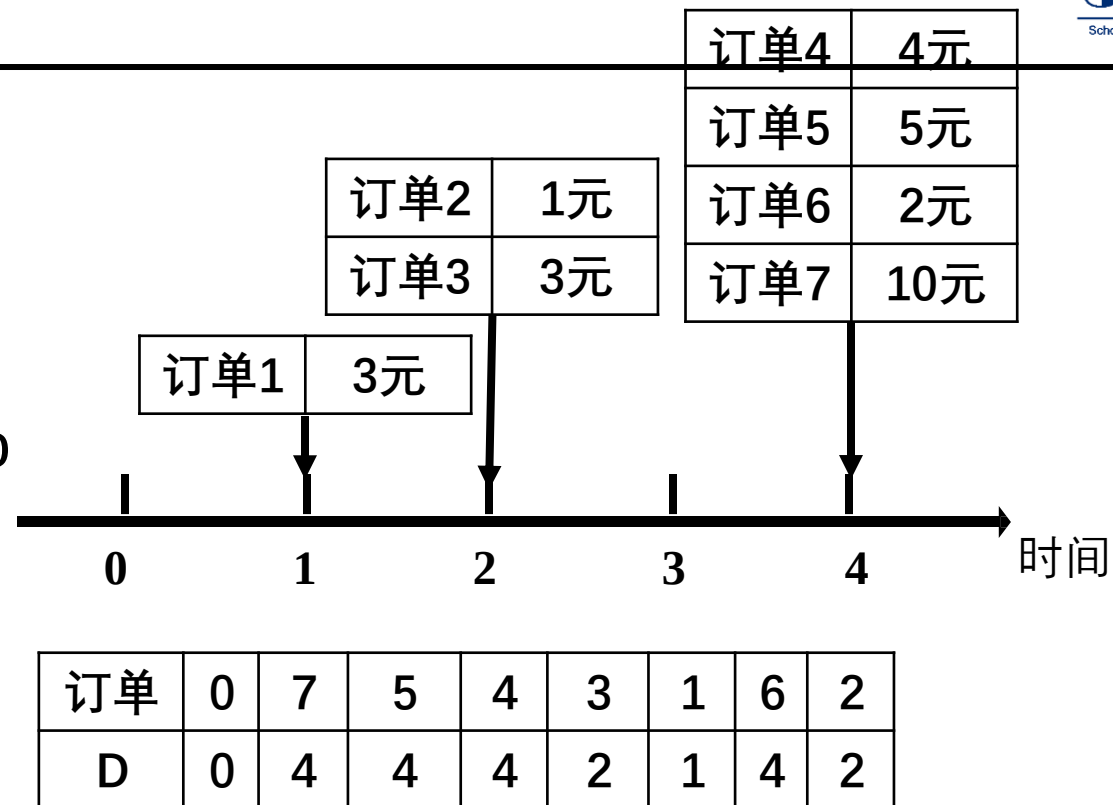
作业排序完整伪代码

```
D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
```

插入作业i

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



边界条件？

$D(0) \leftarrow J(0) \leftarrow 0; k \leftarrow 1; J(1) \leftarrow 1$

for $i \leftarrow 2$ to n do

$r \leftarrow k$

 while $D(J(r)) > D(i)$ and $D(J(r)) \neq r$ do

$r \leftarrow r - 1$

 end

 if $D(J(r)) \leq D(i)$ and $D(i) > r$ then

 for $l \leftarrow k$ to $r+1$ by -1 do

$J(l+1) \leftarrow J(l)$

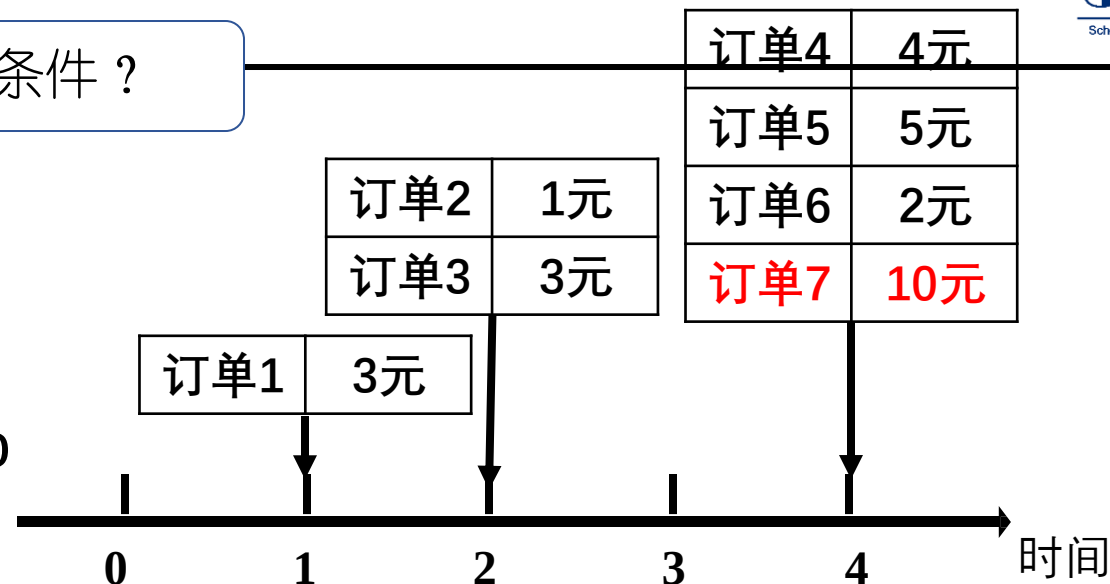
 end

$J(r+1) \leftarrow i; k \leftarrow k+1;$

 end

end

end JS



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

订单排列	0	7
D	0	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do

```

```

  r ← k

```

```

  while  $D(J(r)) > D(i)$  and  $D(J(r)) \neq r$  do
    r ← r - 1
  end

```

```

end

```

```

  if  $D(J(r)) \leq D(i)$  and  $D(i) > r$  then

```

```

    for l ← k to r+1 by -1 do

```

```

      J(l+1) ← J(l)
    end

```

```

  end

```

```

  J(r+1) ← i; k ← k+1;
end

```

```

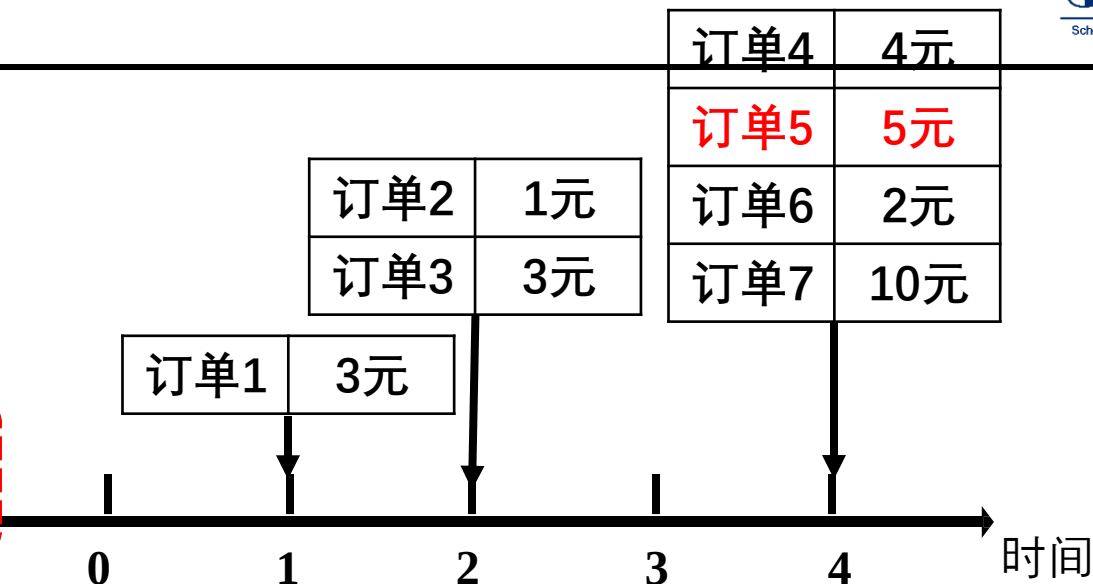
end

```

```

end JS

```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

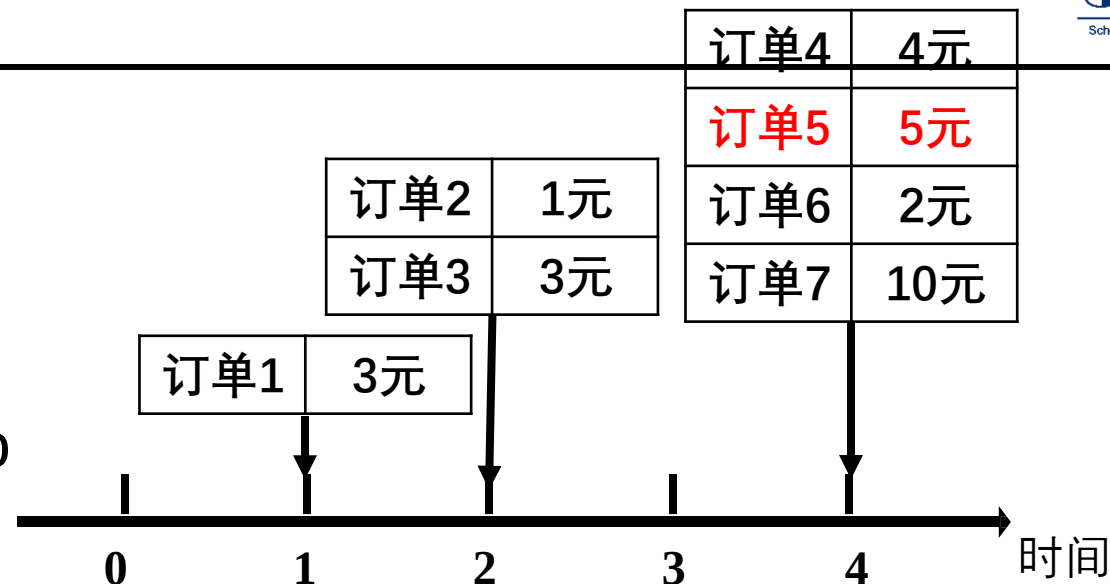
$i=2, D(i)=4, k=1$

$r=1$

订单排列	0	7
D	0	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

$i=2, D(i)=4, k=1$

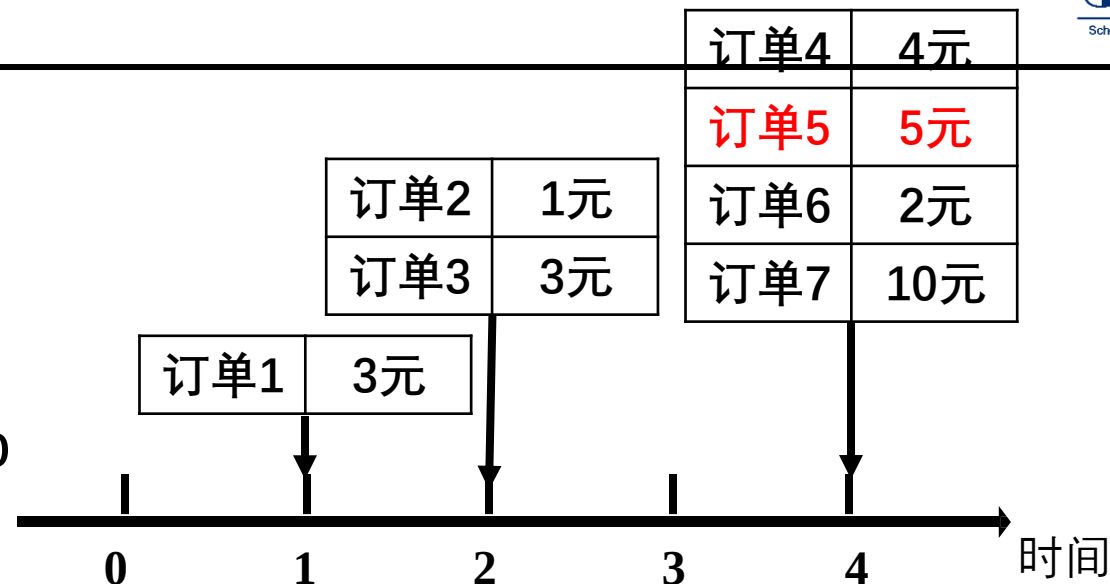
$r=1$

满足条件, 插入订单5

订单排列	0	7
D	0	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

订单排列	0	7	5
D	0	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do

```

```

    r ← k

```

```

    while  $D(J(r)) > D(i)$  and  $D(J(r)) \neq r$  do
        r ← r - 1
    end

```

```

end

```

```

    if  $D(J(r)) \leq D(i)$  and  $D(i) > r$  then

```

```

        for l ← k to r+1 by -1 do

```

```

            J(l+1) ← J(l)

```

```

        end

```

```

        J(r+1) ← i; k ← k+1;

```

```

    end

```

```

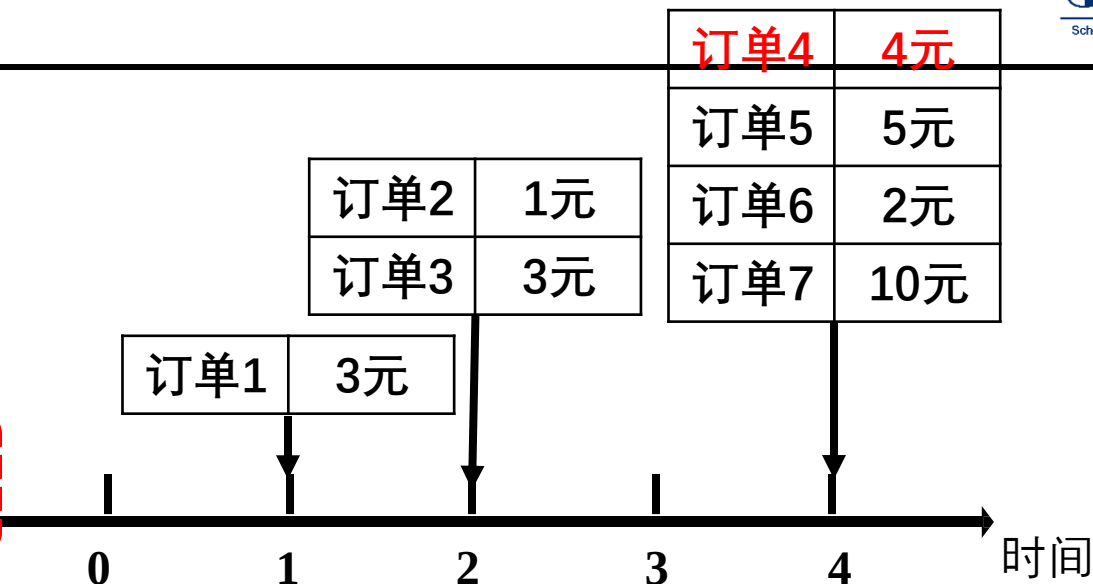
end

```

```

end JS

```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

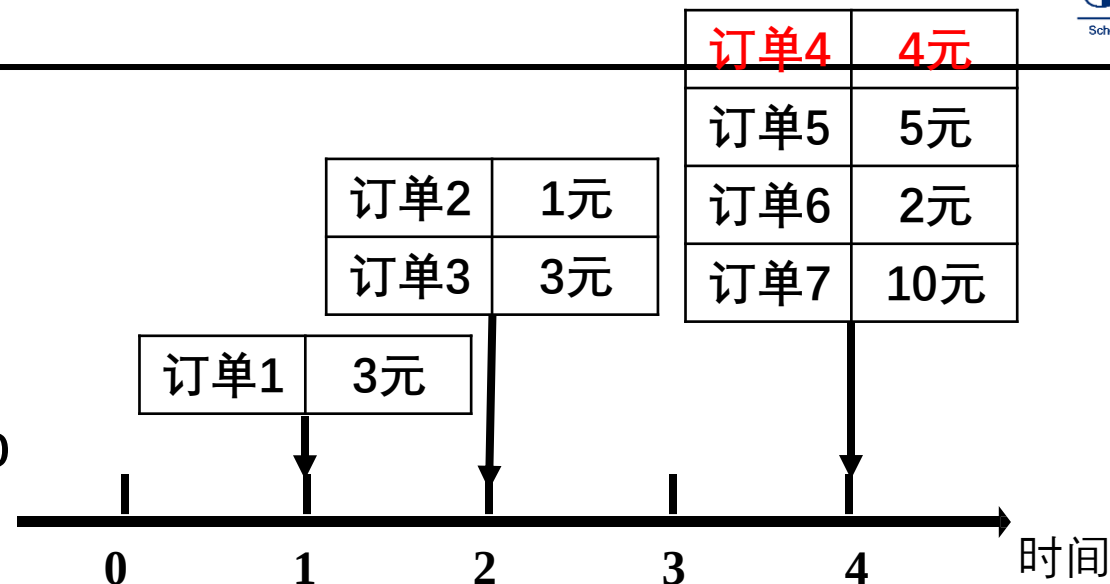
$i=3, D(i)=4, k=2$

$r=2$

订单排列	0	7	5
D	0	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

$i=3, D(i)=4, k=2$

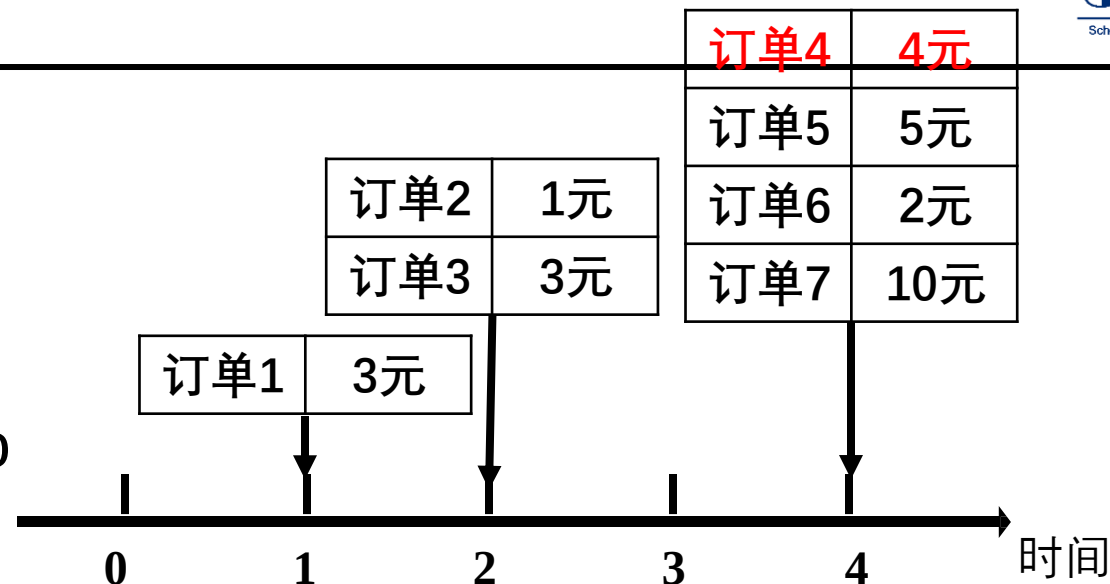
$r=2$

满足条件，插入订单4

订单排列	0	7	5
D	0	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

订单排列	0	7	5	4
D	0	4	4	4

```
D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
```

```
  r ← k
```

```
  while  $D(J(r)) > D(i)$  and  $D(J(r)) \neq r$  do
    r ← r - 1
```

```
  end
```

```
  if  $D(J(r)) \leq D(i)$  and  $D(i) > r$  then
```

```
    for l ← k to r+1 by -1 do
```

```
      J(l+1) ← J(l)
```

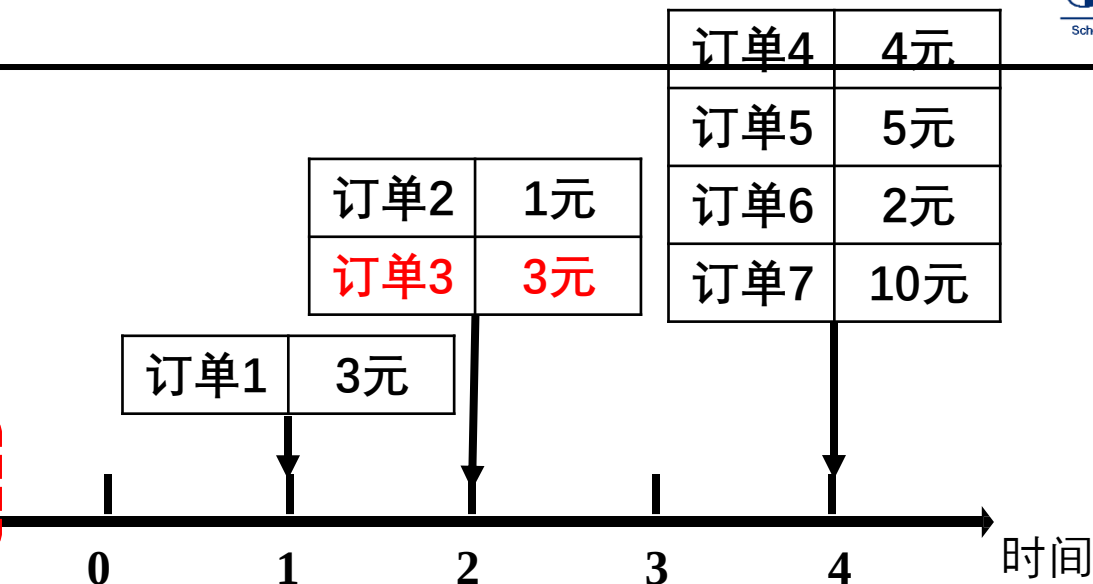
```
    end
```

```
    J(r+1) ← i; k ← k+1;
```

```
  end
```

```
end
```

```
end JS
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

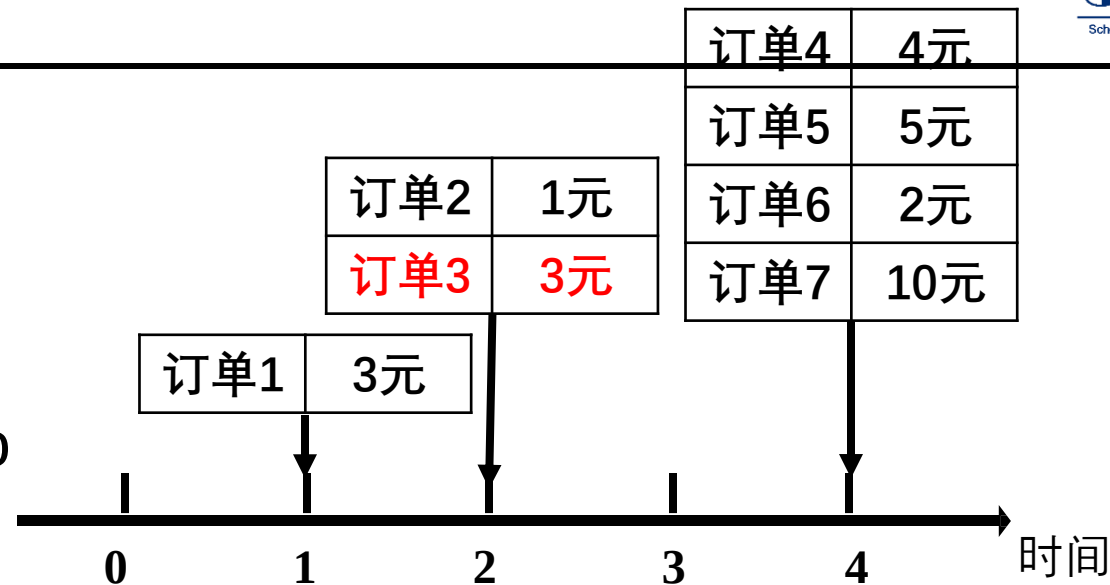
$i=4, D(i)=2, k=3$

$r=0$

订单排列	0	7	5	4
D	0	4	4	4


```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

$i=4, D(i)=2, k=3$

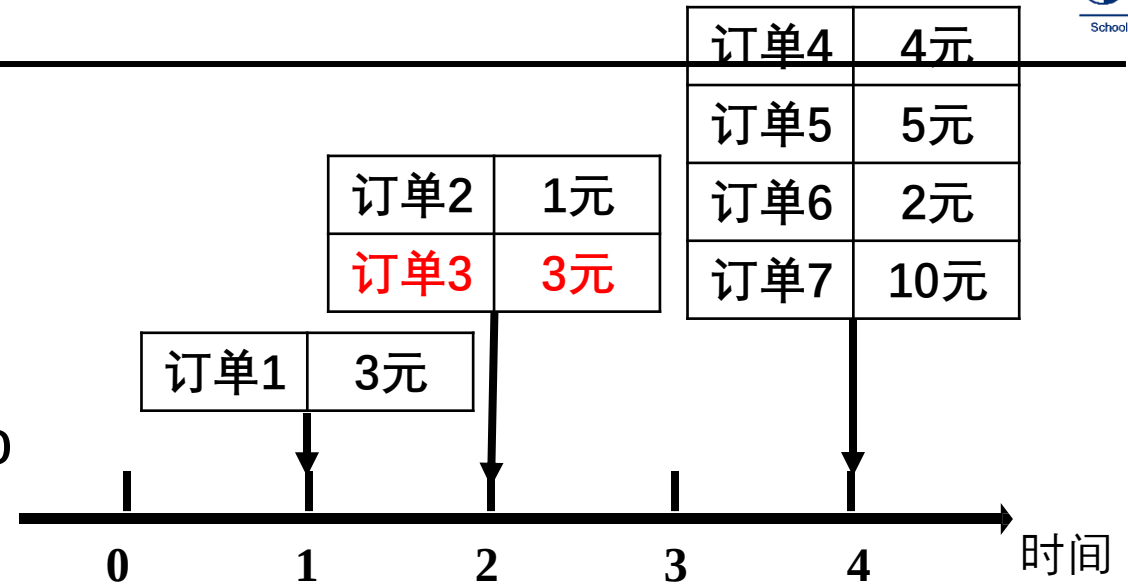
$r=0$

满足条件, 插入订单3

订单排列	0	7	5	4
D	0	4	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

订单排列	0	3	7	5	4
D	0	2	4	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do

```

```

  r ← k

```

```

  while  $D(J(r)) > D(i)$  and  $D(J(r)) \neq r$  do
    r ← r - 1
  end

```

```

end

```

```

  if  $D(J(r)) \leq D(i)$  and  $D(i) > r$  then

```

```

    for l ← k to r+1 by -1 do

```

```

      J(l+1) ← J(l)
    end

```

```

  end

```

```

  J(r+1) ← i; k ← k+1;

```

```

end

```

```

end

```

```

end JS

```

订单 r 不可再往后移!

1 3元



订单4	4元
订单5	5元
订单6	2元
订单7	10元

订单2	1元
订单3	3元

订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

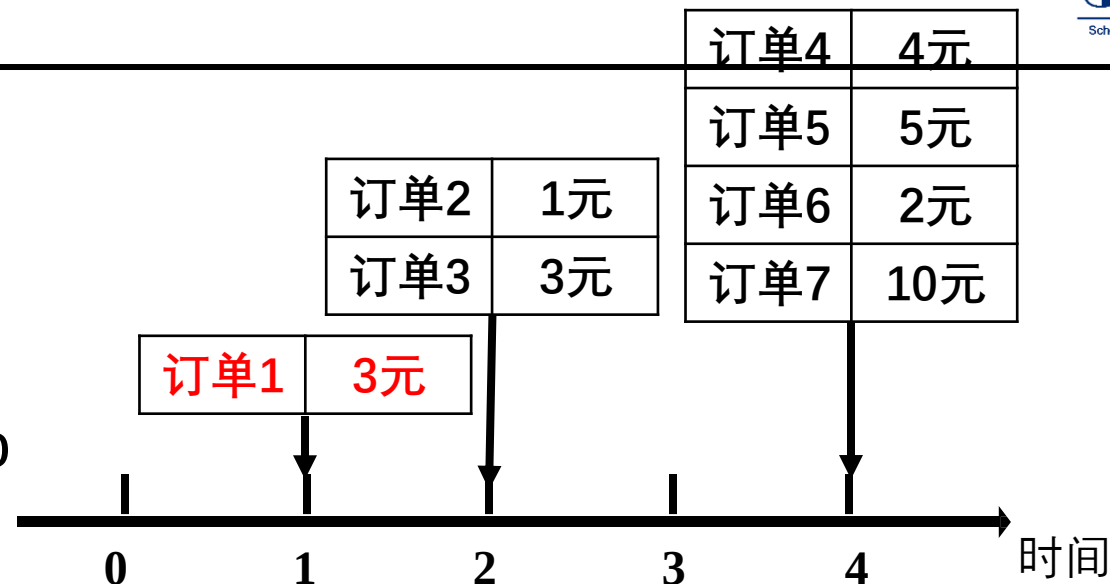
$i=5, D(i)=1, k=4$

$r=4$

订单排列	0	3	7	5	4
D	0	2	4	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

$i=5, D(i)=1, k=4$

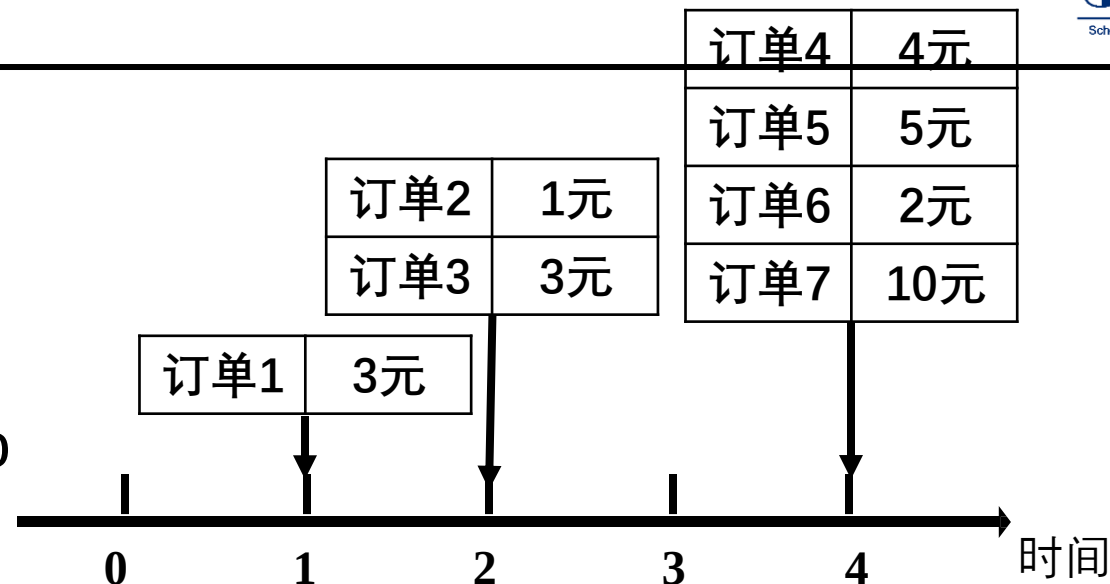
$r=4$

不满足条件！且后续订单都不可能满足，返回结果

订单排列	0	3	7	5	4
D	0	2	4	4	4

```

D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
    r ← k
    while D(J(r)) > D(i) and D(J(r)) ≠ r do
        r ← r - 1
    end
    if D(J(r)) ≤ D(i) and D(i) > r then
        for l ← k to r+1 by -1 do
            J(l+1) ← J(l)
        end
        J(r+1) ← i; k ← k+1;
    end
end
end JS
    
```



订单	0	7	5	4	3	1	6	2
D	0	4	4	4	2	1	4	2

订单排列	0	3	7	5	4
D	0	2	4	4	4

```
D(0) ← J(0) ← 0; k ← 1; J(1) ← 1
for i ← 2 to n do
  r ← k
  while D(J(r)) > D(i) and D(J(r)) ≠ r do
    r ← r - 1
  end
  if D(J(r)) ≤ D(i) and D(i) > r then
    for l ← k to r+1 by -1 do
      J(l+1) ← J(l)
    end
    J(r+1) ← i; k ← k+1;
  end
end
end JS
```

- 两个关键参数:
 - 作业数 n
 - J的最终作业数 $s \leq n$
- 内层while至多循环 $k \leq s \leq n$ 次
- 外层for执行循环 $n-1$ 次
- JS算法的时间复杂度为 $O(n^2)$

思考1: 算法JS的时间复杂度? $O(n^2)$ 思考2: 能否降低时间复杂度?

- 若还没给作业 i 分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则 i 不计入 J 中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



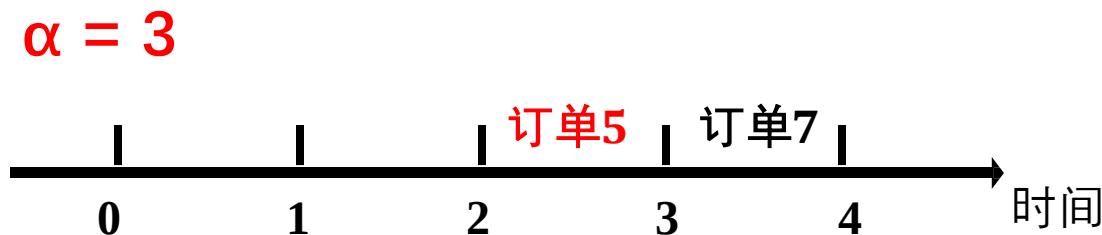
- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



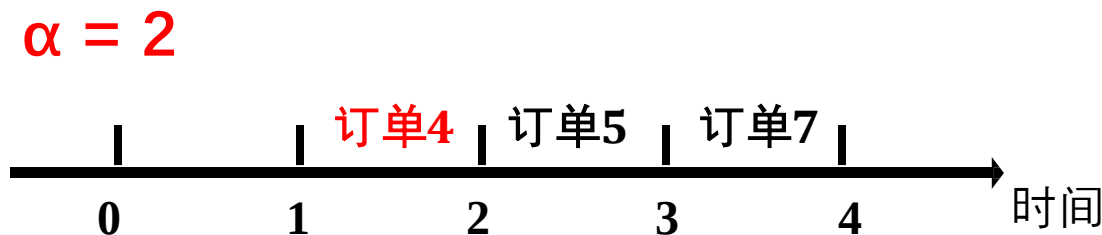
- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



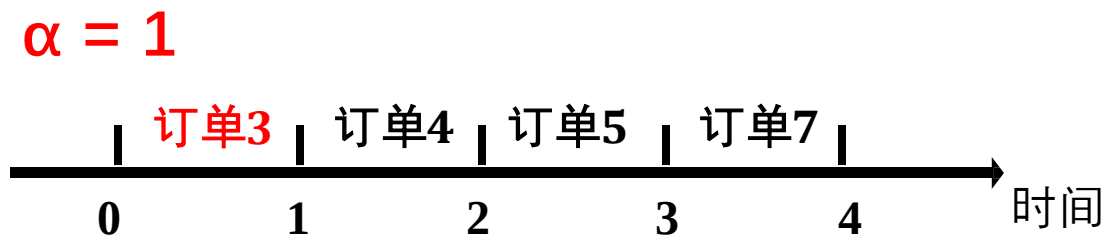
- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

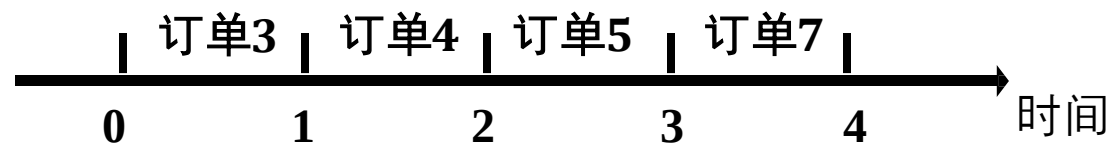
订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在：去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2

不存在这样的 α !



- 若还没给作业*i*分配处理时间, 则分给它时间片 $[\alpha - 1, \alpha]$, 其中 α 是不大于 d_i 的最大空时间片。若不存在这样的 α , 则*i*不计入*J*中。
- 旨在: 去掉内层while循环, 算法的复杂度由 $O(n^2)$ 降低到 $O(n \log n)$

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



思考: 如何找到 α ?

时间换空间

定义一个数组记录期限*d*当前可用的最大空时间片?

- 定义一维数组 F , $F(d_j)$ 表示时间期限为 d_j 的作业 j 当前可用的最大可用空时间片。

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2

期限	1	2	3	4
F	1	2	3	4



- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



期限	1	2	3	4
F	1	2	3	4



期限	1	2	3	4
F	1	2	3	3

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



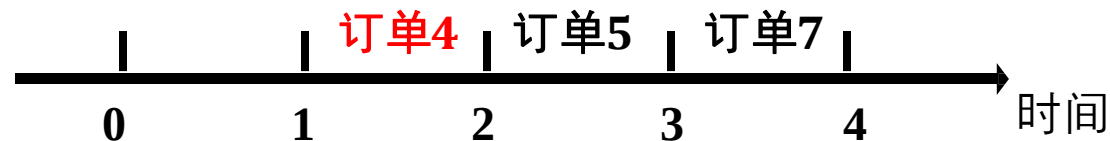
期限	1	2	3	4
F	1	2	3	3



期限	1	2	3	4
F	1	2	2	2

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



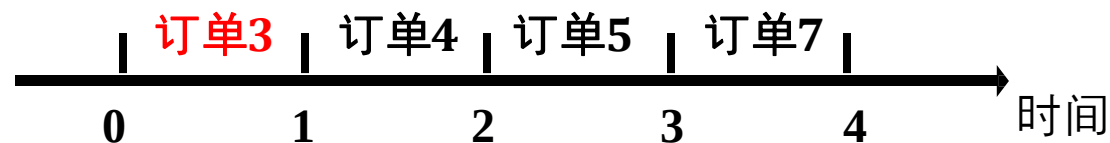
期限	1	2	3	4
F	1	2	2	2



期限	1	2	3	4
F	1	1	1	1

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2



期限	1	2	3	4
F	1	1	1	1

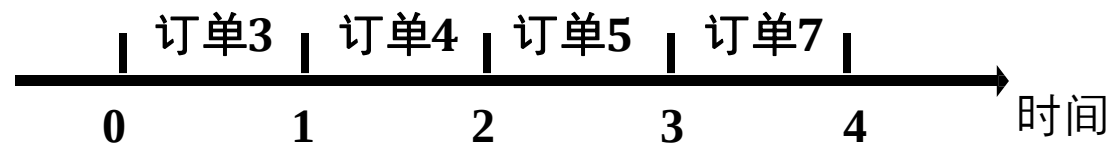


期限	1	2	3	4
F	0	0	0	0

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。

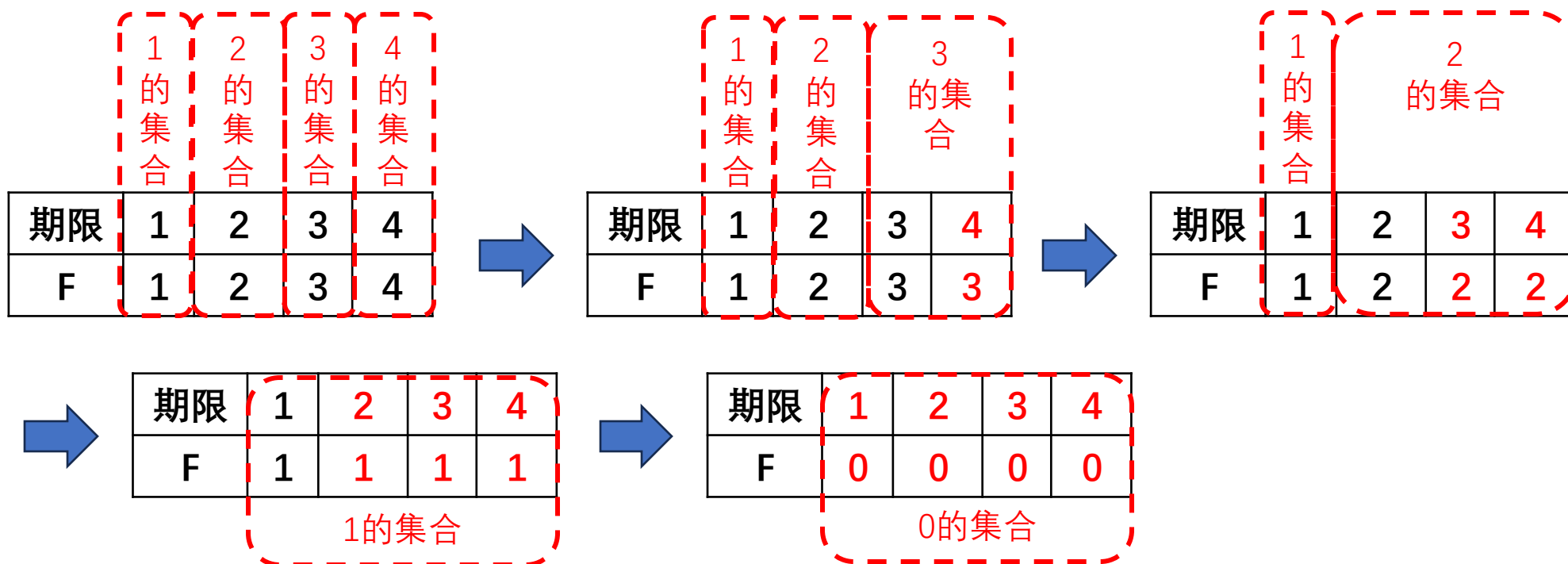
订单	7	5	4	3	1	6	2
D	4	4	4	2	1	4	2

期限	1	2	3	4
F	0	0	0	0



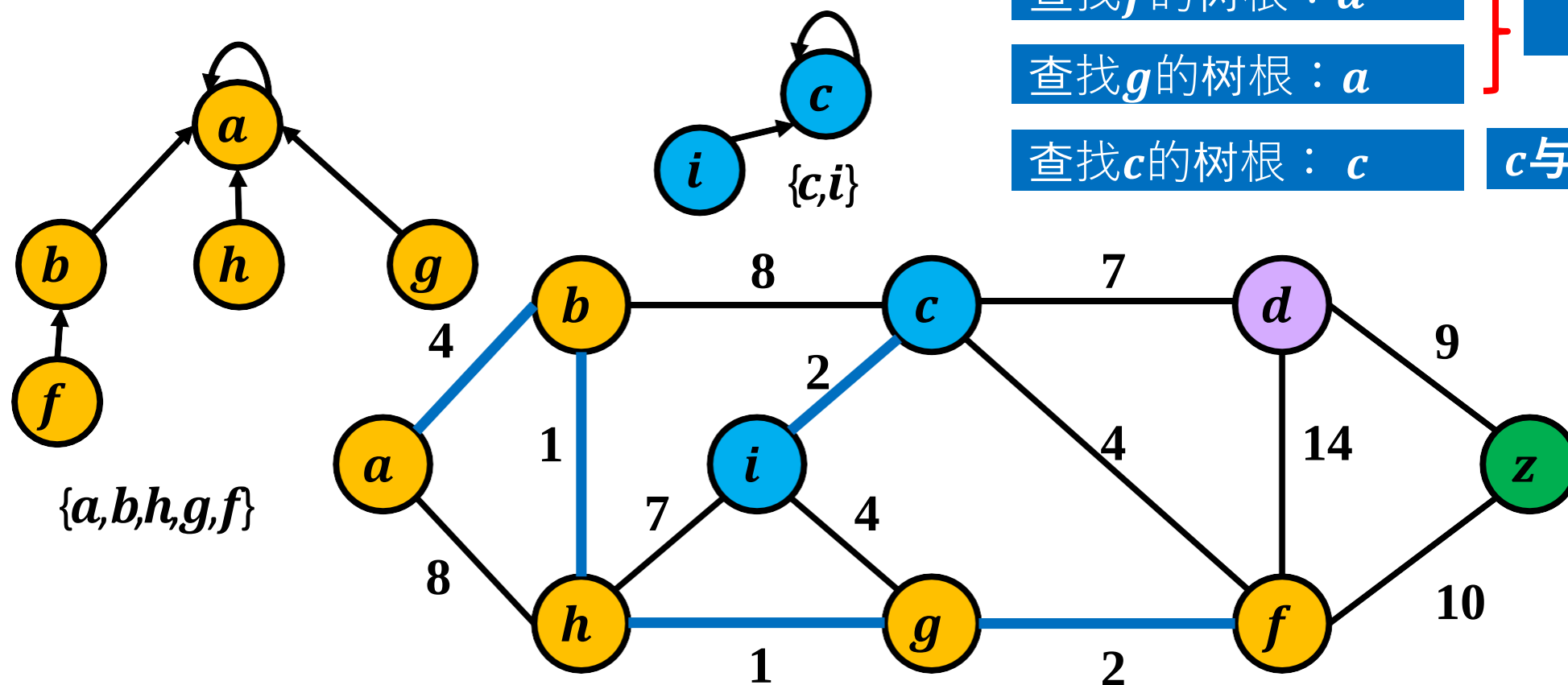
$F(\text{订单}j) = 0$ ，无法插入

- 定义一维数组F， $F(d_j)$ 表示时间期限为 d_j 的作业j当前可用的最大可用空时间片。
- 直接维护F，复杂度依然为 $O(n)$ ，需要更高效的数据结构！
- 把F看作多个最大可用空时间的不相交集合并！

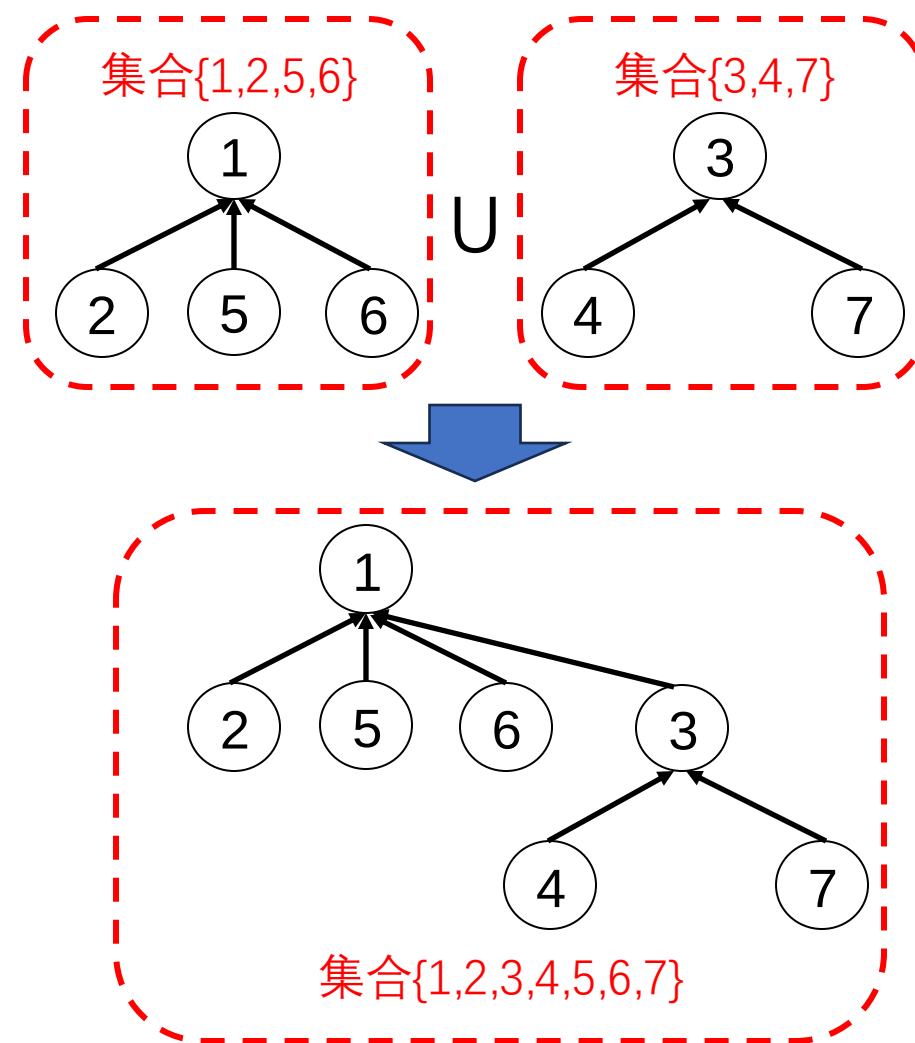


不相交集集合回顾

- 初始化集合：创建根结点，并设置一条指向自身的边
- 判定顶点是否在同一集合：回溯查找树根，检查树根是否相同



- 把F看作多个最大可用空时间的不相交集！
- 采用**不相交集**
 - 集合的合并操作
 - **Union-Set(i, j)**
 - 查询元素所在集合
 - **Find-Set(k)**
 - **Create-Set(x) : $O(1)$**
 - **Find-Set(x) : $O(\log n)$**
 - **Union-Set(x) : $O(\log n)$**



- 初始

- $p_1 \geq p_2 \cdots \geq p_n$
- $b = \min\{n, \max\{d_j\}\}$
- $F(i) \leftarrow i$

F的大小不超过b

F记录每个集合对应的最大可用空时间

- 依次检验每个作业i

- 寻找D(i)所在树的根节点j
- 如果 $F(j) \neq 0$, $F(j)$ 时间片可以分配给作业i, 因此, 将i并入到作业集合J中,
- 寻找 $F(j)-1$ 所在树的根节点l, 将l和j所在的两个树合并到一起。
- $F(j) \leftarrow F(l)$

Find-Set(min(n, D(i)))

Union-Set(l, j)

更新合并后集合的最大可用空时间

思考：为何？有其他实现方法吗？

基于不相交集集合的优化算法

// 找最优解 $J = J(1), \dots, J(k)$, 已知 $p_1 \geq p_2 \cdots \geq p_n$, $b = \min\{n, \max\{d_j\}\}$

for $i \leftarrow 0$ to b do

$F(i) \leftarrow i$;

Create-Set(i)

end

$k \leftarrow 0$

for $i \leftarrow 1$ to n do

$j \leftarrow \text{Find-Set}(\min(n, D(i)))$

 if $F(j) \neq 0$ then

$k \leftarrow k + 1$; $J(k) \leftarrow i$

$l \leftarrow \text{Find-Set}(F(j) - 1)$

Union-Set(l, j)

$F(j) \leftarrow F(l)$

 end

end

End FJS

构建不相交集集合, 设置集合的性质

基于不相交集集合的优化算法

// 找最优解 $J=J(1), \dots, J(k)$, 已知 $p_1 \geq p_2 \cdots \geq p_n$, $b = \min\{n, \max\{d_j\}\}$

for $i \leftarrow 0$ to b do

$F(i) \leftarrow i$;

Create-Set(i)

end

$k \leftarrow 0$

for $i \leftarrow 1$ to n do

$j \leftarrow \text{Find-Set}(\min(n, D(i)))$

 if $F(j) \neq 0$ then

$k \leftarrow k + 1$; $J(k) \leftarrow i$

$l \leftarrow \text{Find-Set}(F(j) - 1)$

Union-Set($\min(l, j)$)

$F(j) \leftarrow F(l)$

 end

end

End FJS

寻找集合根节点，获得最大可用空时间

基于不相交集集合的优化算法

// 找最优解 $J = J(1), \dots, J(k)$, 已知 $p_1 \geq p_2 \cdots \geq p_n$, $b = \min\{n, \max\{d_j\}\}$

for $i \leftarrow 0$ to b do

$F(i) \leftarrow i$;

Create-Set(i)

end

$k \leftarrow 0$

for $i \leftarrow 1$ to n do

$j \leftarrow \text{Find-Set}(\min(n, D(i)))$

 if $F(j) \neq 0$ then

$k \leftarrow k + 1$; $J(k) \leftarrow i$

$l \leftarrow \text{Find-Set}(F(j) - 1)$

Union-Set($\min(l, j)$)

$F(j) \leftarrow F(l)$

 end

end

End FJS

如果还有空余时间片，插入作业

基于不相交集集合的优化算法

// 找最优解 $J = J(1), \dots, J(k)$, 已知 $p_1 \geq p_2 \cdots \geq p_n$, $b = \min\{n, \max\{d_j\}\}$

for $i \leftarrow 0$ to b do

$F(i) \leftarrow i$;

Create-Set(i)

end

$k \leftarrow 0$

for $i \leftarrow 1$ to n do

$j \leftarrow \text{Find-Set}(\min(n, D(i)))$

 if $F(j) \neq 0$ then

$k \leftarrow k + 1; J(k) \leftarrow i$

$l \leftarrow \text{Find-Set}(F(j) - 1)$

Union-Set($\min(l, j)$)

$F(j) \leftarrow F(l)$

 end

end

End FJS

合并集合，设置新集合的性质

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```
for i ← 0 to b do
  F(i) ← i;
  Create-Set(i)
end
```

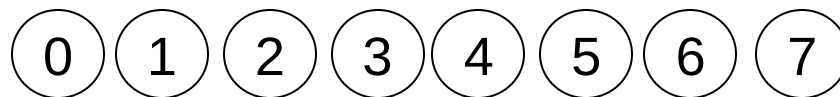
考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

无

0 1 2 3 4 5 6 7

不相交集合



J

ϕ

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 4, j = 4$
 $l = 3$
 $\text{Union-Set}(3, 4)$
 $F(4) \cup F(3)$

考虑
作业

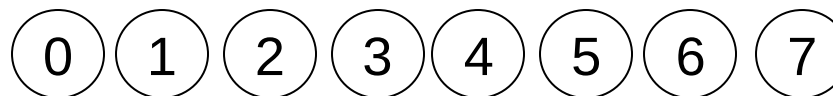
不相交集合并

J

无

F(0)(1)(2)(3)(4)(5)(6)(7)

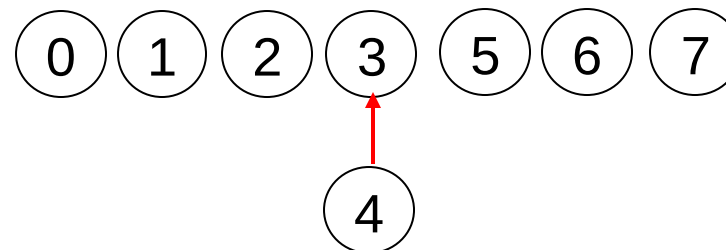
0 1 2 3 4 5 6 7



$\emptyset$

1

0 1 2 3 **3** 5 6 7



{1}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 2, j = 2$
 $l = 1$
 $\text{Union-Set}(1, 2)$
 $F(2) \cup F(1)$

考虑
作业

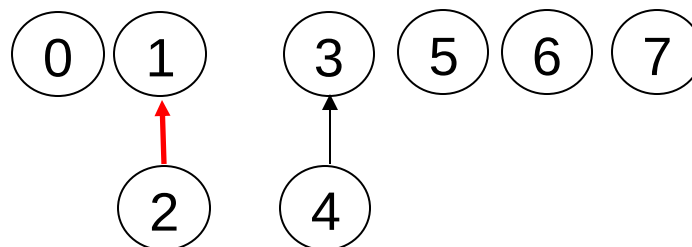
F(0)(1)(2)(3)(4)(5)(6)(7)

2

0 1 **1** 3 3 5 6 7

不相交集集合

J



{1,2}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 4, j = 3$
 $l = 1$
 $\text{Union-Set}(1, 3)$
 $F(3) \cup F(1)$

考虑
作业

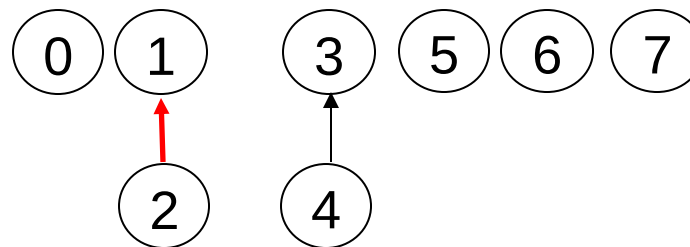
不相交集合并

J

2

F(0)(1)(2)(3)(4)(5)(6)(7)

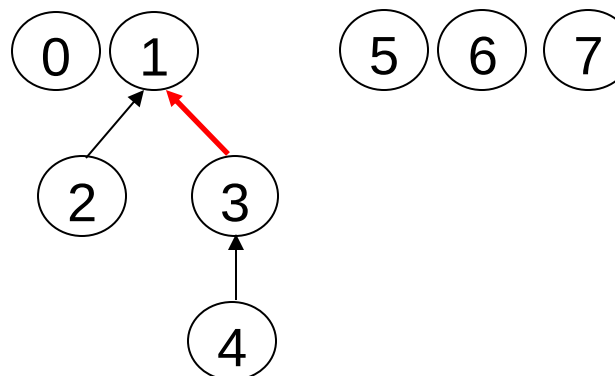
0 1 **1** 3 3 5 6 7



{1,2}

3

0 1 1 **1** 3 5 6 7



{1,2,3}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

考虑
作业

4

$F(0)(1)(2)(3)(4)(5)(6)(7)$

0 0 1 1 3 5 6 7

思考：为何如此合并？

```

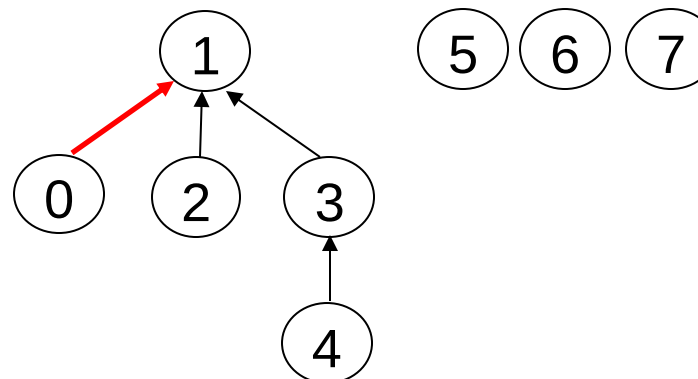
j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

不相交集合

$D(i) = 3, j = 1$
 $l = 0$
 Union-Set(0, 1)
 $F(1) \cup F(0)$

J

{1, 2, 3, 4}



- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 4, j = 1$
 $F(j) = 0$

考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

4

0 0 1 1 3 5 6 7

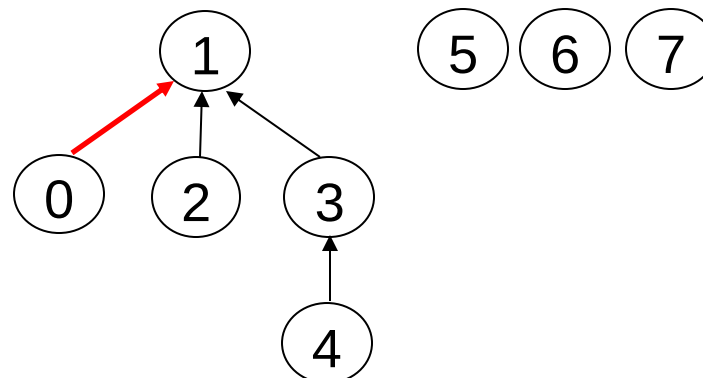
思考：为何如此合并？

5
(舍弃)

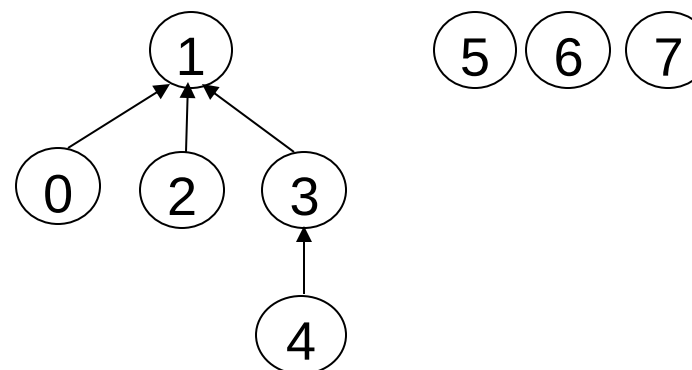
0 0 1 1 3 5 6 7

不相交集合并

J



{1,2,3,4}



{1,2,3,4}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 7, j = 7$
 $l = 6$
 $\text{Union-Set}(6, 7)$
 $F(7) \cup F(6)$

考虑
作业

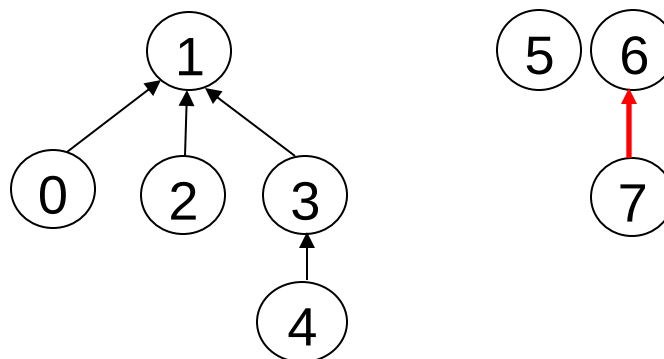
F(0)(1)(2)(3)(4)(5)(6)(7)

6

0 0 1 1 3 5 6 6

不相交集

J



{1,2,3,4,6}

- 设 $n=7$,
- $(p_1, p_2, \dots, p_7) = (35, 30, 25, 20, 15, 10, 5)$
- $(d_1, d_2, \dots, d_7) = (4, 2, 4, 3, 4, 8, 3)$

```

j ← Find-Set(min(n, D(i)))
if F(j) ≠ 0 then
    k ← k + 1; J(k) ← i
    l ← Find-Set(F(j) - 1)
    Union-Set(l, j)
    F(j) ← F(l)
end
    
```

$D(i) = 7, j = 7$
 $l = 6$
 $\text{Union-Set}(6, 7)$
 $F(7) \cup F(6)$

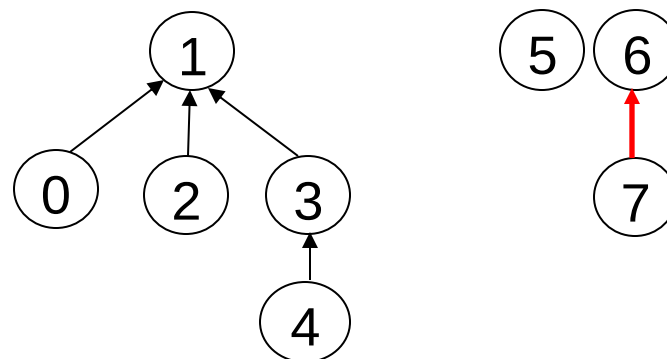
考虑
作业

F(0)(1)(2)(3)(4)(5)(6)(7)

6

0 0 1 1 3 5 6 6

不相交集集合



J

{1, 2, 3, 4, 6}

7

0 0 1 1 3 5 6 6

(舍弃)

同上

最优解 $J = \{1, 2, 3, 4, 6\}$, 处理次序 4, 2, 3, 1, 6, 效益值 120

// 找最优解 $J = J(1), \dots, J(k)$, 已知 $p_1 \geq p_2 \dots \geq p_n$, $b = \min\{n, \max\{d_j\}\}$

for $i \leftarrow 0$ to **b** do

$F(i) \leftarrow i$;

Create-Set(i)

end

$k \leftarrow 0$

for $i \leftarrow 1$ to n do

$j \leftarrow \text{Find-Set}(\min(n, D(i)))$

 if $F(j) \neq 0$ then

$k \leftarrow k + 1$; $J(k) \leftarrow i$

$l \leftarrow \text{Find-Set}(F(j) - 1)$

Union-Set(l, j)

$F(j) \leftarrow F(l)$

 end

end

End FJS

- 最多 $2n$ 次FIND-Set和 n 次UNION-Set
- **Find-Set(x): $O(\log n)$**
- **Union-Set(x): $O(\log n)$**
- 复杂度: **$O(n \log n)$**

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 最优前缀码问题
- 5.5 货郎担问题
- 5.6 小结

- 在计算机中，常用二进制串对不同字符进行编码

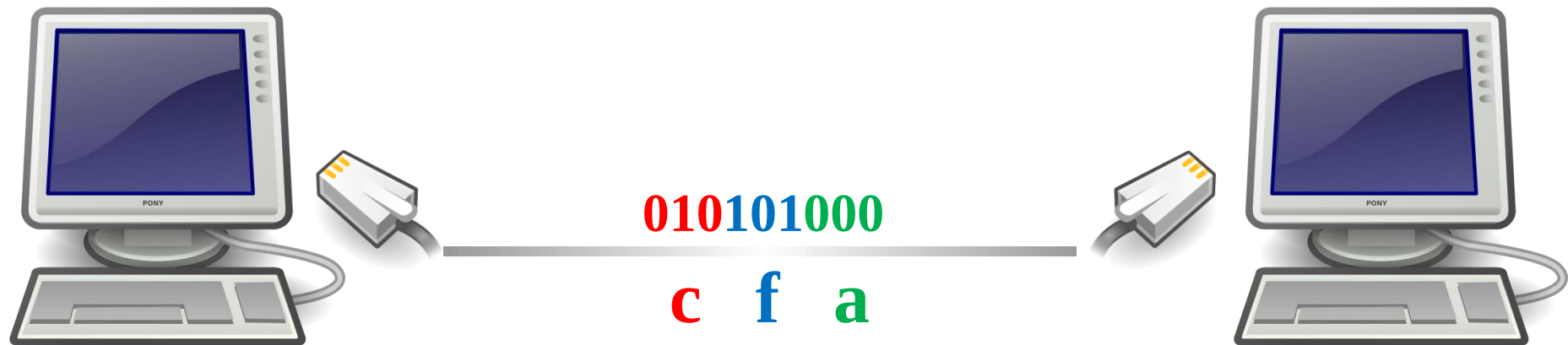
	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式	000	001	010	011	100	101



问题背景：字符编码

- 在计算机中，常用二进制串对不同字符进行编码

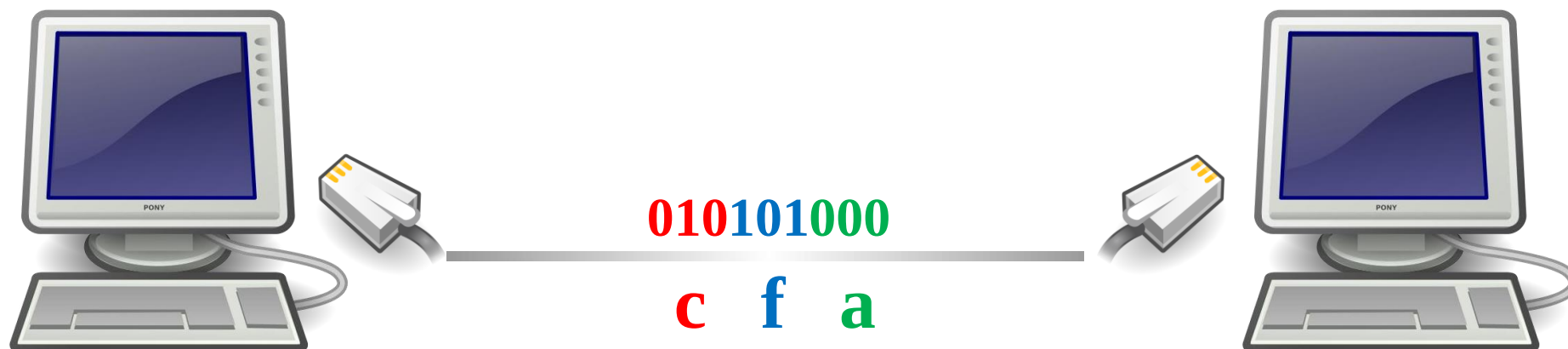
	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式	000	001	010	011	100	101



问题背景：字符编码

- 在计算机中，常用二进制串对不同字符进行编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式	000	001	010	011	100	101



问题：哪些编码方式是可行的？

问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 100

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 1001100

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cf****a*** → 1001100**0**

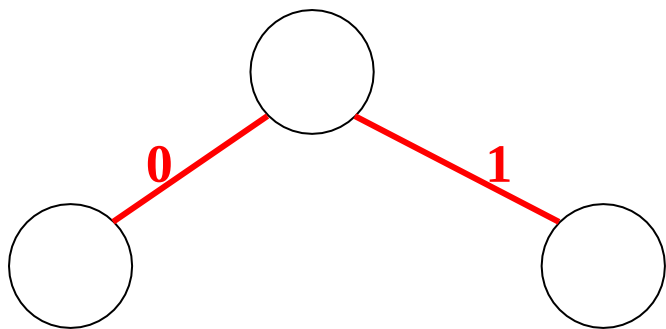
	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cf****a*** → 1001100**0**

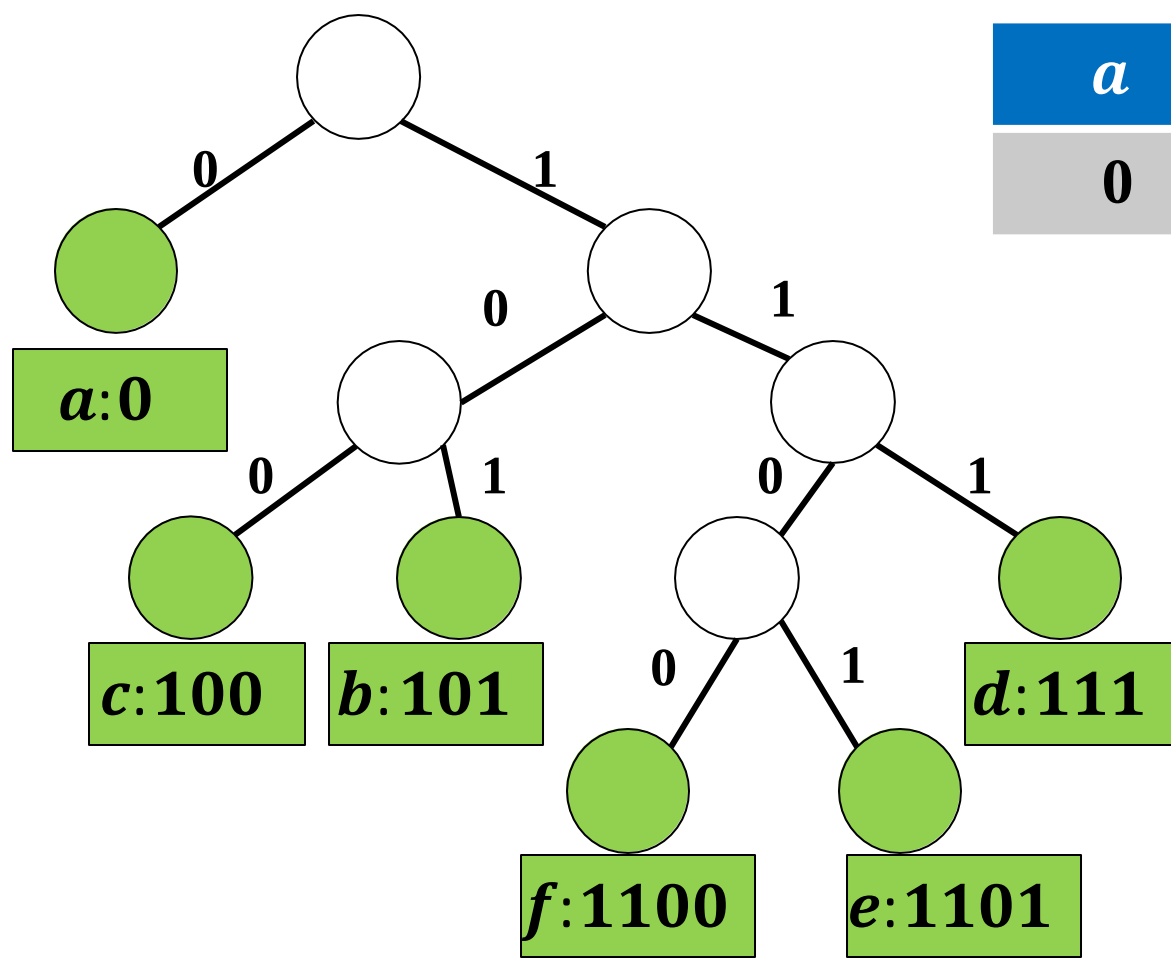
● 解码：10011000 → ?

- 编码树
 - 顶点到左结点的边标记**0**，到右结点的边标记**1**



- 编码树

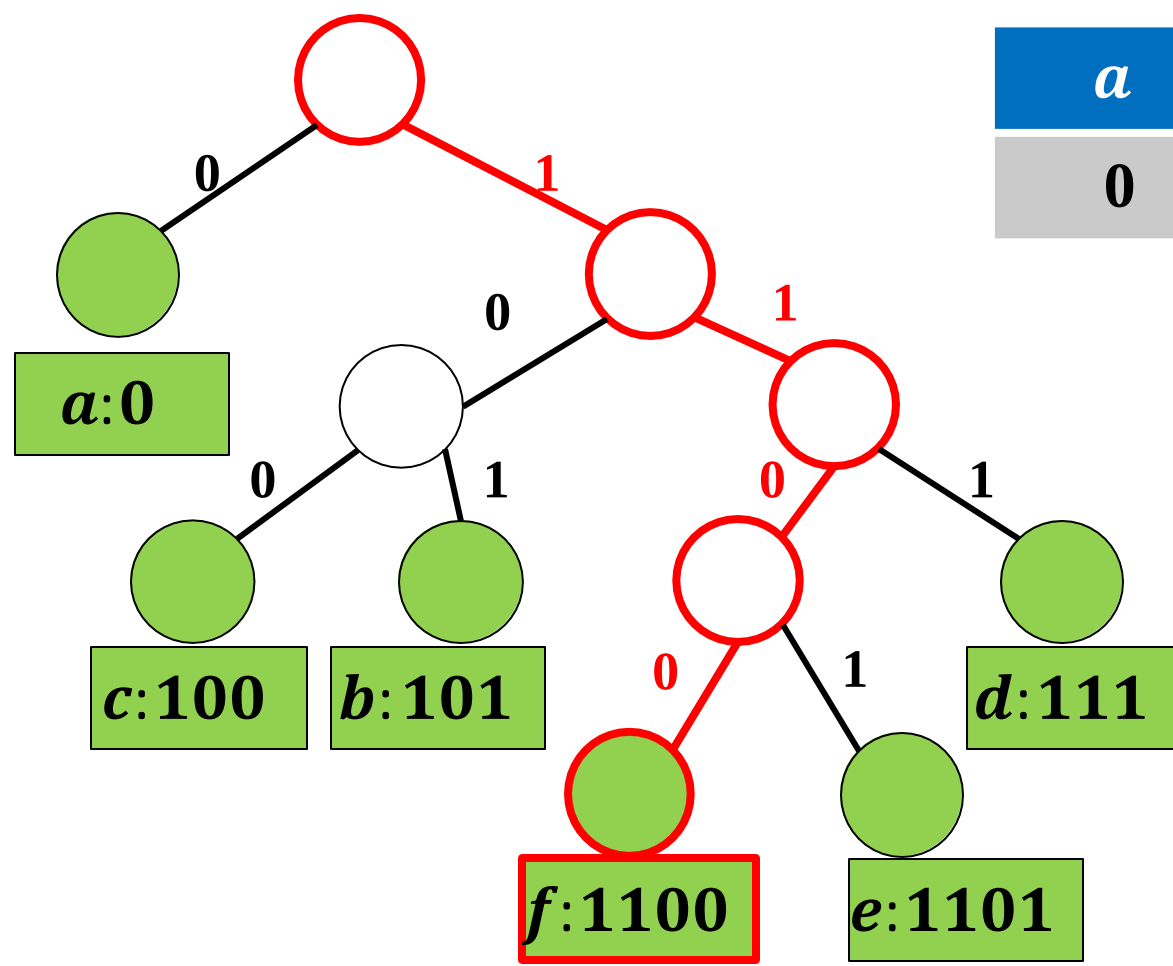
- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树



<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

- 编码树

- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树
- 每条根到叶子的路径对应每个字符的二进制串

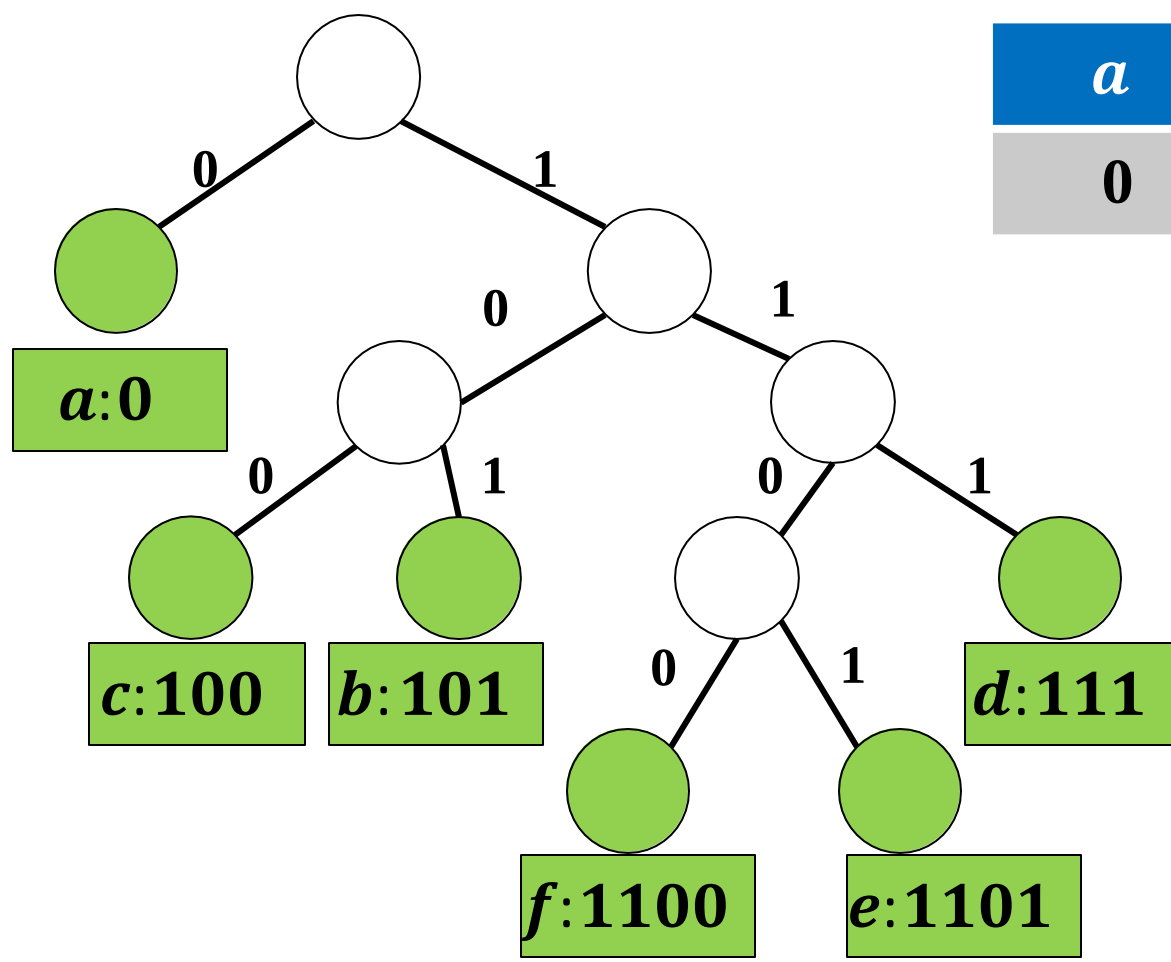


<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

1100表示*f*

- 编码树

- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树
- 每条根到叶子的路径对应每个字符的二进制串

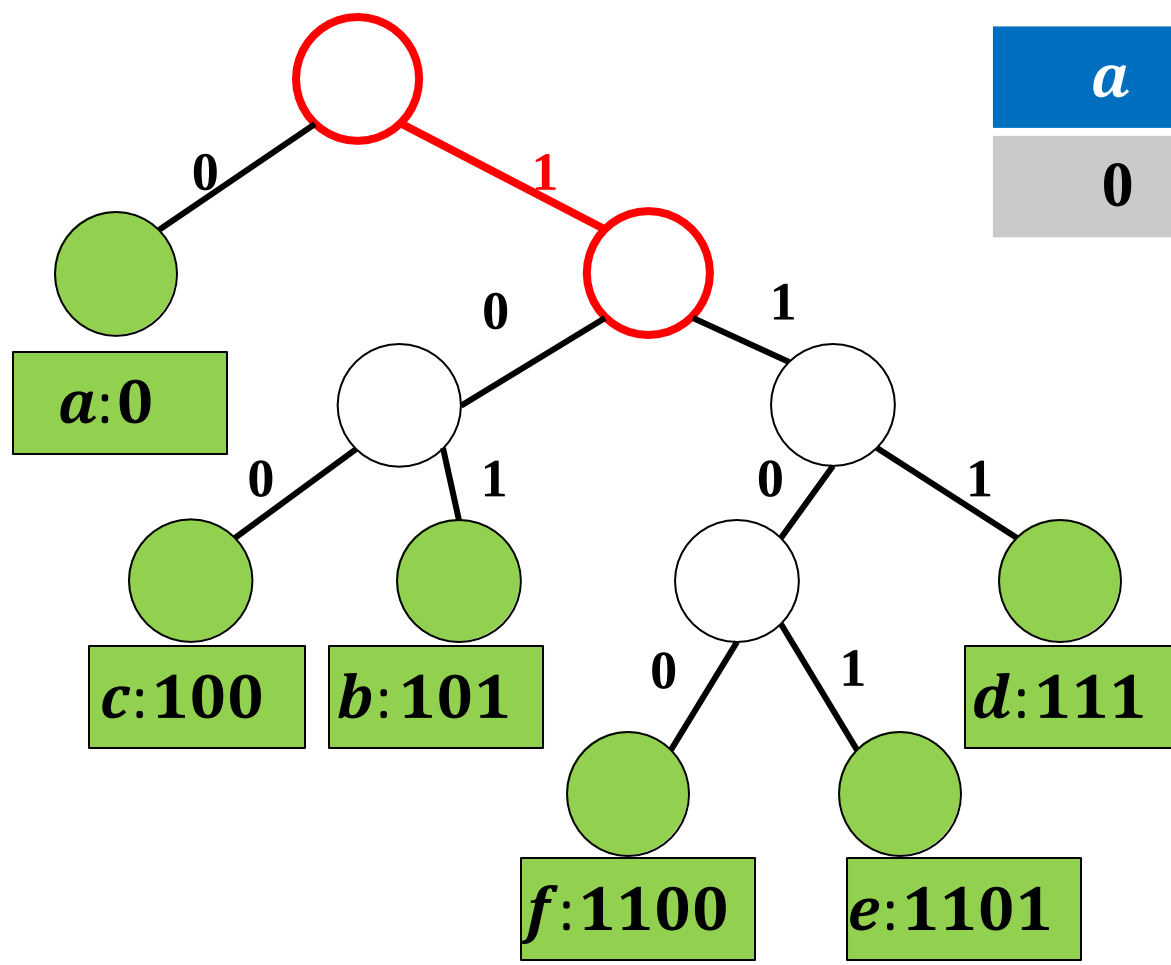


<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

解析1101

- 编码树

- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树
- 每条根到叶子的路径对应每个字符的二进制串

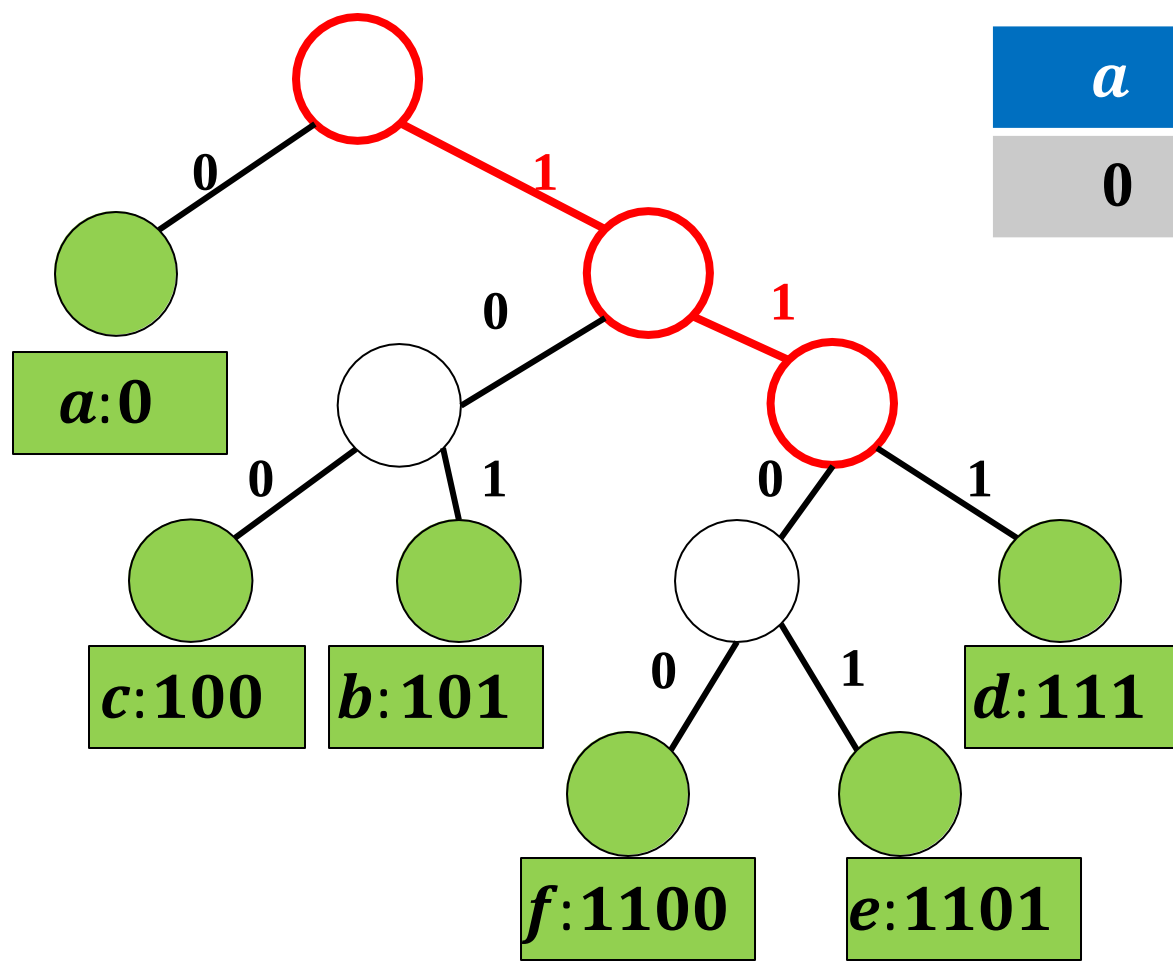


<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

解析**1101**

- 编码树

- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树
- 每条根到叶子的路径对应每个字符的二进制串

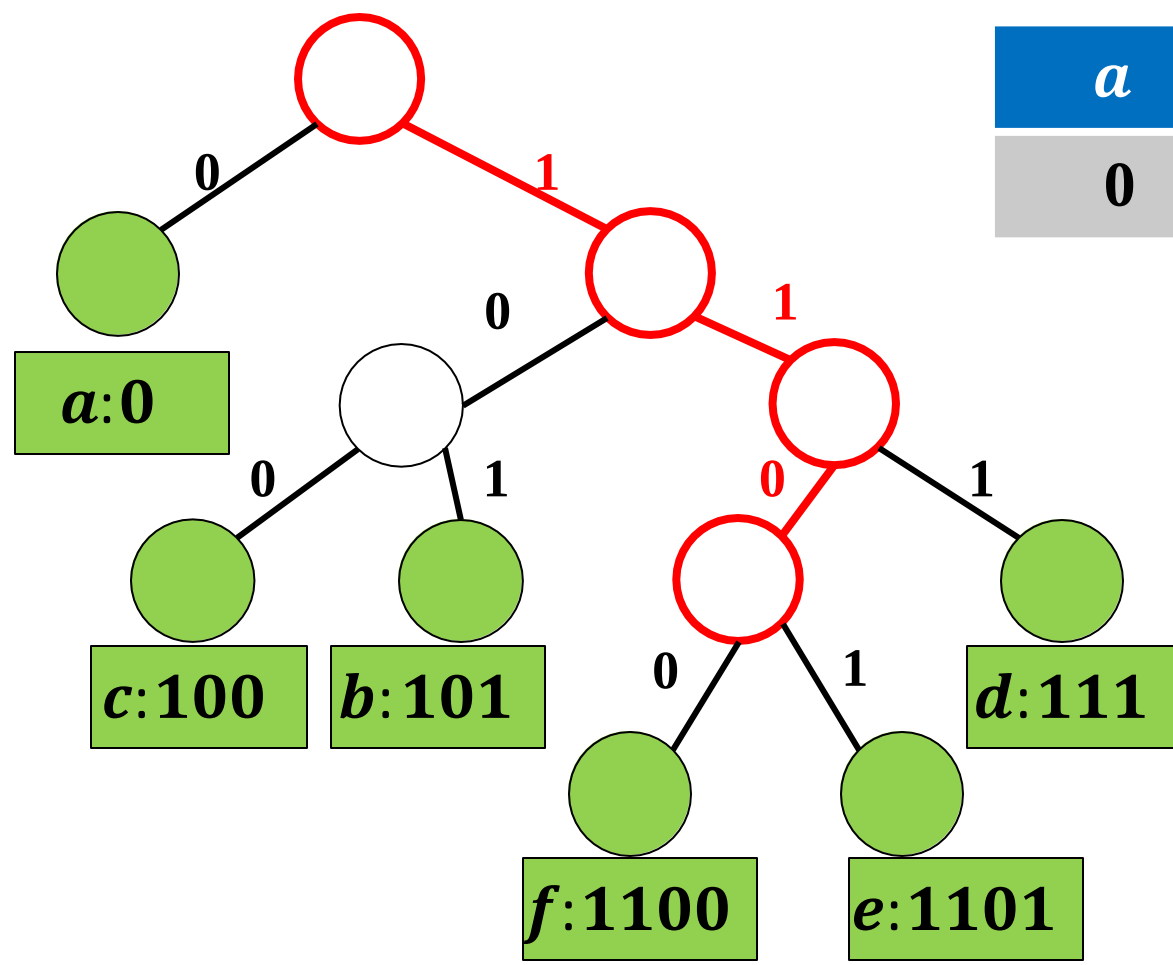


<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

解析**1101**

- 编码树

- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树
- 每条根到叶子的路径对应每个字符的二进制串

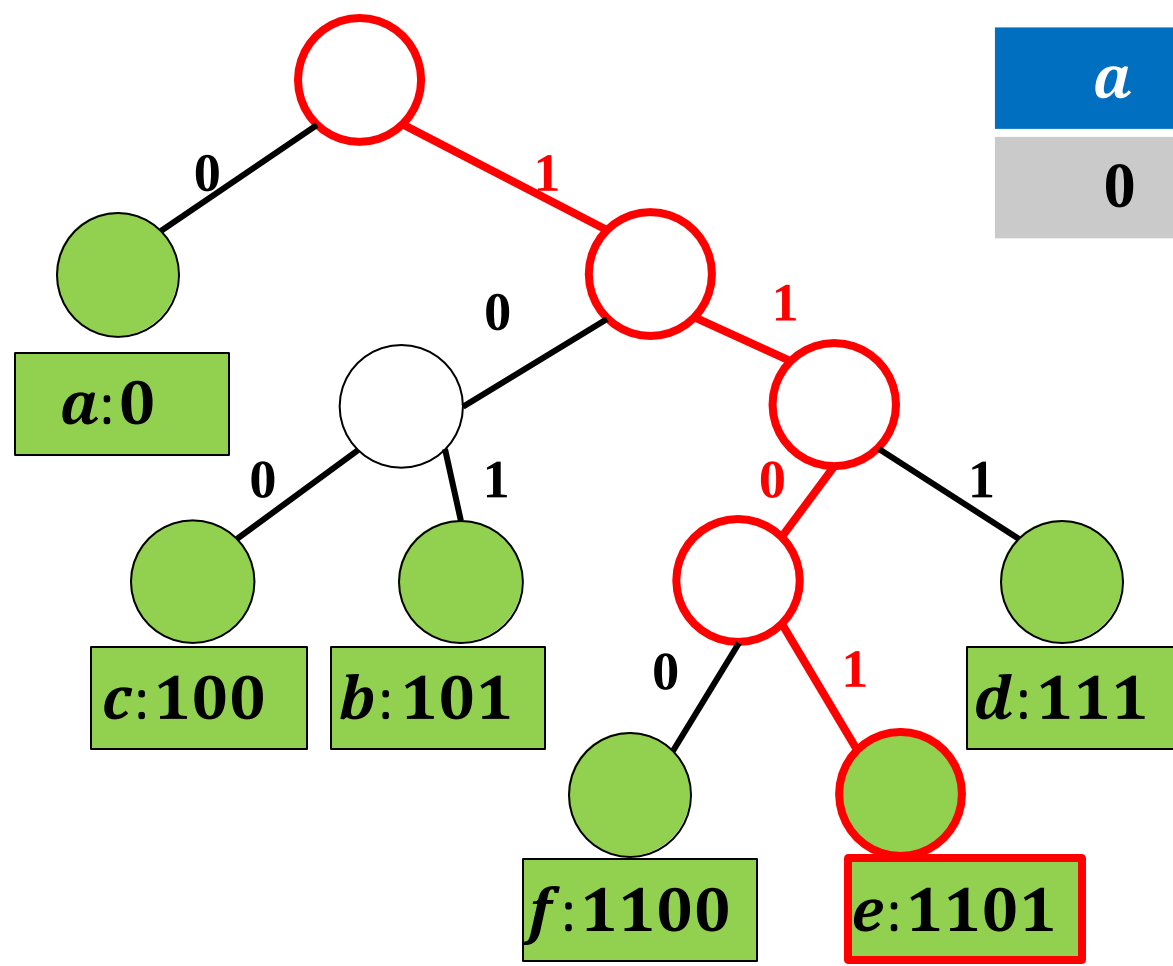


<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

解析**1101**

- 编码树

- 顶点到左结点的边标记**0**，到右结点的边标记**1**，通过编码方案构造编码树
- 每条根到叶子的路径对应每个字符的二进制串



<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
0	101	100	111	1101	1100

1101解析结果为*e*

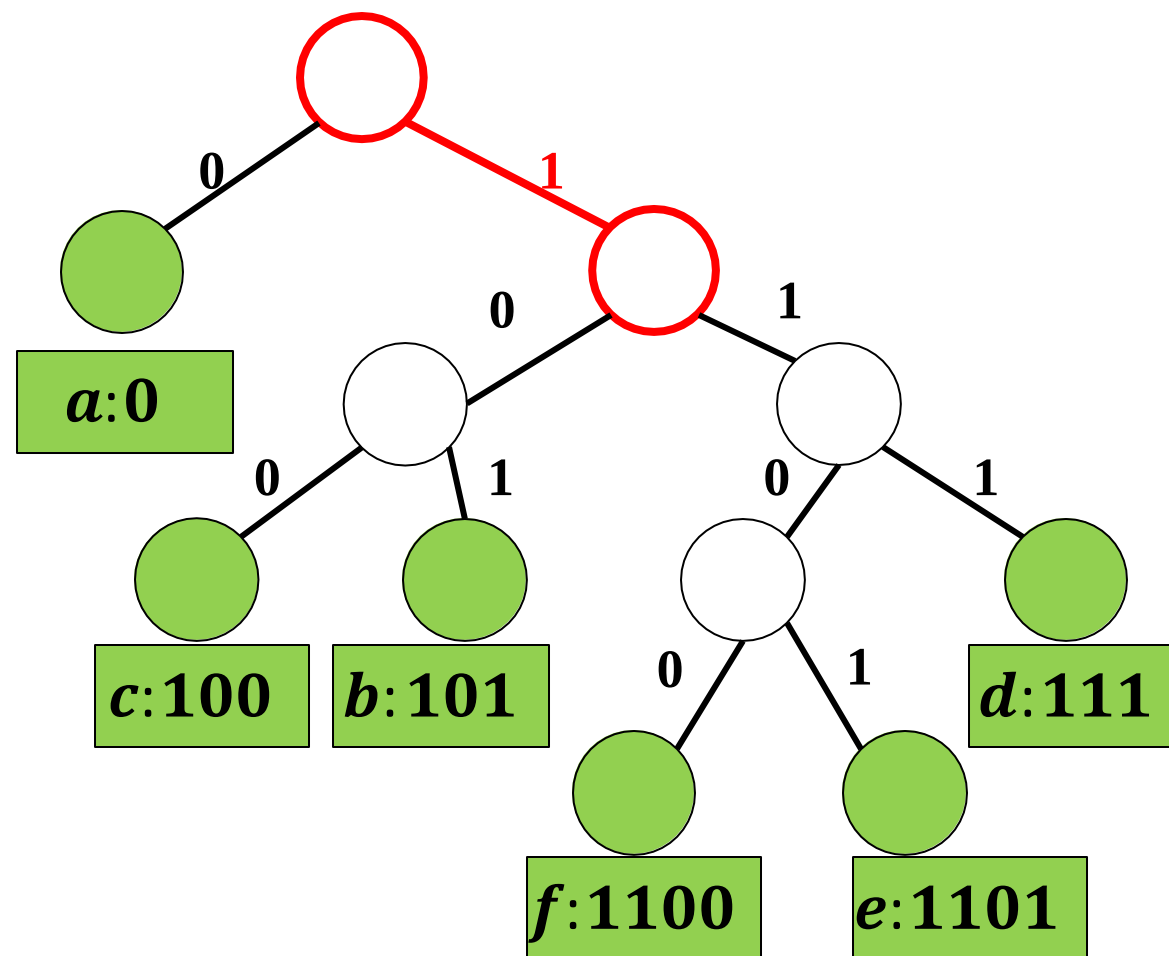
问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 10011000

● 解码：**1**0011000 →



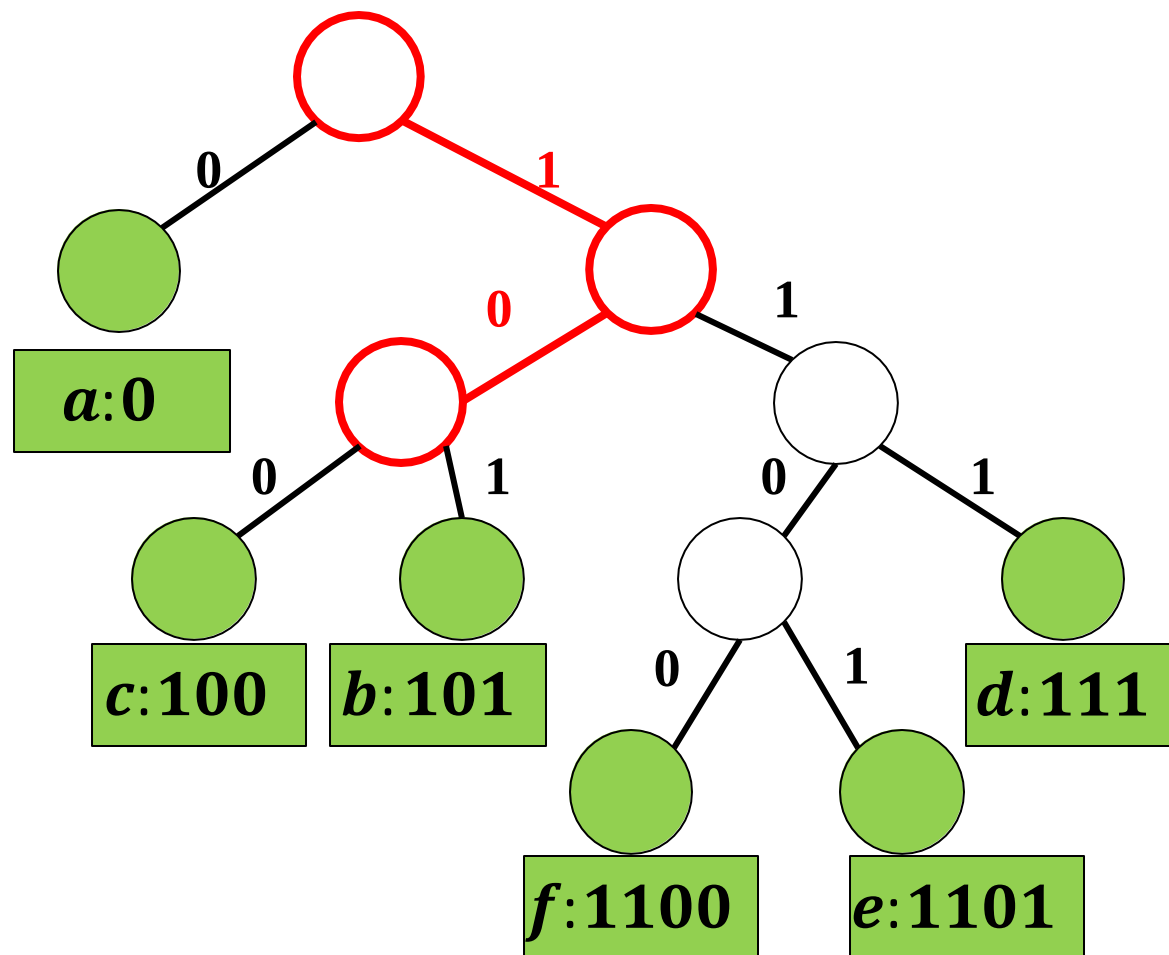
问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 10011000

● 解码：**100**11000 →



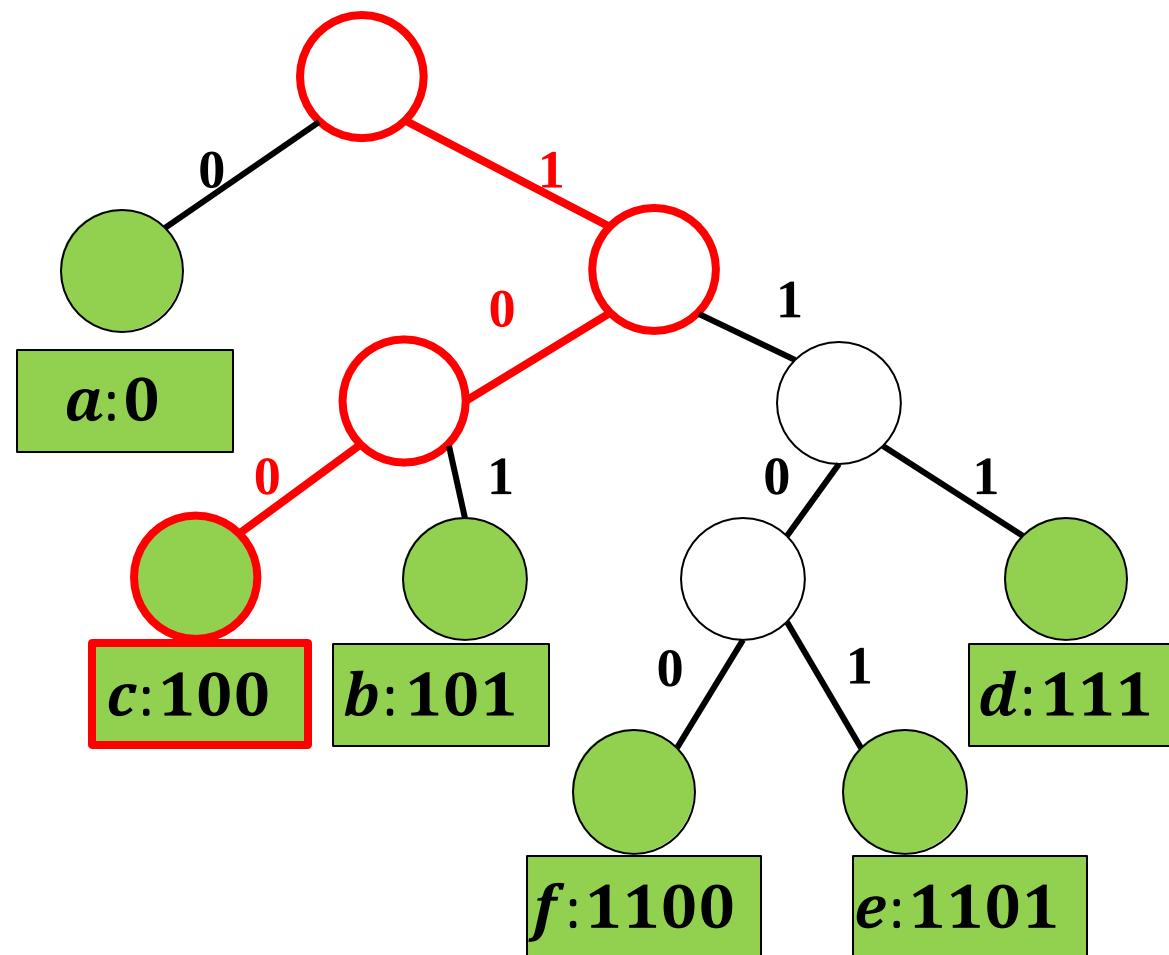
问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 10011000

● 解码：**100**11000 → *c*



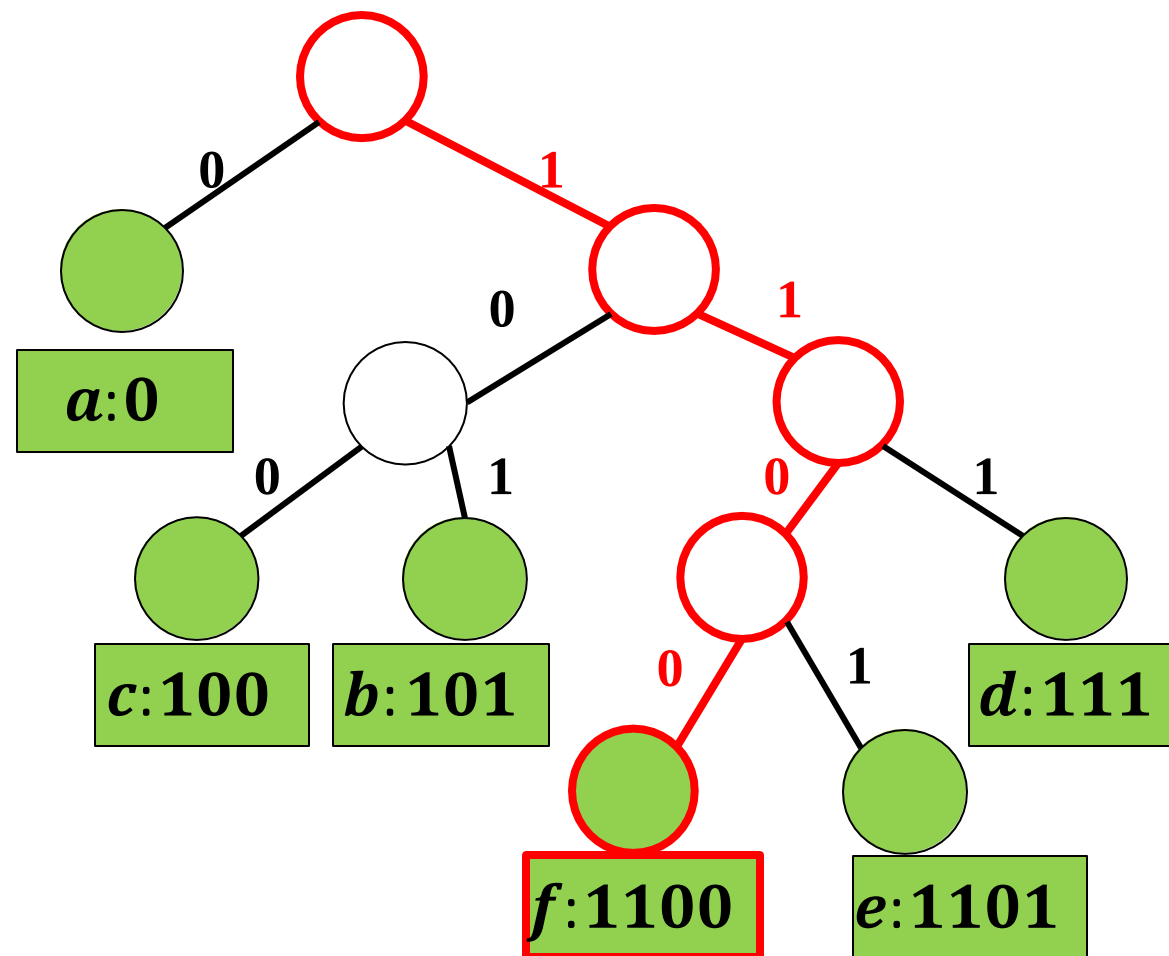
问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 10011000

● 解码：10011000 → *cf*



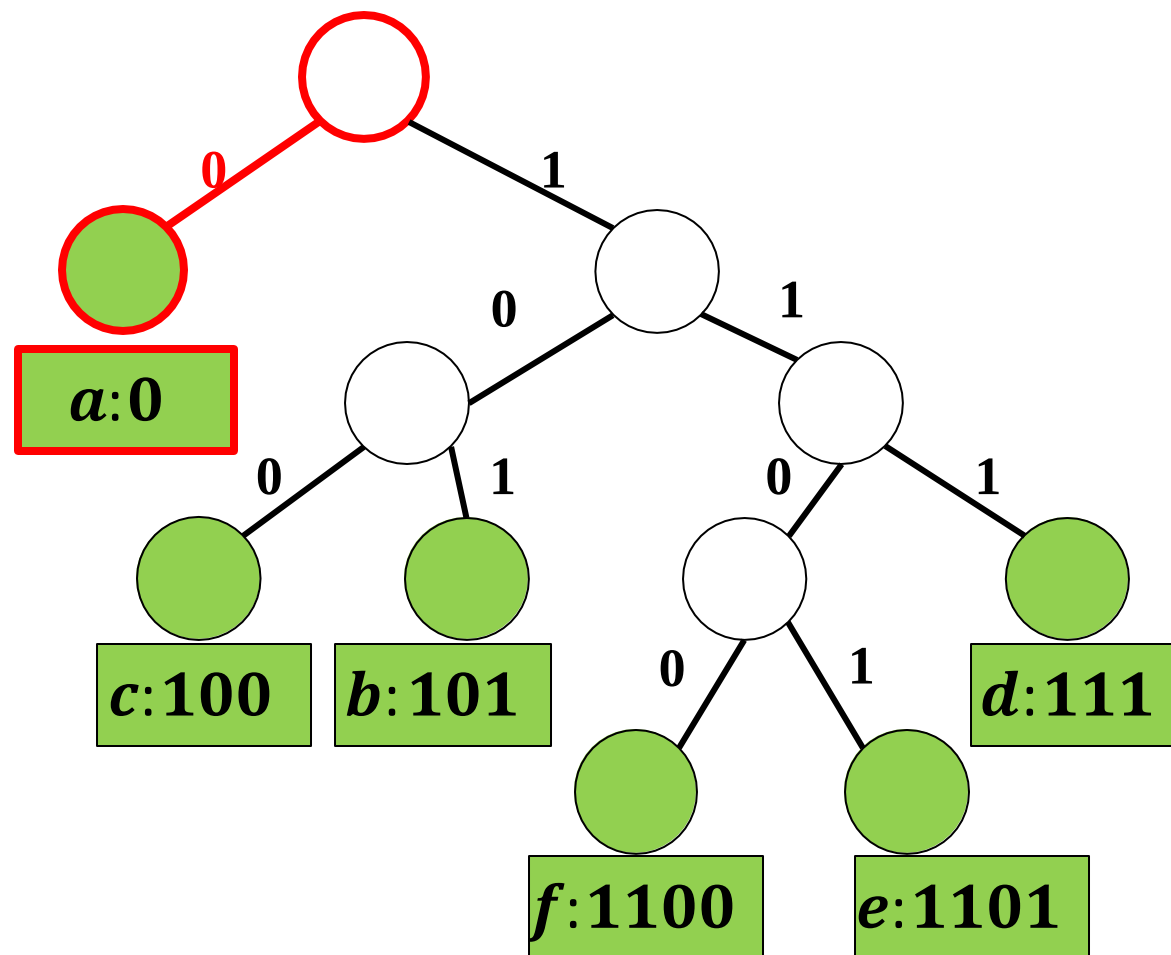
问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 10011000

● 解码：10011000 → *cfa*



问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

● 编码方式1：对"*cfa*"编码

● 编码：*cfa* → 10011000

● 解码：10011000 → *cfa*

● 编码方式2：对"*cfa*"编码

● 编码：*cfa* → 00

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 编码方式1：对"*cfa*"编码

- 编码：*cfa* → 10011000

- 解码：10011000 → *cfa*

- 编码方式2：对"*cfa*"编码

- 编码：*cfa* → 0011

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 编码方式1：对"*cfa*"编码

- 编码：*cfa* → 10011000

- 解码：10011000 → *cfa*

- 编码方式2：对"*cfa*"编码

- 编码：*cf****a*** → 0011**0**

问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 编码方式1：对"*cfa*"编码

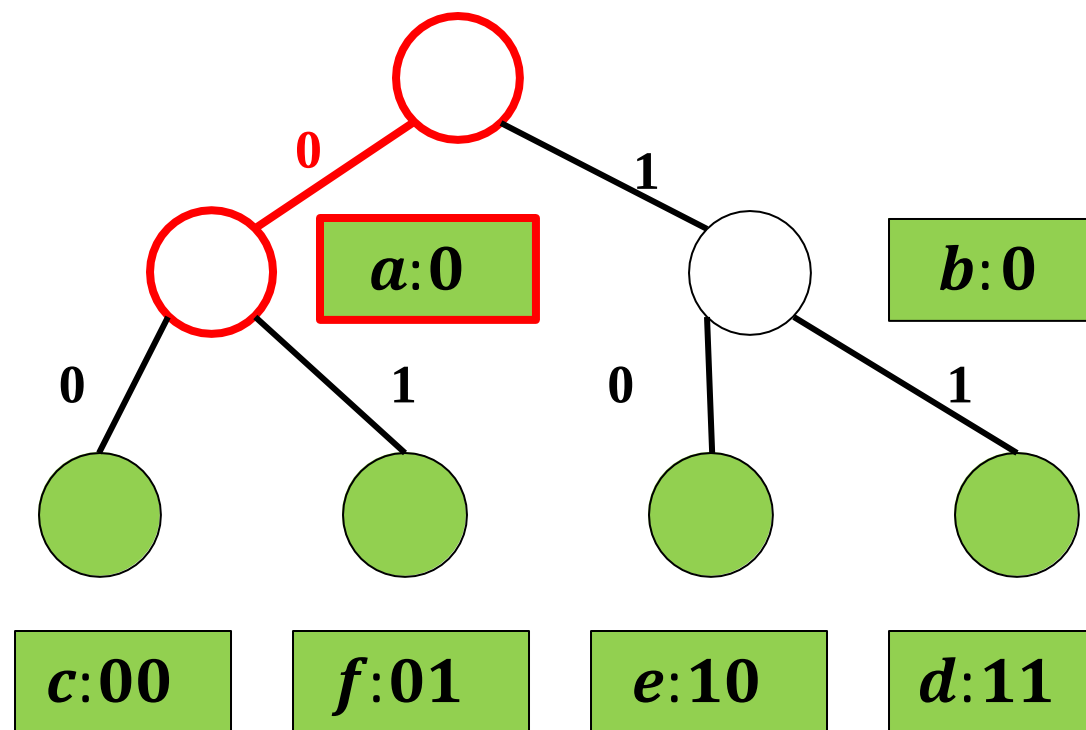
- 编码：*cfa* → 10011000

- 解码：10011000 → *cfa*

- 编码方式2：对"*cfa*"编码

- 编码：*cfa* → 00110

- 解码：00110 → *a*



问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 编码方式1：对"*cfa*"编码

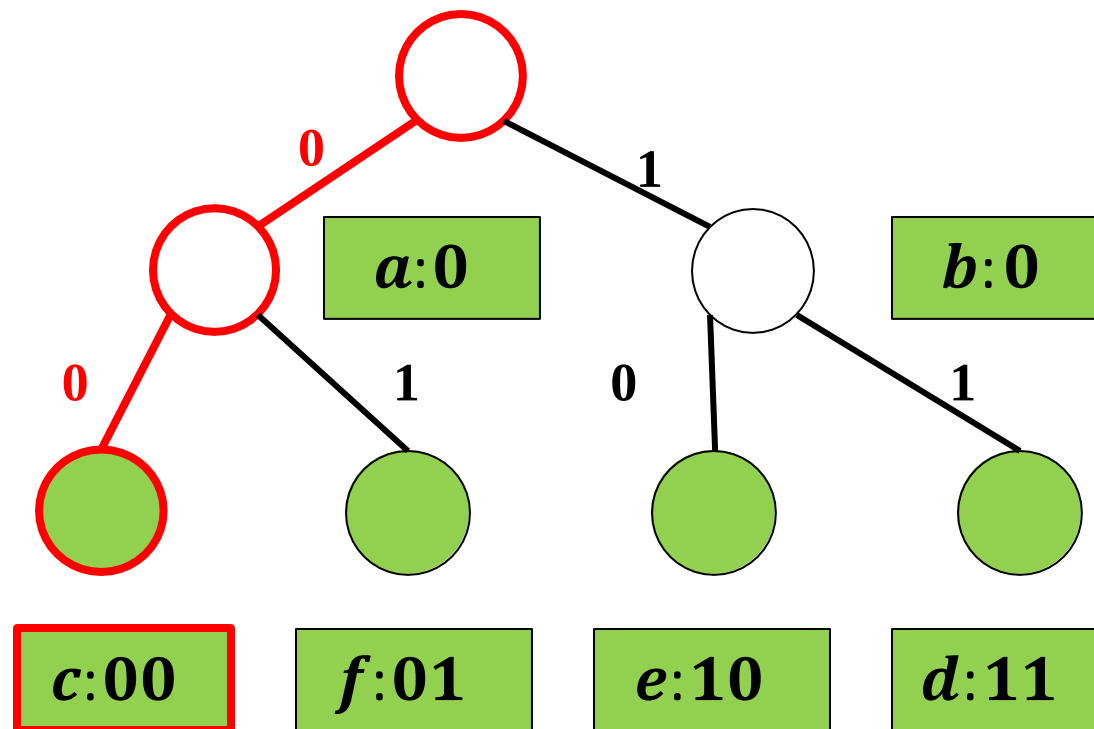
- 编码：*cfa* → 10011000

- 解码：10011000 → *cfa*

- 编码方式2：对"*cfa*"编码

- 编码：*cfa* → 00110

- 解码：00110 → *c*



问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 编码方式1：对"*cfa*"编码

- 编码：*cfa* → 10011000

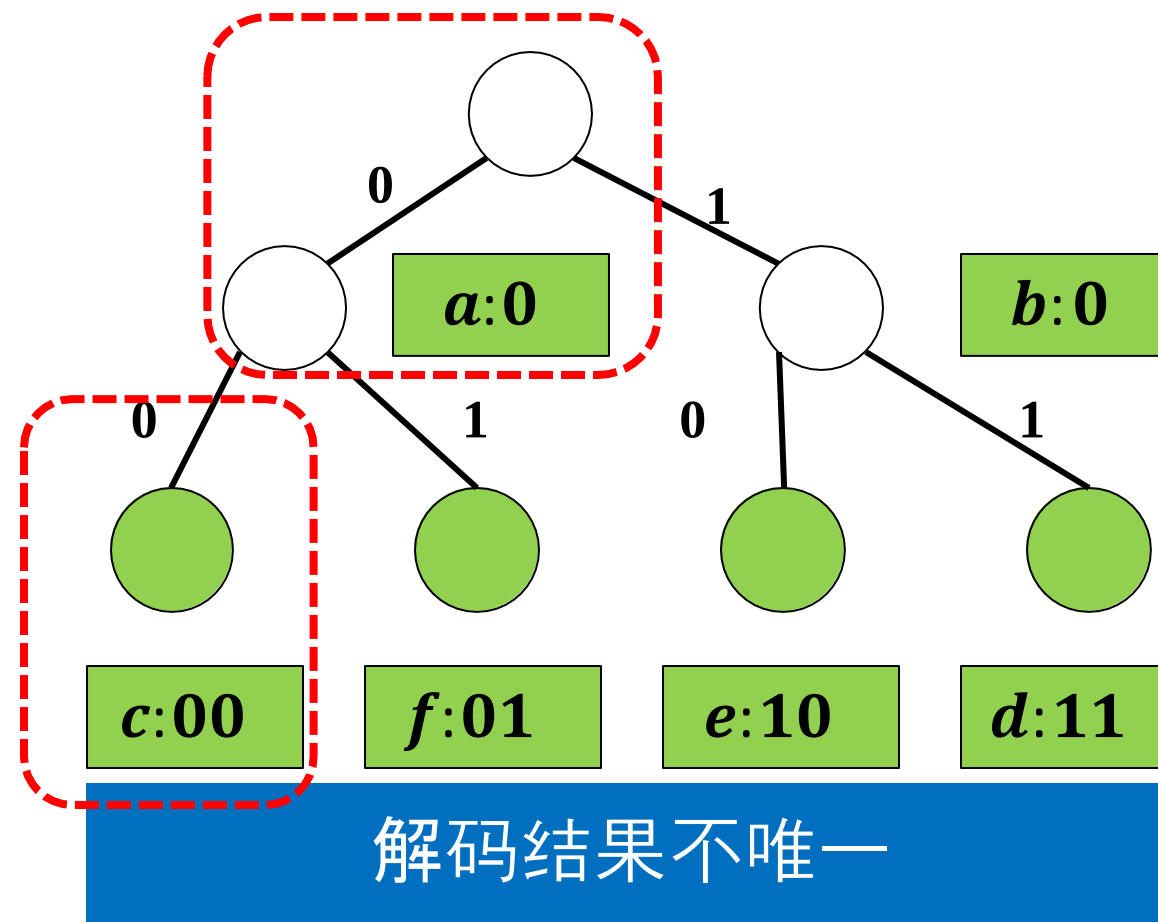
- 解码：10011000 → *cfa*

- 编码方式2：对"*cfa*"编码

- 编码：*cfa* → 00110

- 解码：00110 → *aabba*

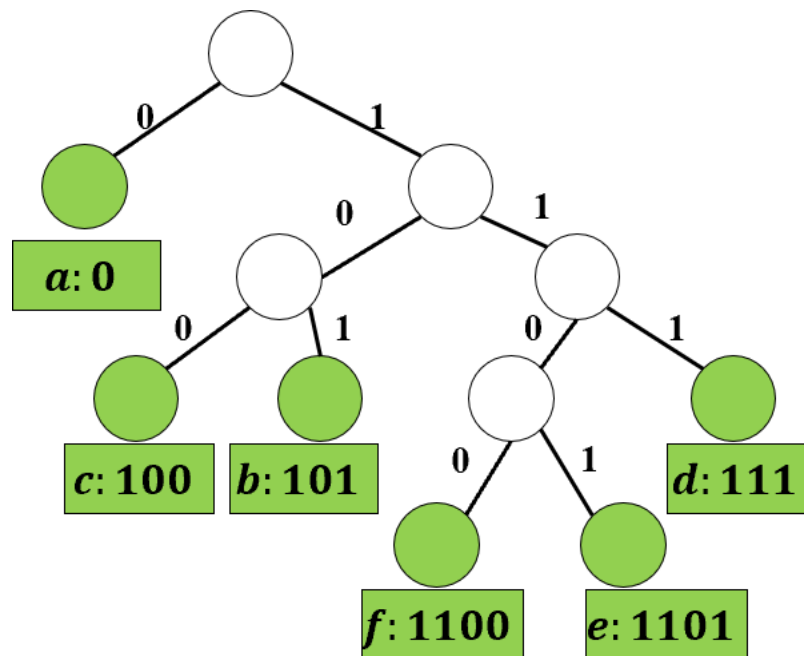
- 解码：00110 → *cfa*



问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 前缀码：编码的任意前缀不是其他编码

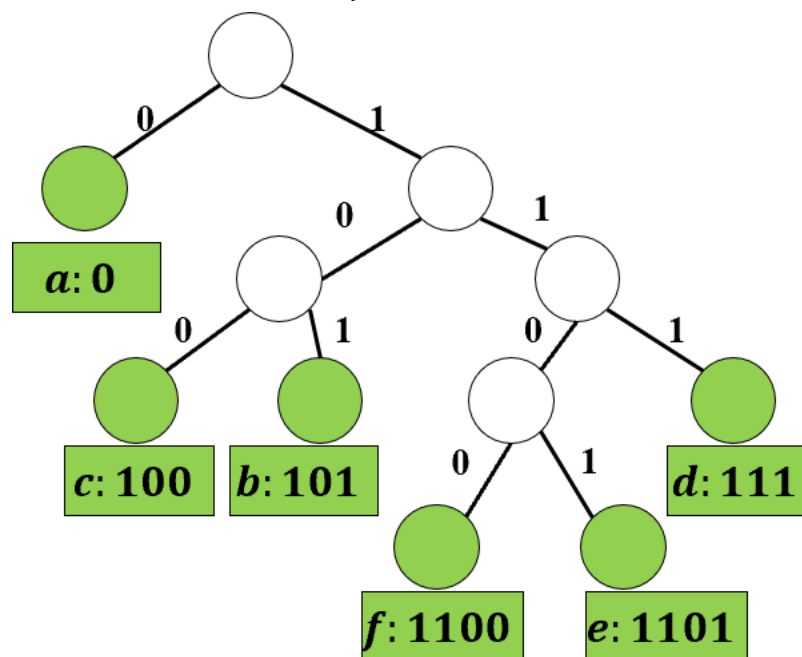


解码结果唯一，编码方式可行

问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
编码方式1	0	101	100	111	1101	1100
编码方式2	0	1	00	01	10	11

- 前缀码：编码的任意前缀不是其他编码



问题：如何比较前缀码之间的优劣？

问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
频数（千次）	45	13	12	16	9	5

- 比较编码后二进制串的总长，其依赖于待编码字符的频数

问题背景：字符编码

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
频数（千次）	45	13	12	16	9	5
前缀码1	0	101	100	111	1101	1100

- 比较编码后二进制串的总长，其依赖于待编码字符的频数
- 使用前缀码1编码
 - $(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1000 = 224\ 000$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
频数（千次）	45	13	12	16	9	5
前缀码1	0	101	100	111	1101	1100
前缀码2	1100	1101	111	100	101	0

- 比较编码后二进制串的总长，其依赖于待编码字符的频数
- 使用前缀码1编码
 - $(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1000 = 224\ 000$
- 使用前缀码2编码
 - $(45 \times 4 + 13 \times 4 + 12 \times 3 + 16 \times 3 + 9 \times 3 + 5 \times 1) \times 1000 = 348\ 000$

问题：如何求得编码后二进制串总长最短的前缀码？

最优前缀码问题

- 输入：字符数 n 以及各个字符的频数 $F = \langle f_1, f_2, \dots, f_n \rangle$
- 输出：解析结果唯一的二进制编码方案 $C = \langle c_1, \dots, c_n \rangle$, 令

$$\min \sum_{i=1}^n |c_i| \cdot f_i$$

优化目标

$|c_i|$ 为字符 i 的编码二进制串长度

提出贪心策略

观察问题特征，构造贪心选择



证明策略正确

假设最优方案，通过替换证明

贪心策略：观察问题特征

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
频数（千次）	45	13	12	16	9	5
前缀码1	0	101	100	111	1101	1100
前缀码2	1100	1101	111	100	101	0

- 比较编码后二进制串的总长，其依赖于待编码字符的频数

- 使用前缀码1编码

$$(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1000 = 224\ 000$$

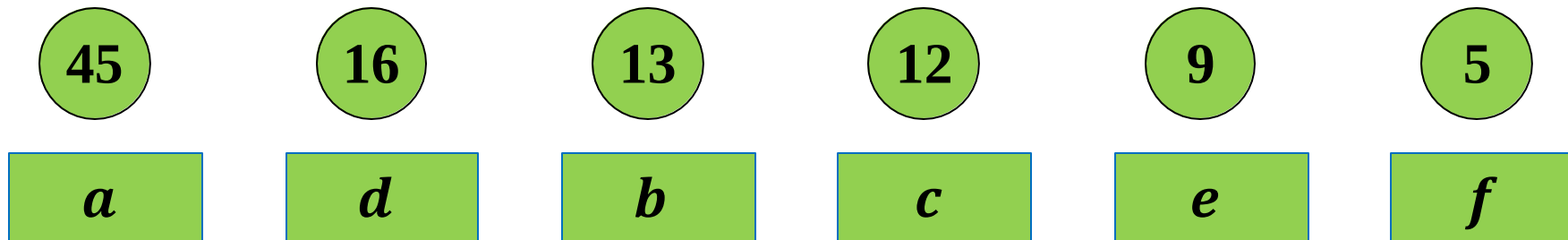
- 使用前缀码2编码

$$(45 \times 4 + 13 \times 4 + 12 \times 3 + 16 \times 3 + 9 \times 3 + 5 \times 1) \times 1000 = 348\ 000$$

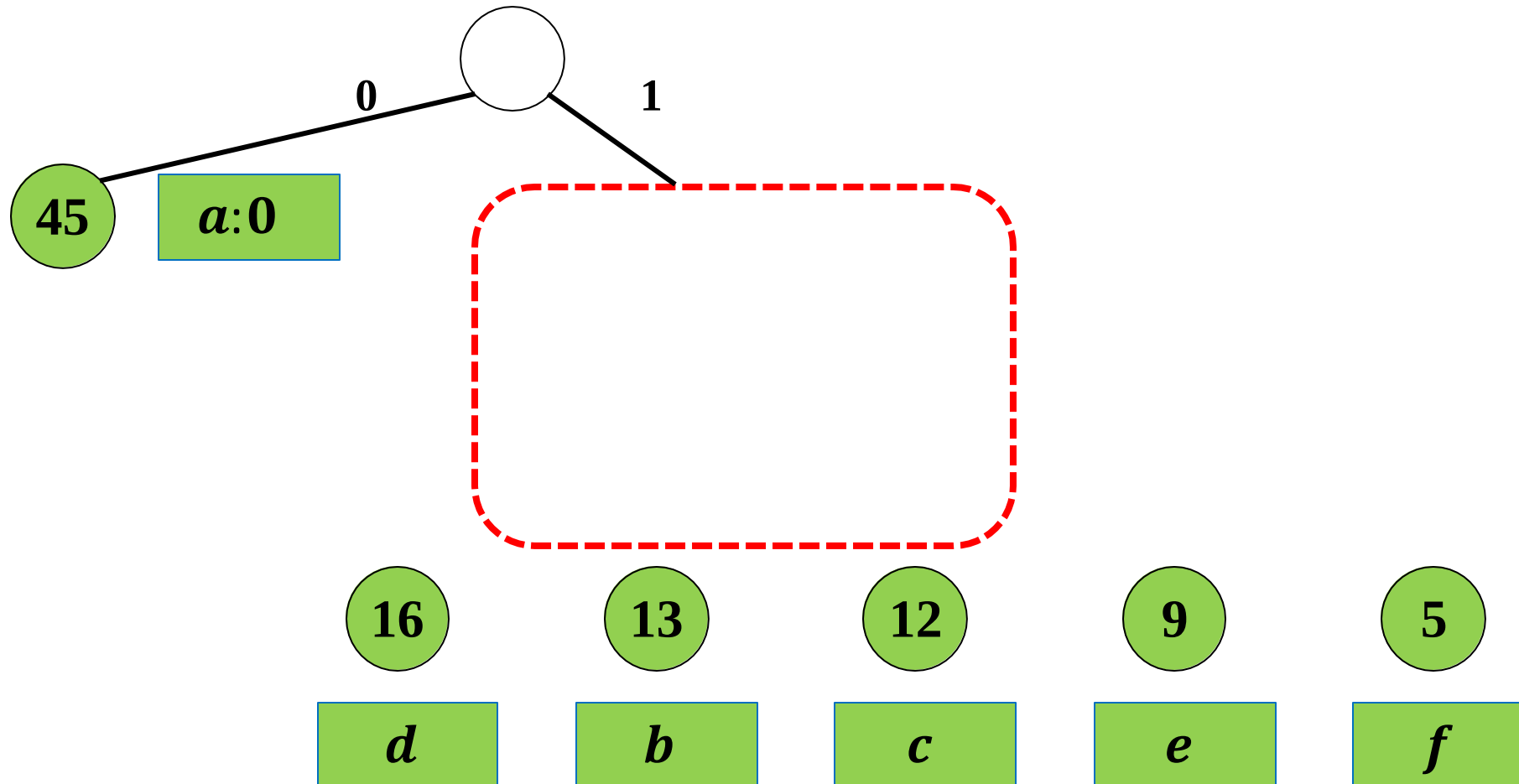
最优解

编码方案适应频数大小，短二进制串编码高频字符

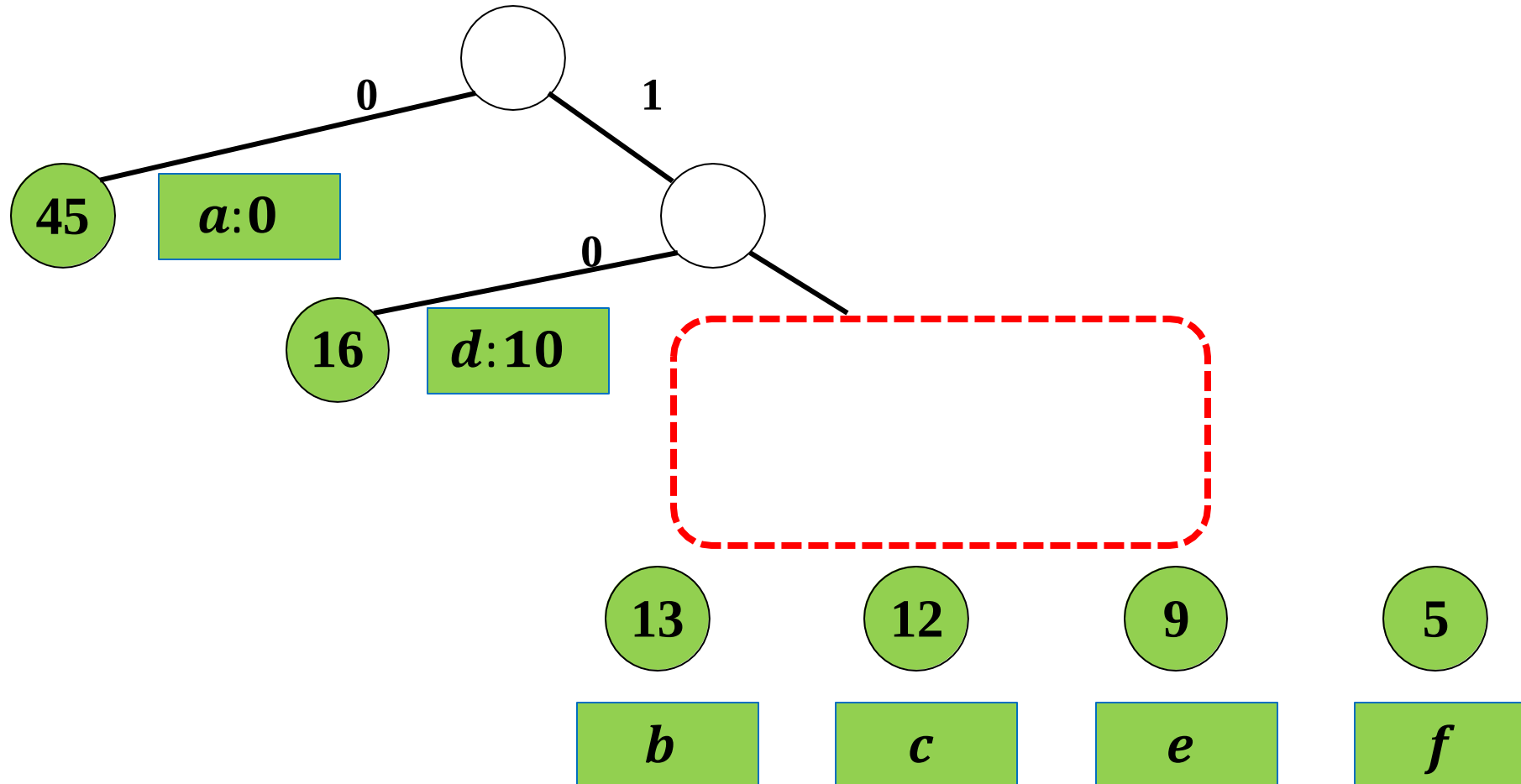
- 优先处理高频字符
 - 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \geq f_2 \geq \dots \geq f_n$)



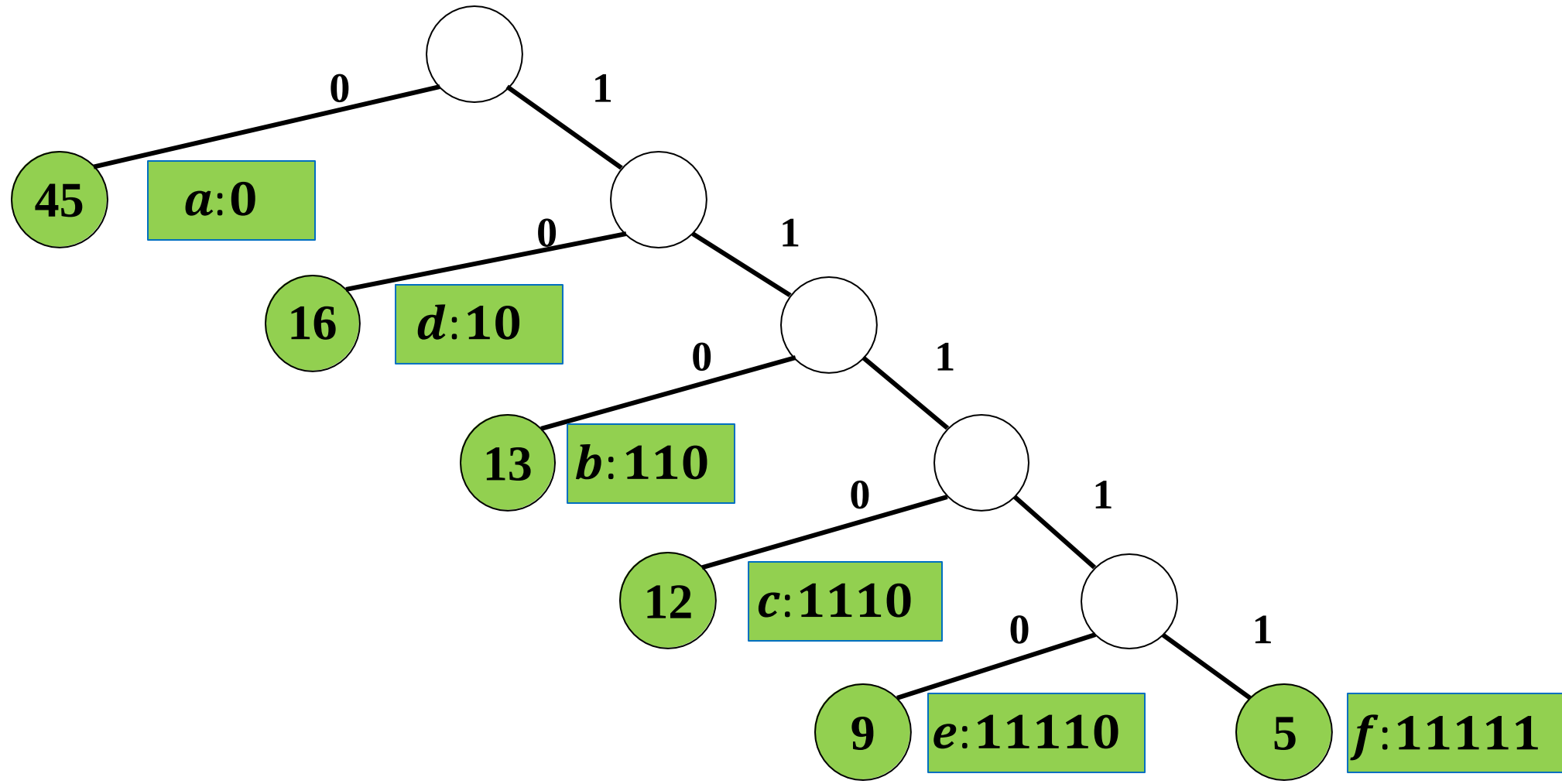
- 优先处理高频字符
 - 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \geq f_2 \geq \dots \geq f_n$)
 - 依次编码高频字符



- 优先处理高频字符
 - 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \geq f_2 \geq \dots \geq f_n$)
 - 依次编码高频字符

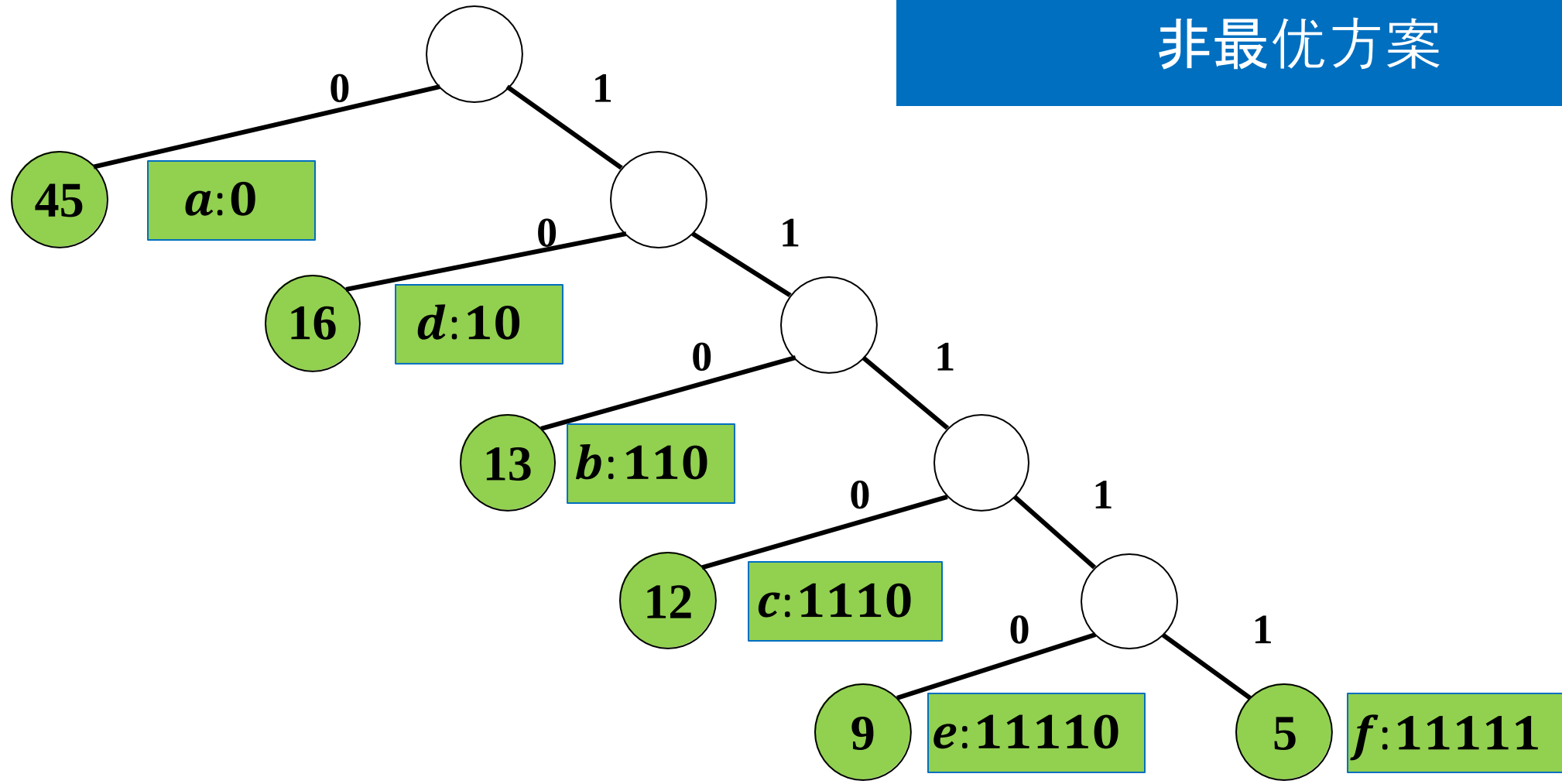


- 优先处理高频字符
 - 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \geq f_2 \geq \dots \geq f_n$)
 - 依次编码高频字符



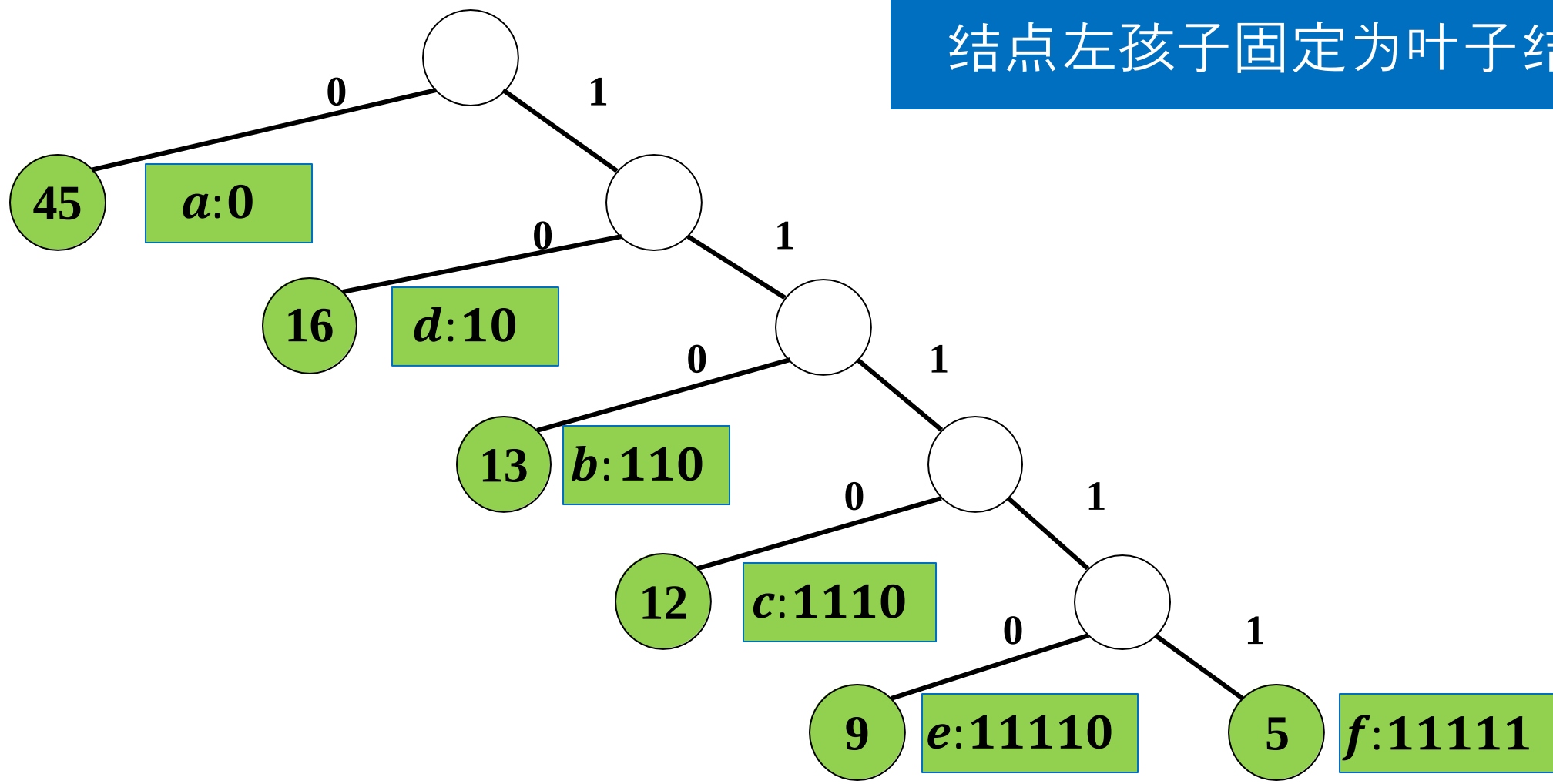
- 优先处理高频字符

- $45 \times 1 + 16 \times 2 + 13 \times 3 + 12 \times 4 + 9 \times 5 + 5 \times 5 = 234 > 224$

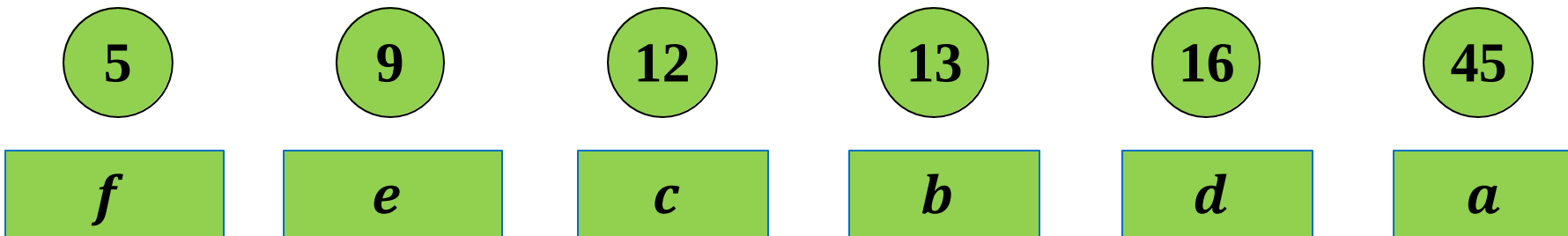


- 优先处理高频字符

- $45 \times 1 + 16 \times 2 + 13 \times 3 + 12 \times 4 + 9 \times 5 + 5 \times 5 = 234 > 224$



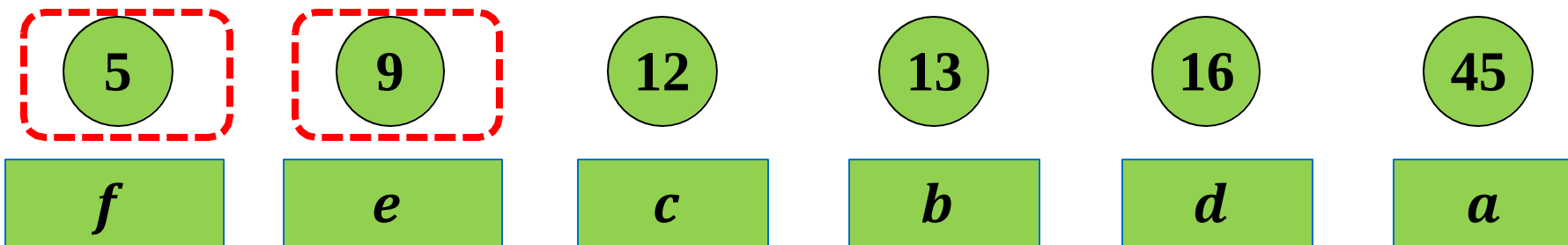
- 优先处理低频字符
 - 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$

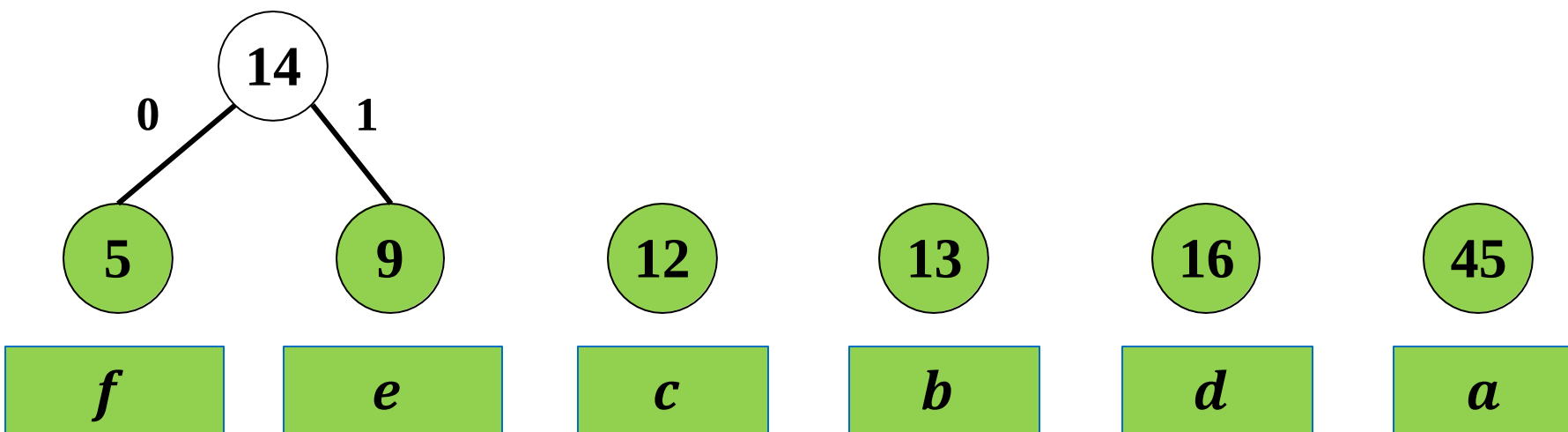
叶子频数	5	9	12	13	16	45
合并频数						



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$

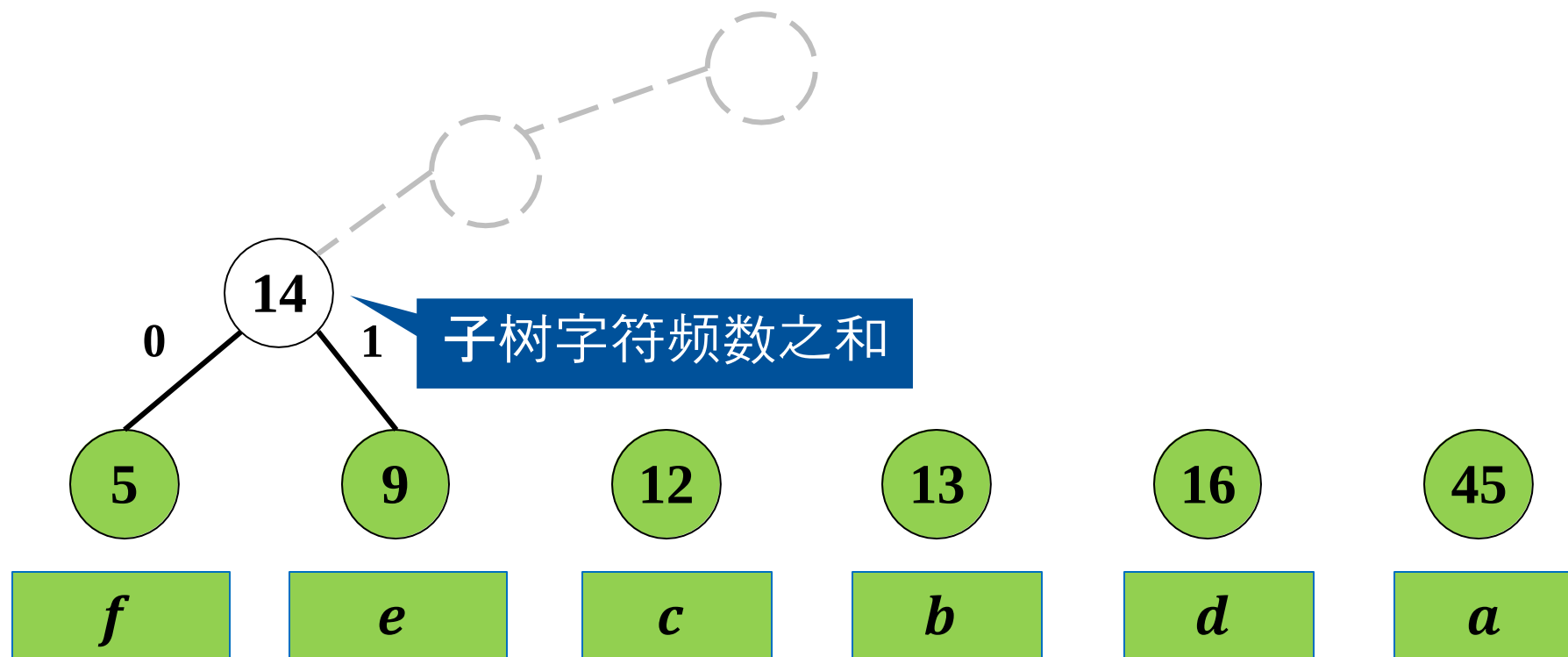
叶子频数			12	13	16	45
合并频数	14					



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$

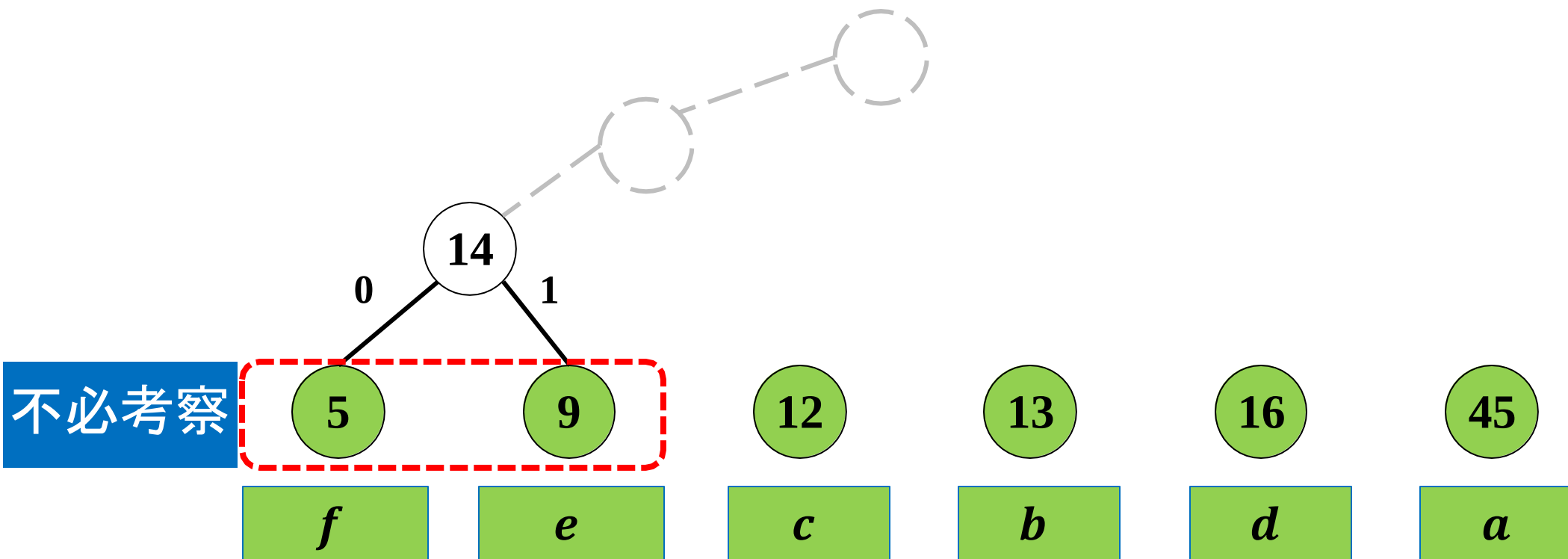
叶子频数			12	13	16	45
合并频数	14					



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$

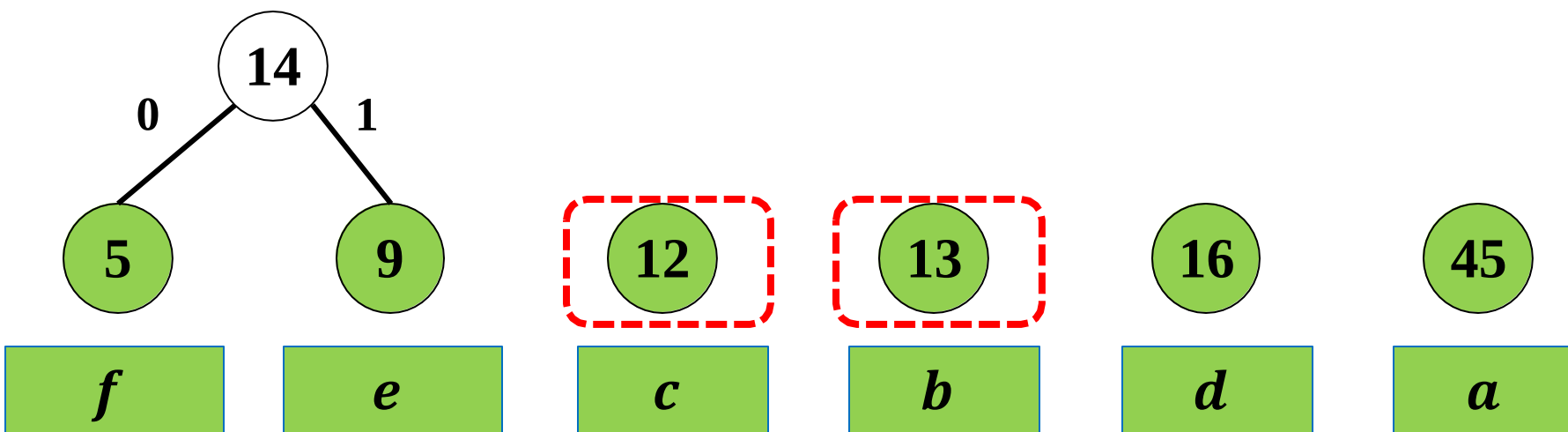
叶子频数			12	13	16	45
合并频数	14					



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

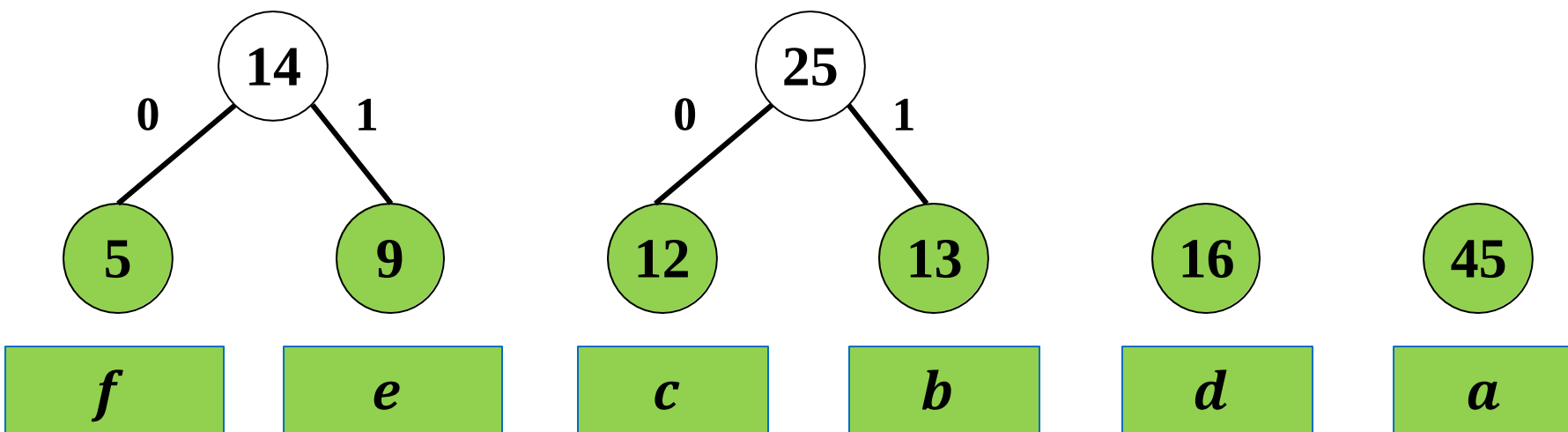
叶子频数			12	13	16	45
合并频数	14					



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

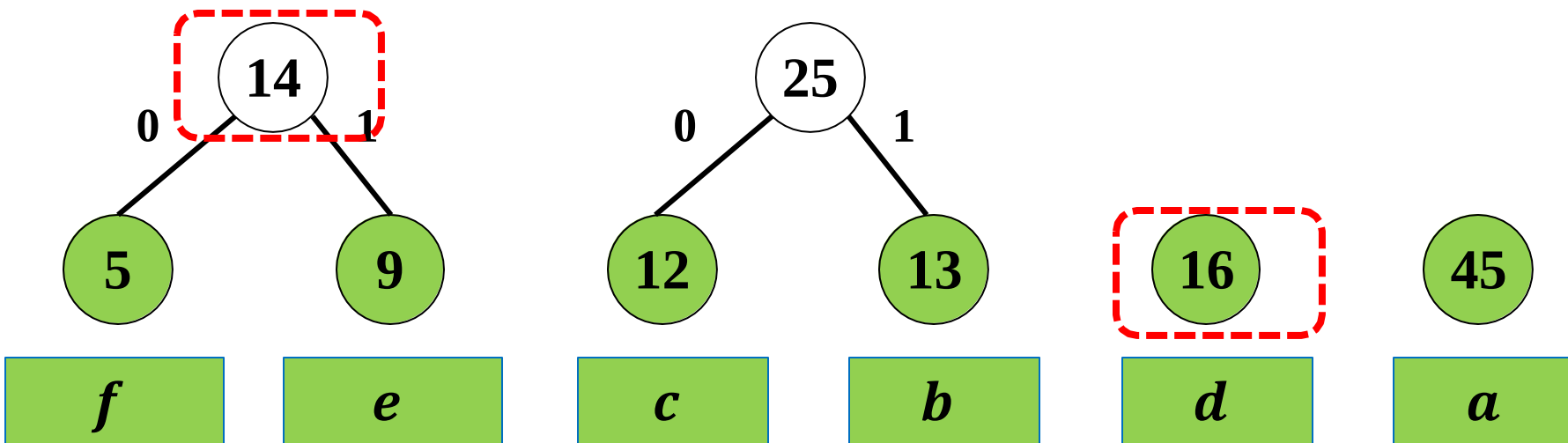
叶子频数					16	45
合并频数	14	25				



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

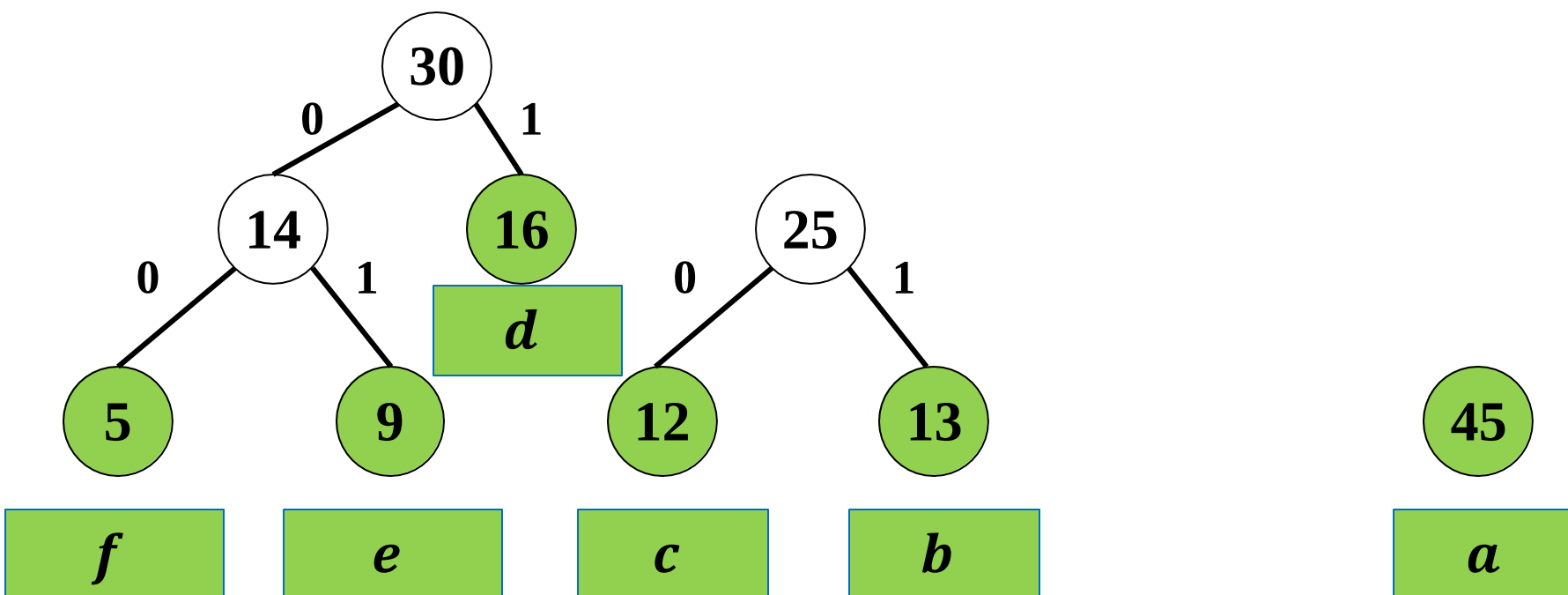
叶子频数					16	45
合并频数	14	25				



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

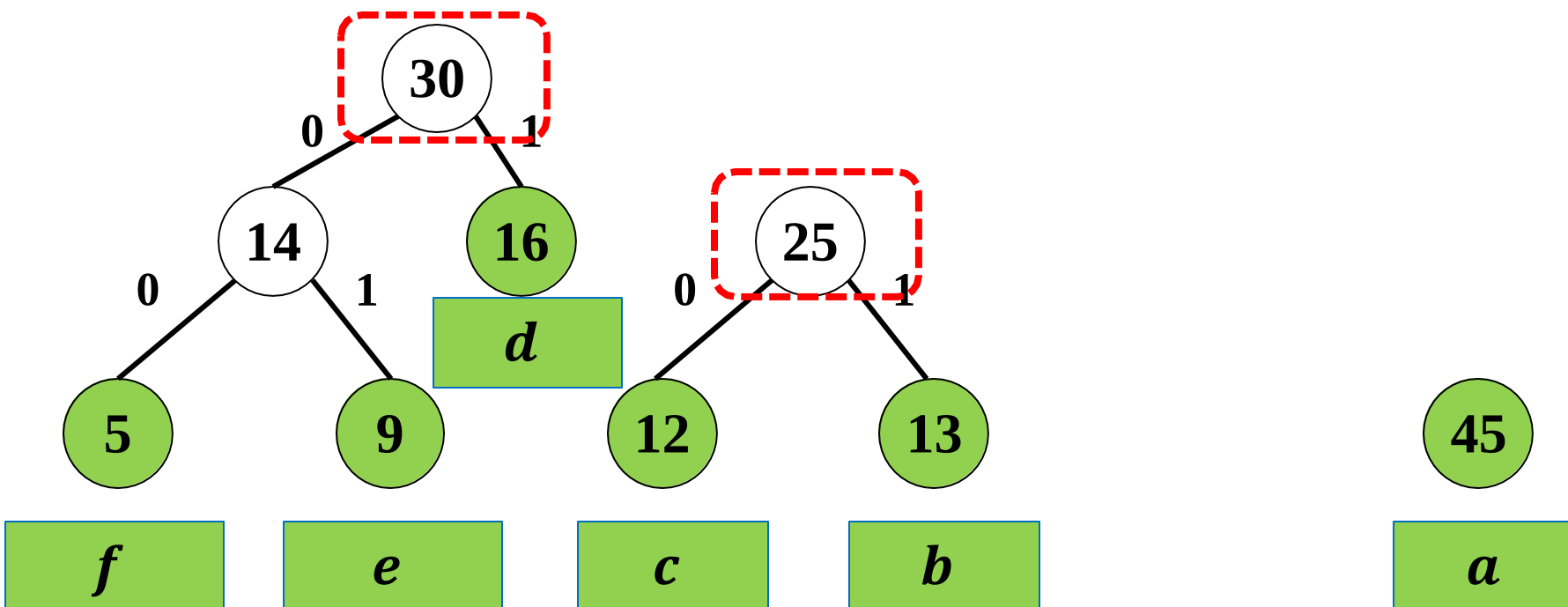
叶子频数						45
合并频数		25	30			



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

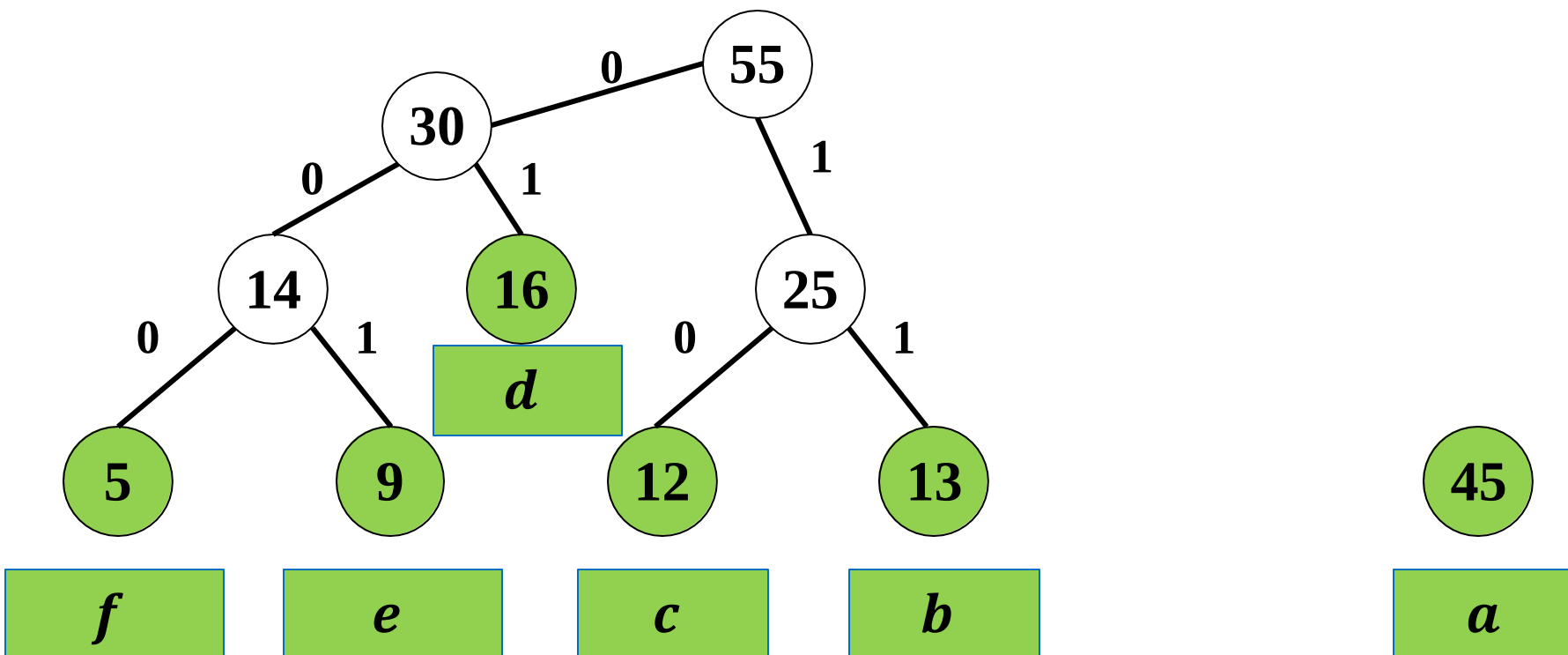
叶子频数						45
合并频数		25	30			



- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

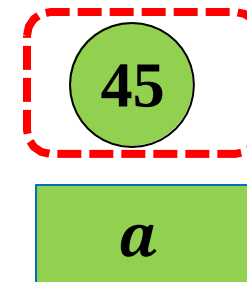
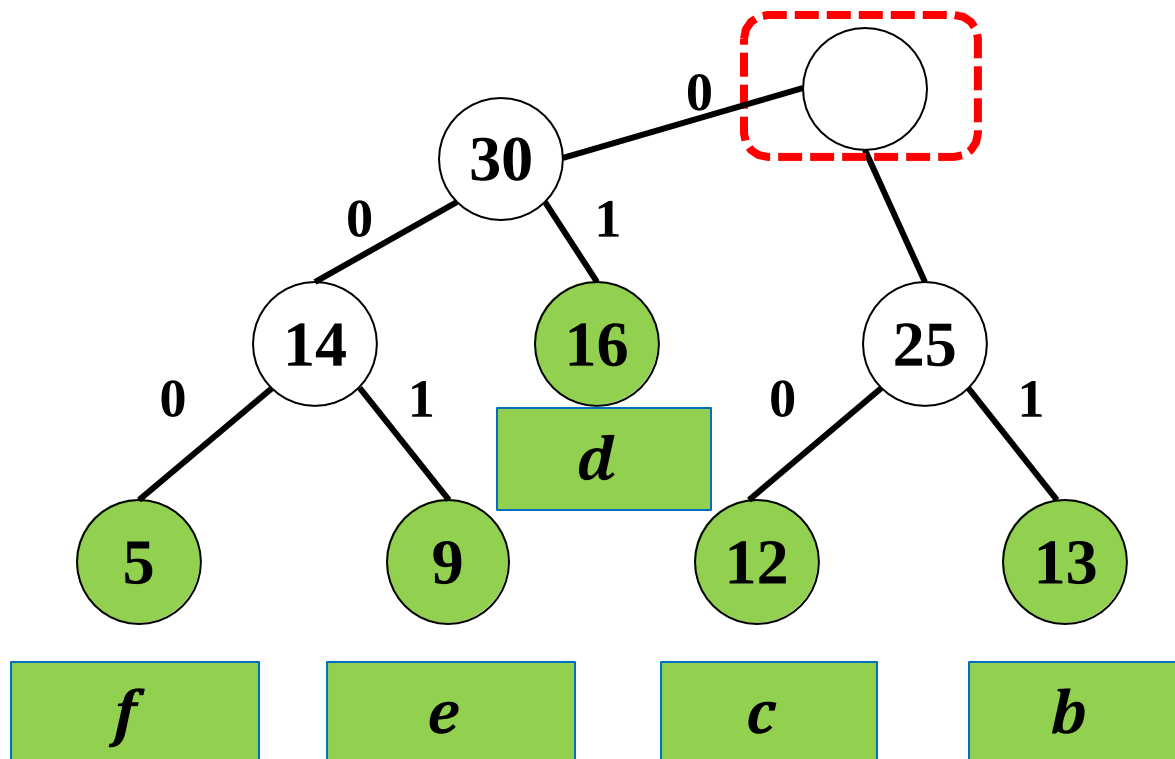
叶子频数						45
合并频数				55		



- 优先处理低频字符

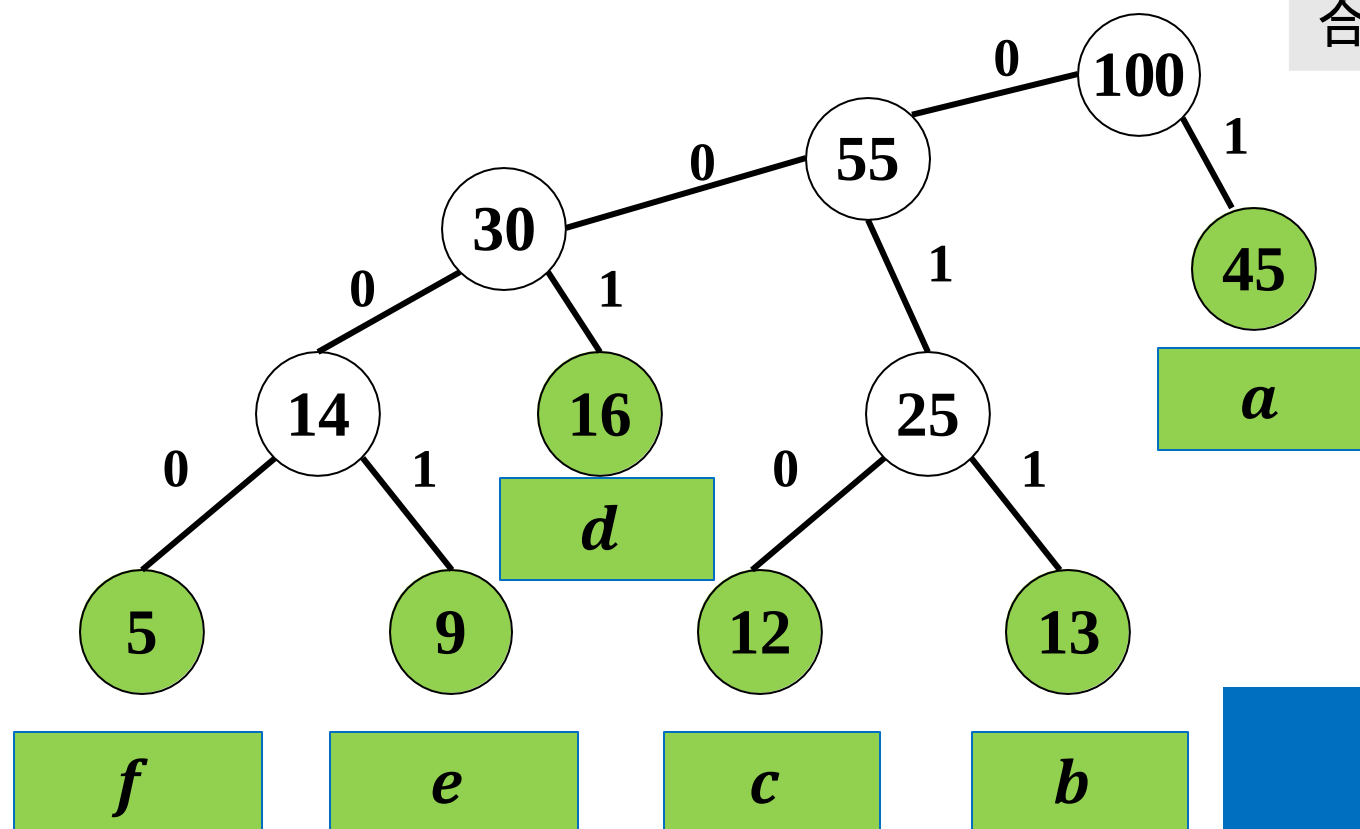
- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程

叶子频数						45
合并频数				55		

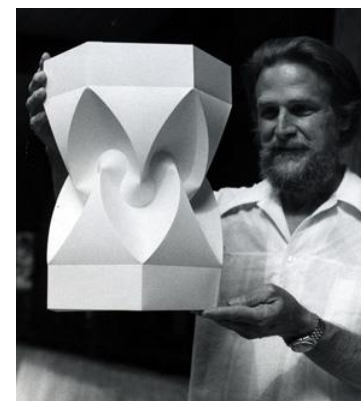


- 优先处理低频字符

- 将字符频数从大到小排序 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 选择两个最小频数 f_1, f_2 , 合并为 $f' = f_1 + f_2$
- 在 $F' = \langle f', f_3, \dots, f_n \rangle$ 重复选择并合并过程



叶子频数						
合并频数					100	

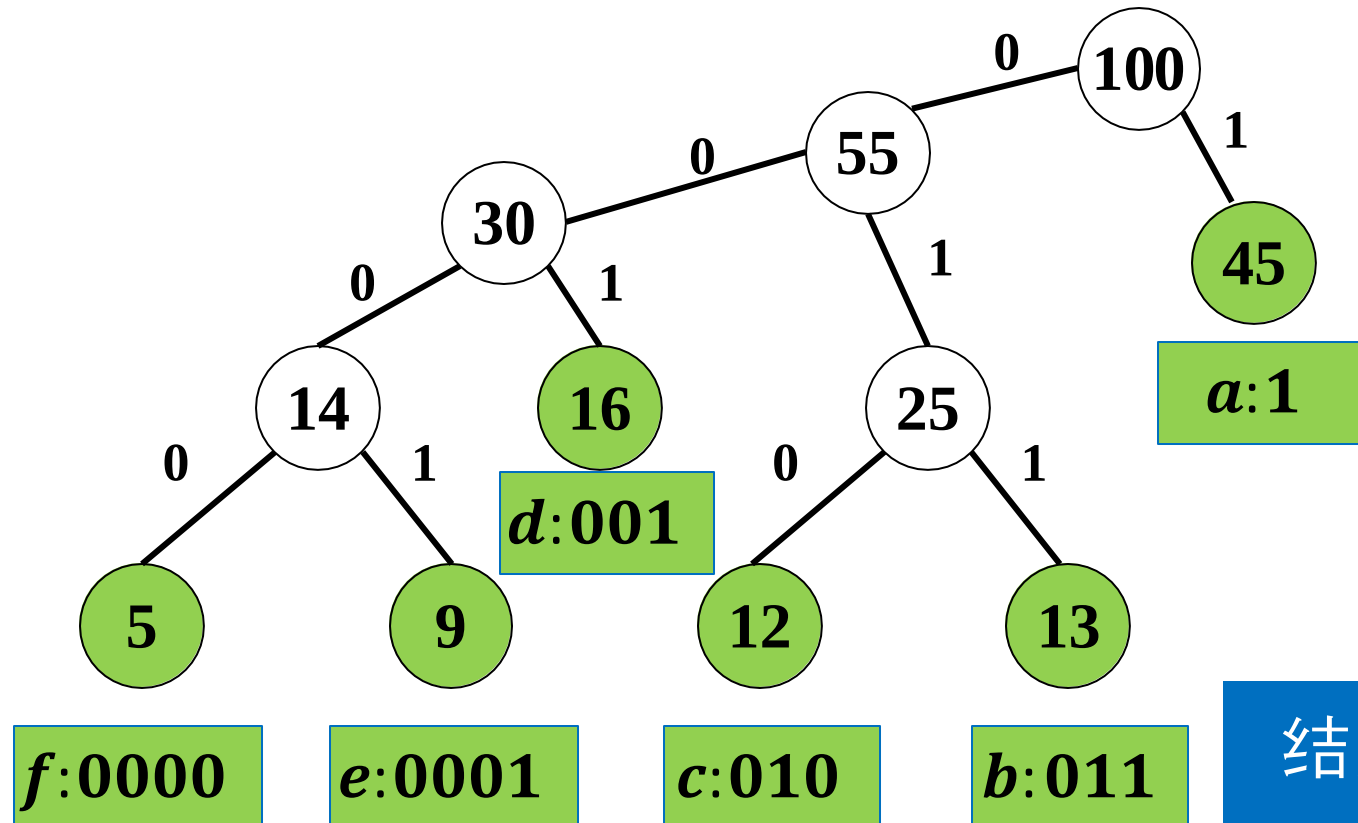


David A. Huffman
1925-1999

霍夫曼编码

- 优先处理低频字符

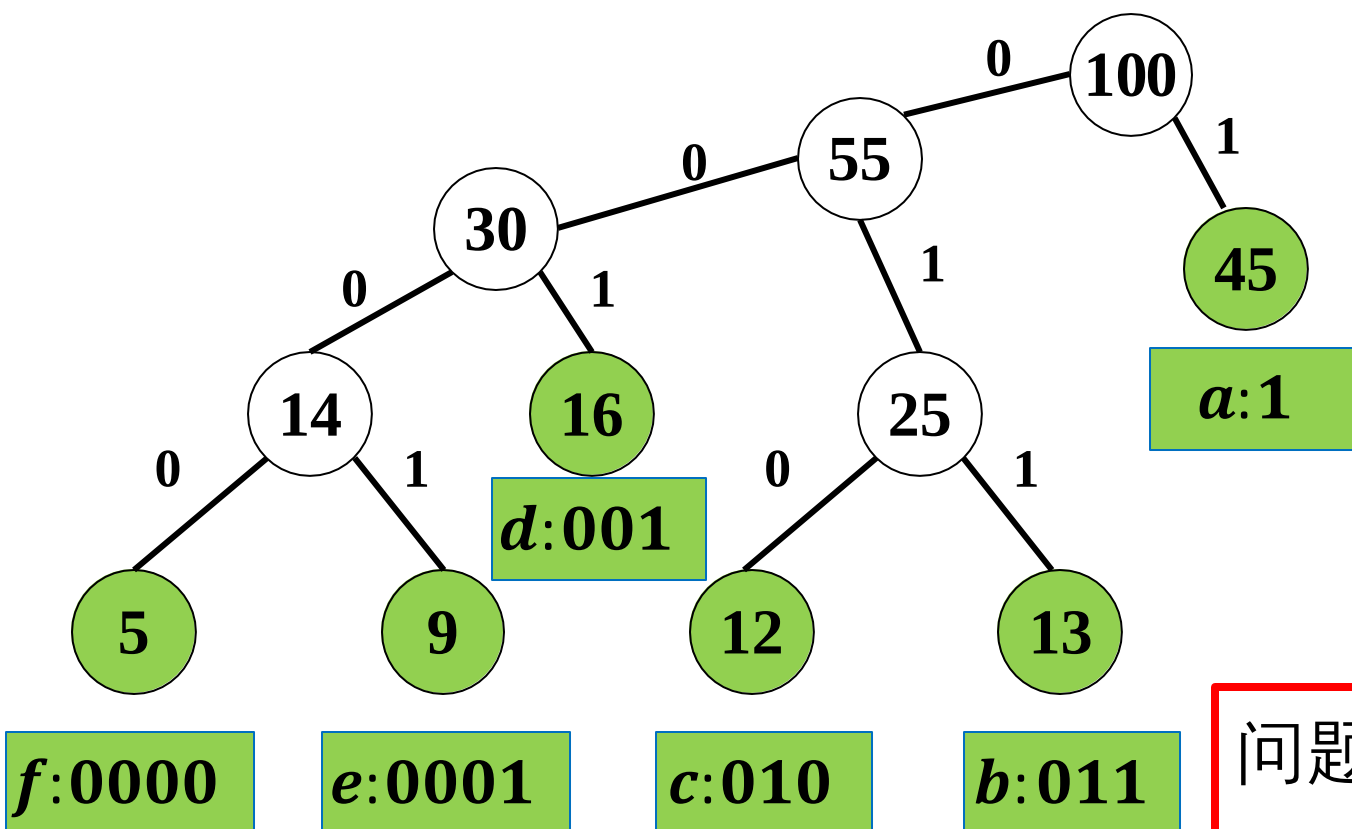
- $45 + 55 + 25 + 30 + 12 + 13 + 14 + 16 + 9 + 5 = 224$ (千次)



结点的孩子可为子树或叶子

- 优先处理低频字符

- $45 + 55 + 25 + 30 + 12 + 13 + 14 + 16 + 9 + 5 = 224$ (千次)



问题：是否为最优编码方案？

提出贪心策略

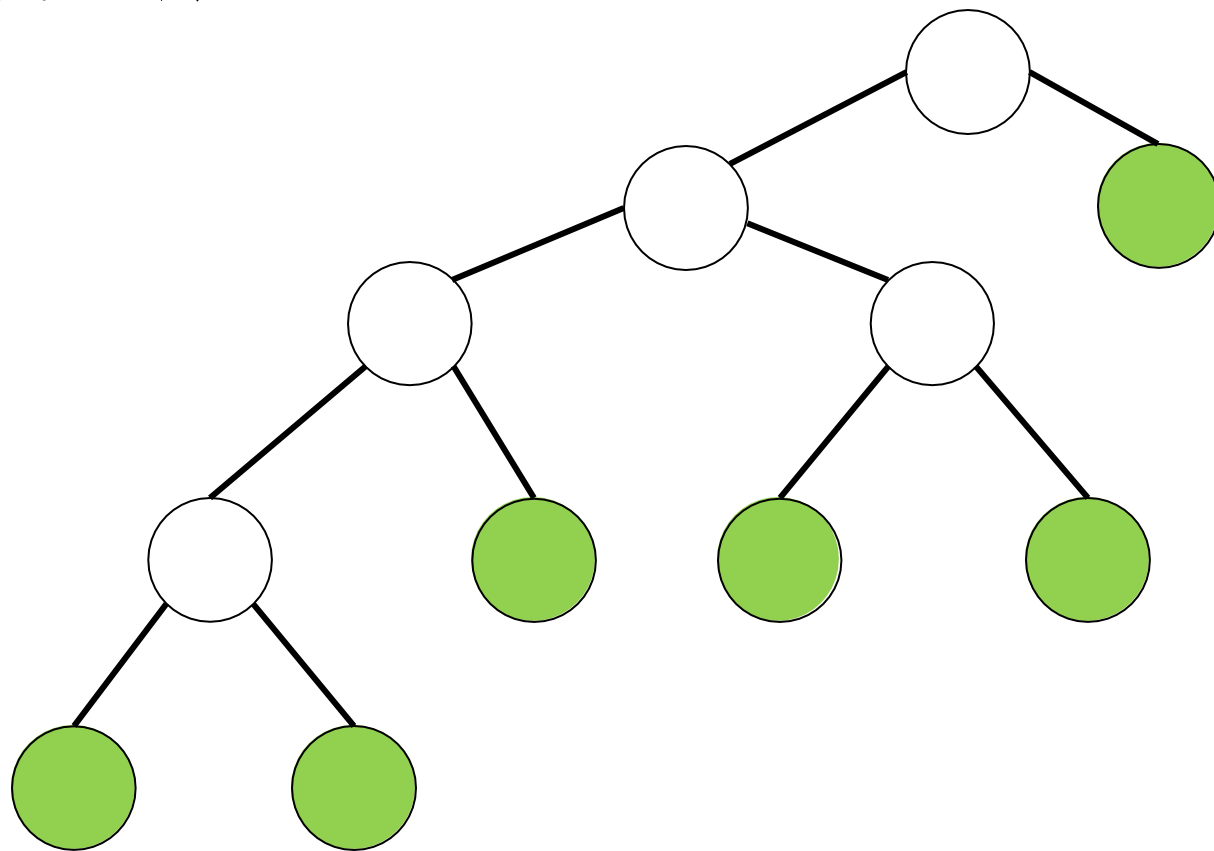


证明策略正确

观察问题特征，构造贪心选择

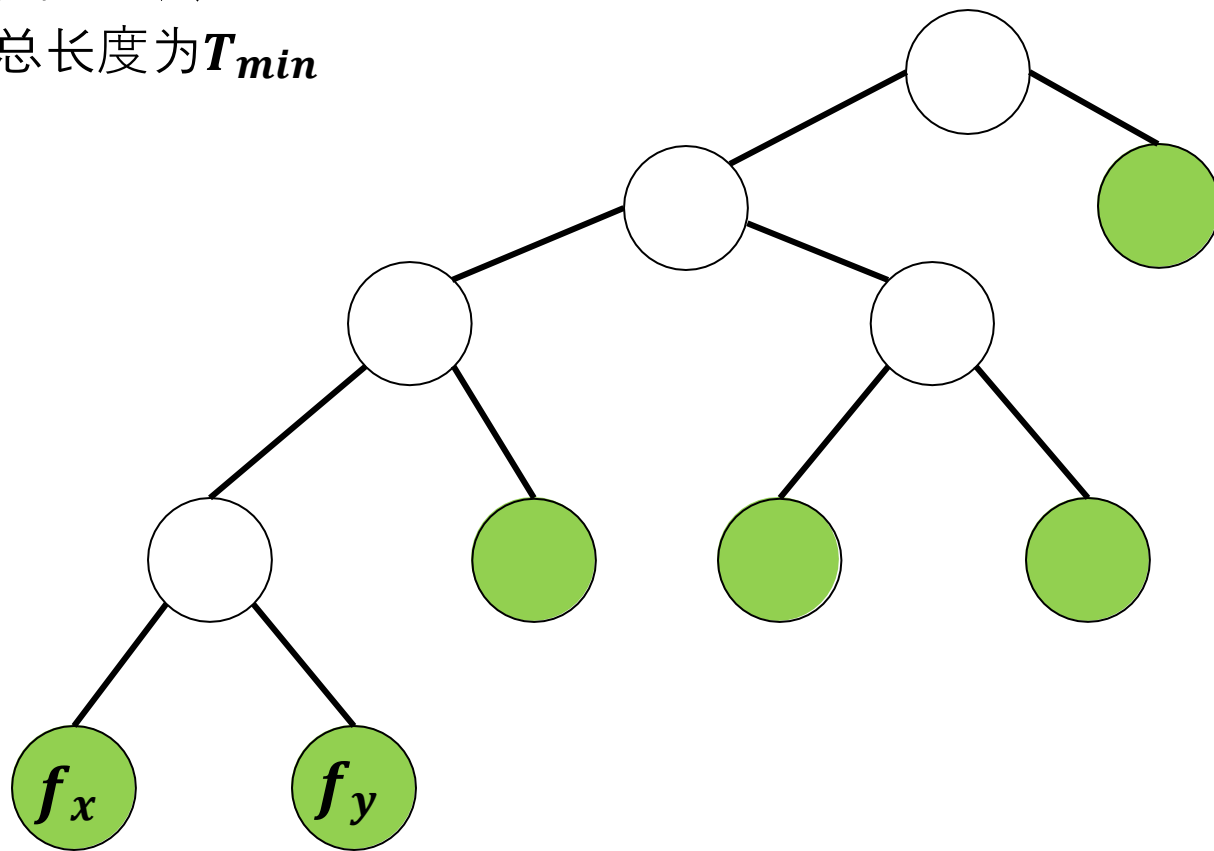
假设最优方案，通过替换证明

- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 证明: 存在最优方案, 使得 f_1, f_2 在编码树最底层



正确性证明

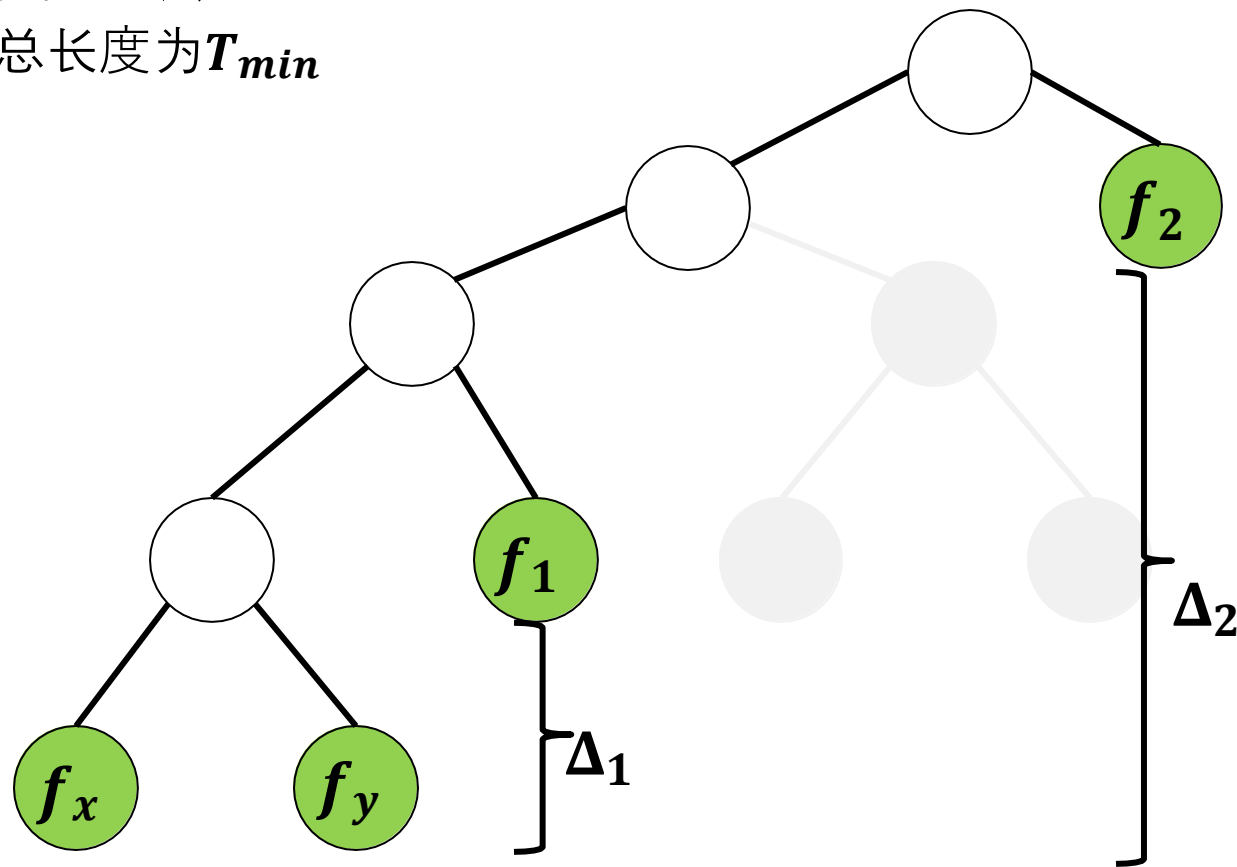
- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 证明: 存在最优方案, 使得 f_1, f_2 在编码树最底层
 - 最优方案最底层两结点是 f_x, f_y ($f_x \leq f_y$), 最小总长度为 T_{min}



最优编码树 T_{min}

正确性证明

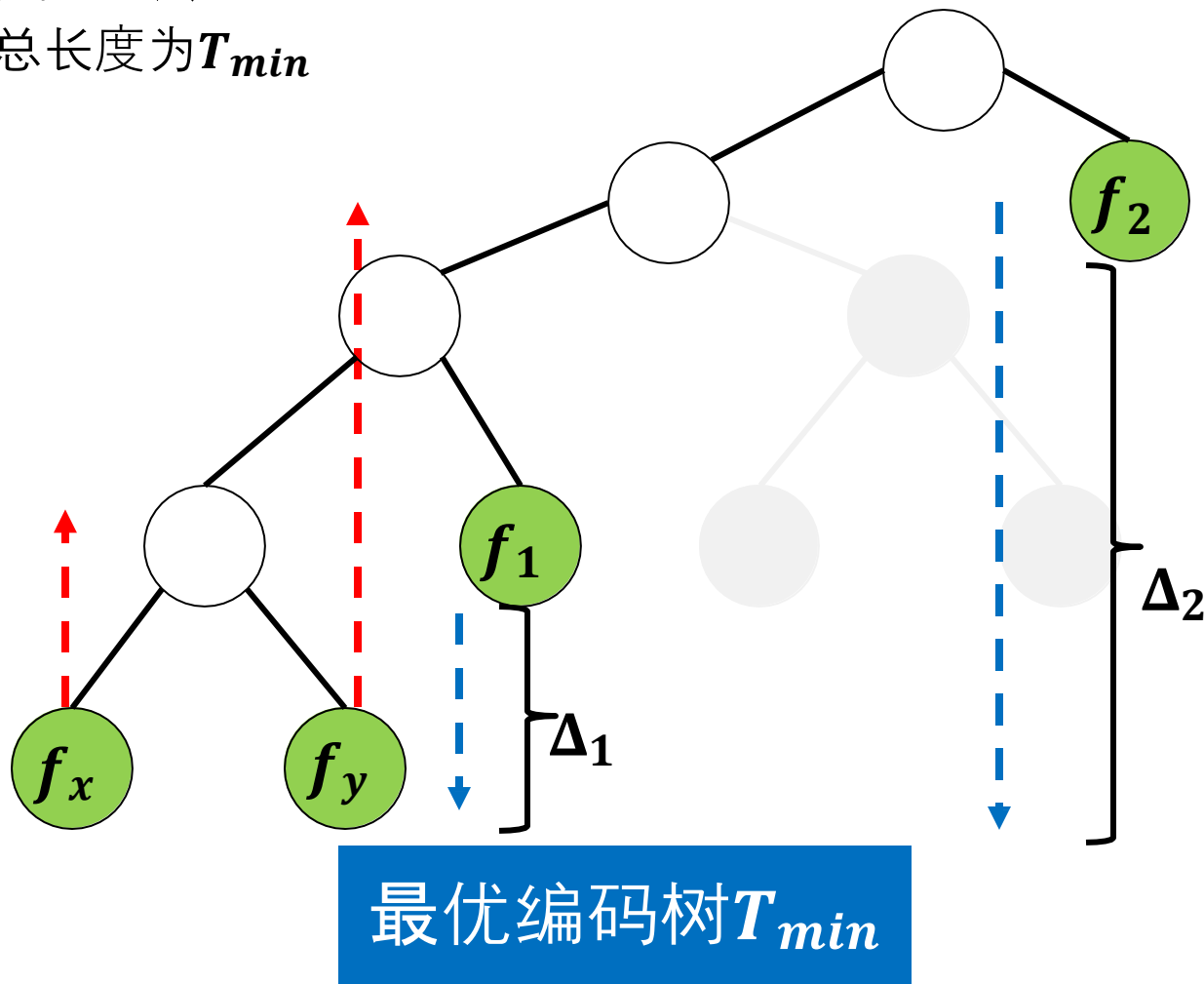
- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 证明: 存在最优方案, 使得 f_1, f_2 在编码树最底层
 - 最优方案最底层两结点是 f_x, f_y ($f_x \leq f_y$), 最小总长度为 T_{min}
 - 根据原始条件有
 - $f_1 \leq f_x, f_2 \leq f_y$
 - $\Delta_1 = d_T(f_x) - d_T(f_1) \geq 0$
 - $\Delta_2 = d_T(f_y) - d_T(f_2) \geq 0$



最优编码树 T_{min}

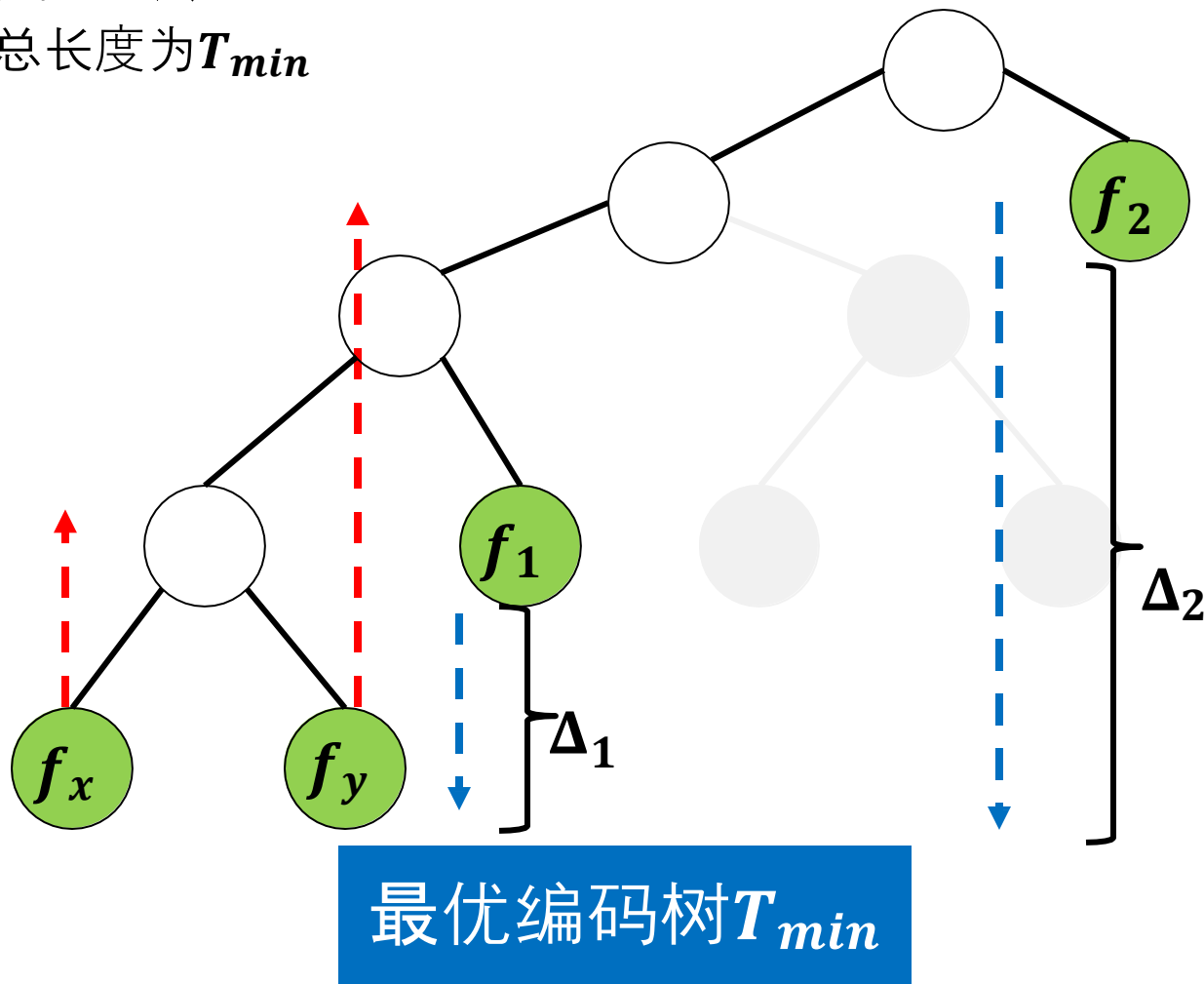
正确性证明

- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 证明: 存在最优方案, 使得 f_1, f_2 在编码树最底层
 - 最优方案最底层两结点是 f_x, f_y ($f_x \leq f_y$), 最小总长度为 T_{min}
 - 根据原始条件有
 - $f_1 \leq f_x, f_2 \leq f_y$
 - $\Delta_1 = d_T(f_x) - d_T(f_1) \geq 0$
 - $\Delta_2 = d_T(f_y) - d_T(f_2) \geq 0$
 - 交换 f_1 与 f_x , f_2 与 f_y 可得
 - $T = T_{min} - \Delta_1 f_x - \Delta_2 f_y + \Delta_1 f_1 + \Delta_2 f_2$



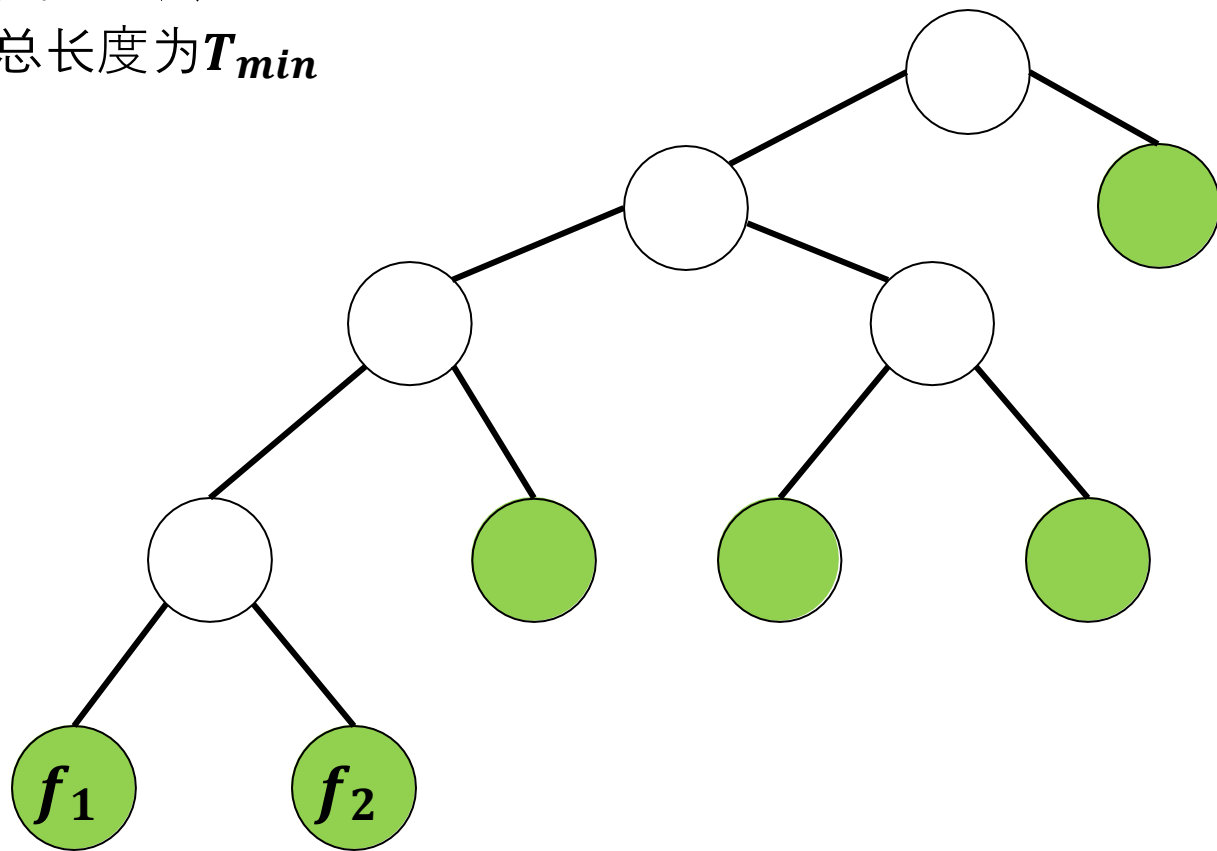
正确性证明

- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 证明: 存在最优方案, 使得 f_1, f_2 在编码树最底层
 - 最优方案最底层两结点是 f_x, f_y ($f_x \leq f_y$), 最小总长度为 T_{min}
 - 根据原始条件有
 - $f_1 \leq f_x, f_2 \leq f_y$
 - $\Delta_1 = d_T(f_x) - d_T(f_1) \geq 0$
 - $\Delta_2 = d_T(f_y) - d_T(f_2) \geq 0$
 - 交换 f_1 与 f_x , f_2 与 f_y 可得
 - $T = T_{min} - \Delta_1 f_x - \Delta_2 f_y + \Delta_1 f_1 + \Delta_2 f_2$
 - $= T_{min} - \Delta_1 (f_x - f_1) - \Delta_2 (f_y - f_2)$
 - $\leq T_{min}$



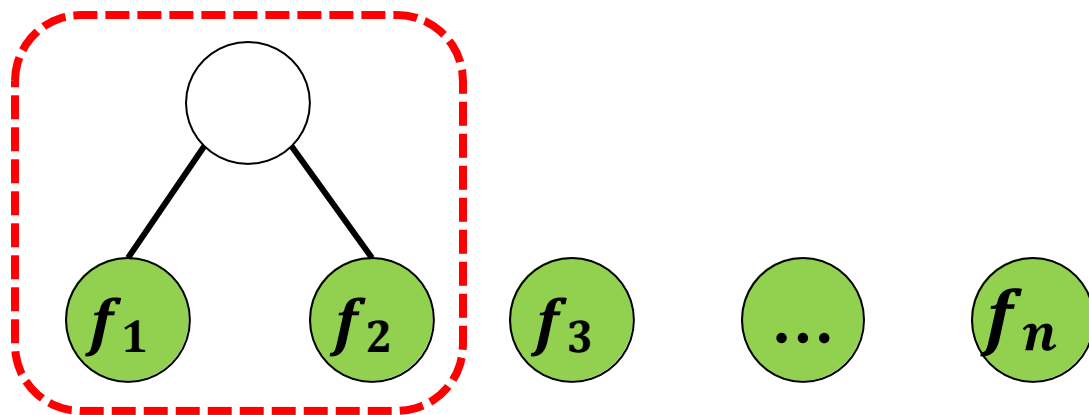
正确性证明

- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 证明: 存在最优方案, 使得 f_1, f_2 在编码树最底层
 - 最优方案最底层两结点是 f_x, f_y ($f_x \leq f_y$), 最小总长度为 T_{min}
 - 根据原始条件有
 - $f_1 \leq f_x, f_2 \leq f_y$
 - $\Delta_1 = d_T(f_x) - d_T(f_1) \geq 0$
 - $\Delta_2 = d_T(f_y) - d_T(f_2) \geq 0$
 - 交换 f_1 与 f_x , f_2 与 f_y 可得
 - $T = T_{min} - \Delta_1 f_x - \Delta_2 f_y + \Delta_1 f_1 + \Delta_2 f_2$
 - $= T_{min} - \Delta_1 (f_x - f_1) - \Delta_2 (f_y - f_2)$
 - $\leq T_{min}$
- 得证

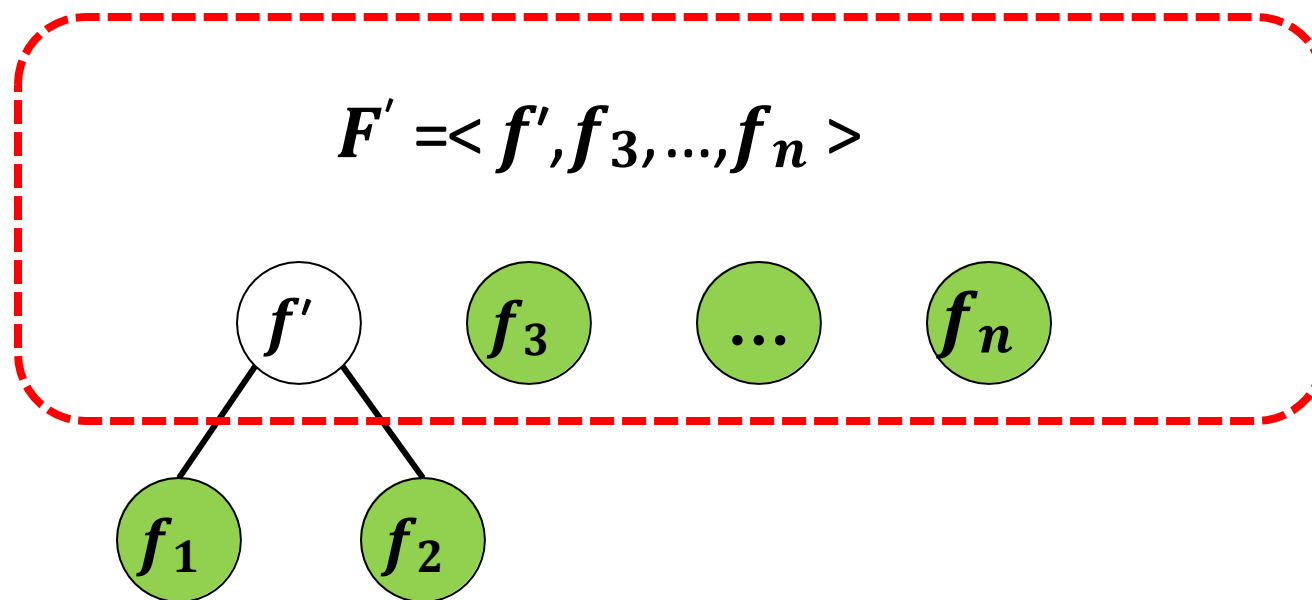


最优编码树 T_{min}

- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 存在最优方案, 使得 f_1, f_2 在编码树最底层



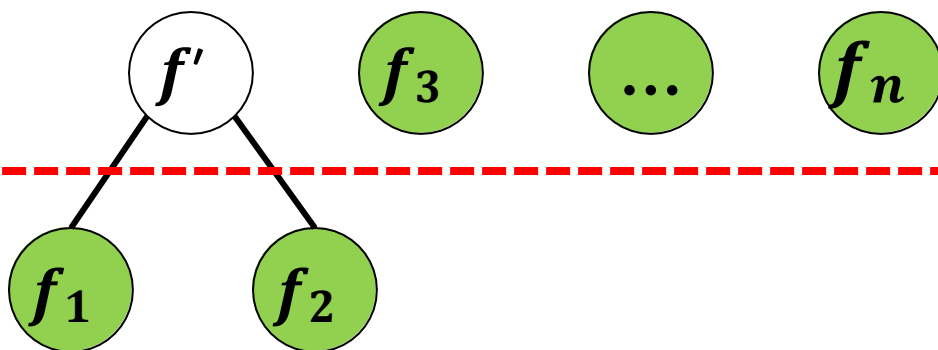
- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 存在最优方案, 使得 f_1, f_2 在编码树最底层
- $F' = \langle f', f_3, \dots, f_n \rangle$ ($f' = f_1 + f_2$) 中, 再寻最优方案



最优子结构性质

- 从小到大排序后, 频数 $F = \langle f_1, f_2, \dots, f_n \rangle$ ($f_1 \leq f_2 \leq \dots \leq f_n$)
- 存在最优方案, 使得 f_1, f_2 在编码树最底层
- $F' = \langle f', f_3, \dots, f_n \rangle$ ($f' = f_1 + f_2$) 中, 再寻最优方案

$F' = \langle f', f_3, \dots, f_n \rangle$ 中选择最小两频数



此过程即为霍夫曼编码, 贪心正确性得证

• Huffman(F, n)

输入: 字符数 n , 各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

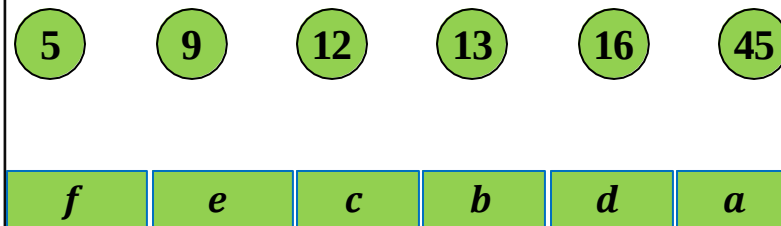
$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

字符频数按从小到大排序



• Huffman(F, n)

输入: 字符数 n , 各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

初始化结点数组

叶子频数	5	9	12	13	16	45
合并频数						



- Huffman(F, n)

输入: 字符数 n , 各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

共需合并 $n-1$ 次

• Huffman(F, n)

输入: 字符数 n ,各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

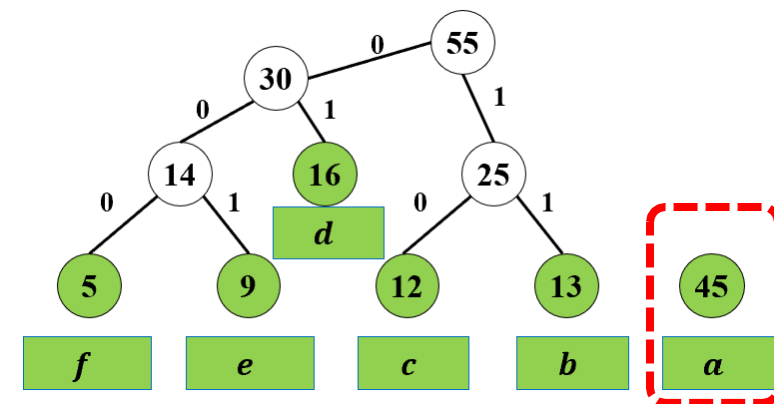
return $\text{ExtractMin}(P, Q)$

叶子频数

45

合并频数

55



选择频数最小的结点

• Huffman(F, n)

输入: 字符数 n ,各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

~~$x \leftarrow \text{ExtractMin}(P, Q)$~~

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

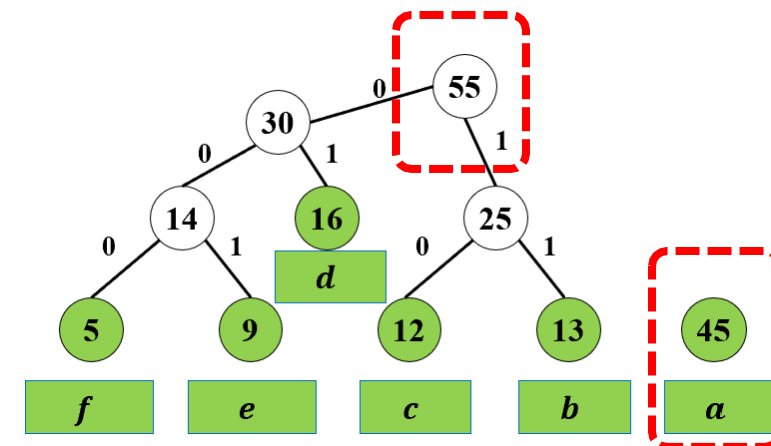
end

return $\text{ExtractMin}(P, Q)$

叶子频数

合并频数

55



选择频数最小的结点

• Huffman(F, n)

输入: 字符数 n ,各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

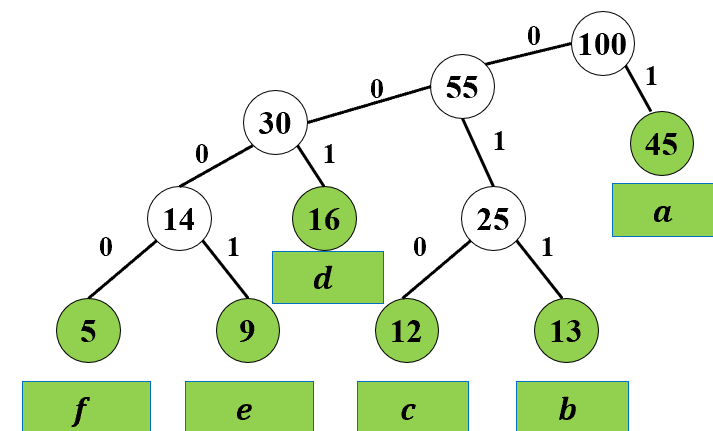
$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

叶子频数

合并频数



合并两个选择的结点

• Huffman(F, n)

输入: 字符数 n ,各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

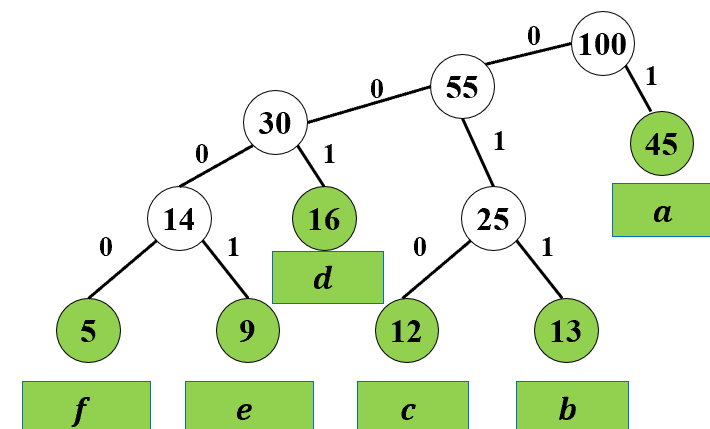
end

return $\text{ExtractMin}(P, Q)$

叶子频数

合并频数

100



存储合并后的结点

- Huffman(F, n)

输入: 字符数 n , 各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序 ----- $O(n \log n)$

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

- Huffman(F, n)

输入: 字符数 n , 各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序 -----

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

$O(n \log n)$

$O(n)$

• Huffman(F, n)

输入: 字符数 n ,各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序 -----

新建结点数数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

$O(n \log n)$

$O(n)$

$O(1)$

叶子频数				16	45
合并频数	14	25		100	

数组开头查找最小值 $O(1)$

- Huffman(F, n)

输入: 字符数 n , 各字符频数 F

输出: 霍夫曼编码树

//预处理

将 F 递增排序 -----

新建结点数组 $P[1..n]$ 和 $Q[1..n]$

for $i \leftarrow 1$ to n do

$P[i].freq \leftarrow F[i]$

$P[i].left \leftarrow NULL$

$P[i].right \leftarrow NULL$

end

$Q \leftarrow \emptyset$

for $i \leftarrow 1$ to $n - 1$ do

 新建结点 z

$x \leftarrow \text{ExtractMin}(P, Q)$

$y \leftarrow \text{ExtractMin}(P, Q)$

$z.freq \leftarrow x.freq + y.freq$

$z.left \leftarrow x$

$z.right \leftarrow y$

$Q.Add(z)$

end

return $\text{ExtractMin}(P, Q)$

$O(n \log n)$

$O(n)$

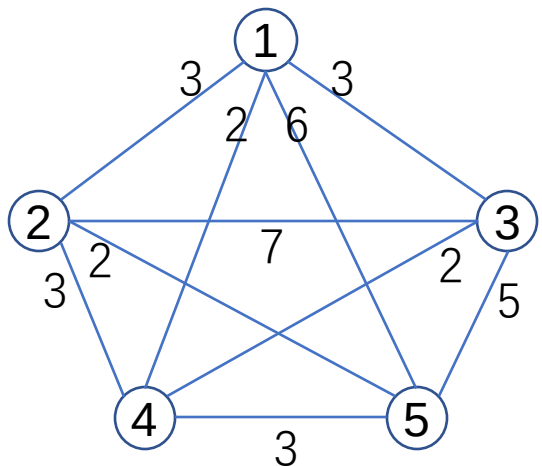
$O(n)$

时间复杂度 : $O(n \log n)$

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 最优前缀码问题
- 5.5 货郎担问题
- 5.6 小结

- 货郎担问题也叫旅行商问题，即TSP问题 (Traveling Salesman Problem)，是数学领域中著名问题之一。
- **问题描述**：某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员从某一城市出发并选择旅行路线，使每个城市经过一次，最后回到原出发城市，而总路程最短。

- 根据目标函数，选择距离当前城市最短距离的城市
 - 从某一个城市开始，每次选择一个新城市，直到所有的城市被走完。
 - 在选择下一个城市时，只考虑当前情况，保证当前选择的城市距离最小且不形成回路。



	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

- 最优解：**算法的时间复杂度？**
- 基于贪心策略的近似解： n 条回路中的最小距离。复杂度？

从城市1出发

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

1->4->3->5->2->1, 总距离14

从城市2出发

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

2->5->4->1->3->2, 总距离17

- 5.1 背包问题
- 5.2 贪心法主要思想
- 5.3 最小生成树问题
 - Prim算法
 - Kruskal算法
- 5.3 带有期限的作业调度问题
- 5.4 最优前缀码问题
- 5.5 货郎担问题
- 5.6 小结

- 贪心法特点：
 - 多步判断，只顾眼前
 - 不考虑子问题的计算结果的优劣
 - 只考虑子问题当前决策的优劣
 - 有时只能获得局部最优解
 - 全局最优解需要经过正确性证明
- 对于许多NP难的组合优化问题，近似算法是一种比较可行的途径
 - 贪心法常用于这些近似算法的设计中。

- 一般方法
 - 掌握约束条件、可行解、最优解、目标函数的定义；理解量度标准的意义；掌握贪心方法适用的问题特点和求解思想等基本知识。能掌握贪心法求解问题的一般过程
- 最小生成树问题
 - Prim算法
 - Kruskal算法
- 带有期限的作业调度问题
- 最优前缀码问题
 - 霍夫曼编码算法
- 货郎担问题
 - 理解贪心法近似解和实际最优解间的差异

能够识别出适合贪心法的可计算性问题、独立设计算法和分析算法复杂度。